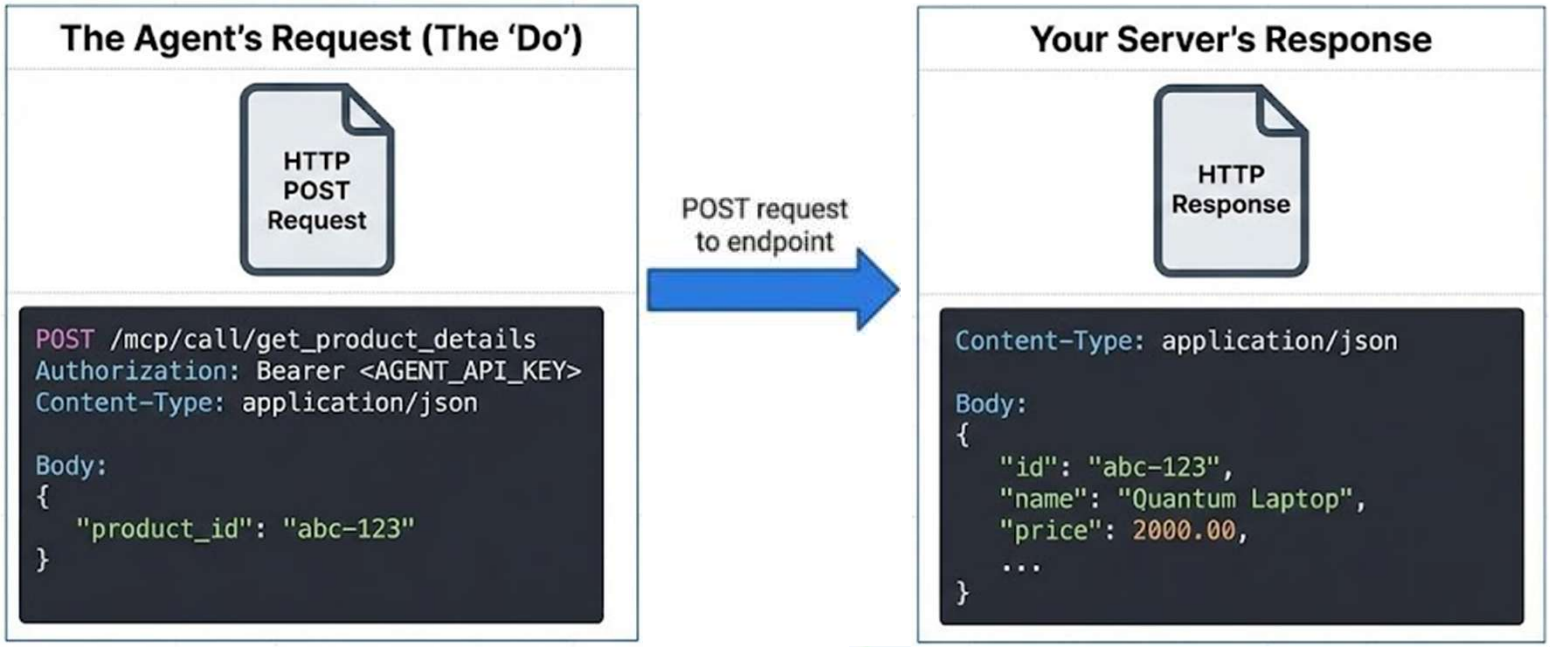
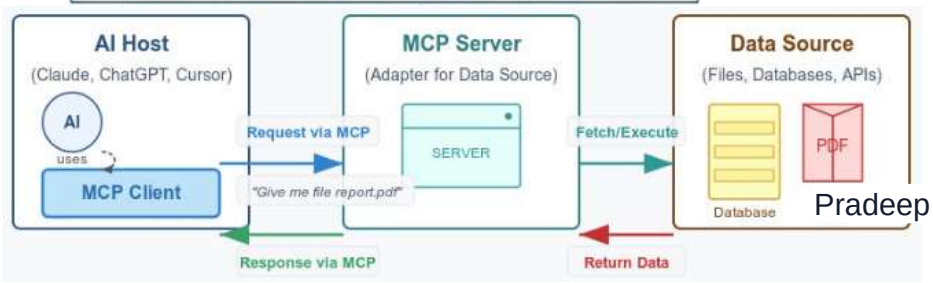
Core Concept 3: The call Endpoint Format

The manifest defined the "**HOW**" The call endpoint is the **"DO"**



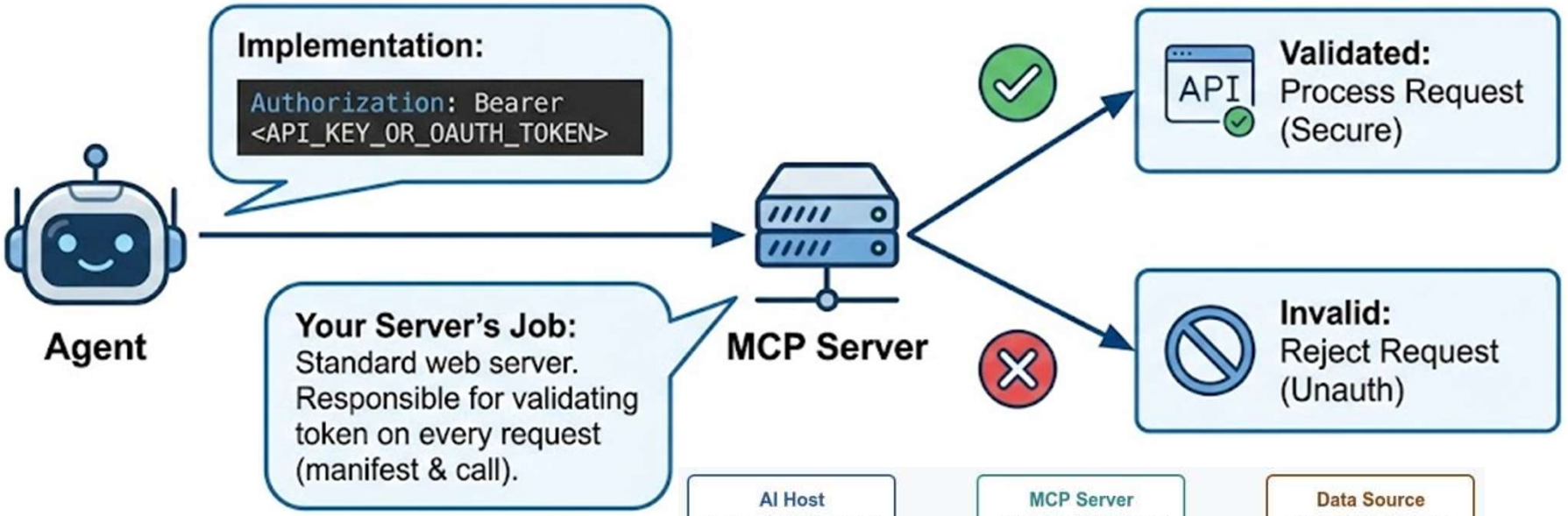
Key Principle: JSON in, JSON out
It's simple, standard, and stateless



Core Concept 4: Security & Authentication

Problem: The manifest and call endpoints are public. **How do we secure them?**

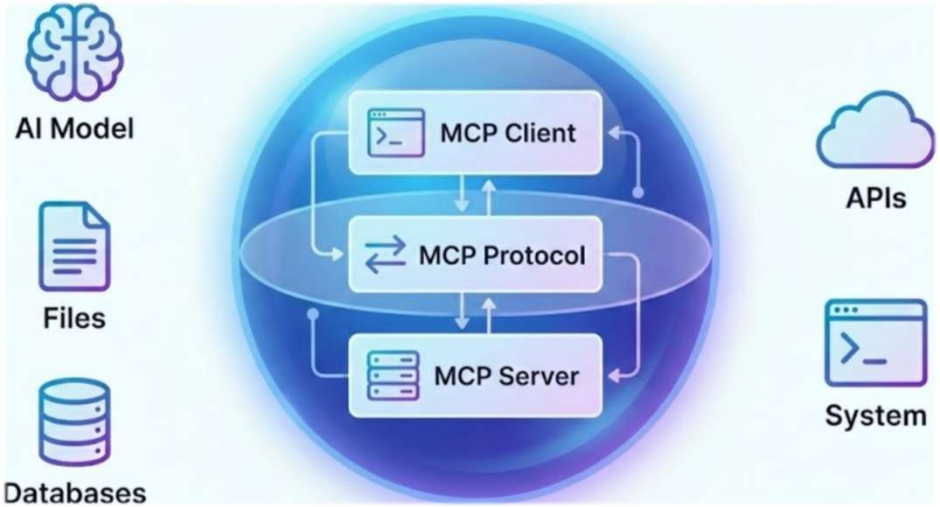
The MCP "Way": MCP leverages existing, proven web standards



Summary: The MCP "Spec" in 4 Parts

We have **mastered** the **4 core concepts** of the **MCP specification**:

1. **The /.mcp/ Path:** The discoverable front door
2. **The manifest.json:** The "User Manual" for the LLM (defines what and how)
3. **The call Endpoint:** The execution mechanism (JSON in, JSON out)
4. **Security:** Leverages standard Authorization headers

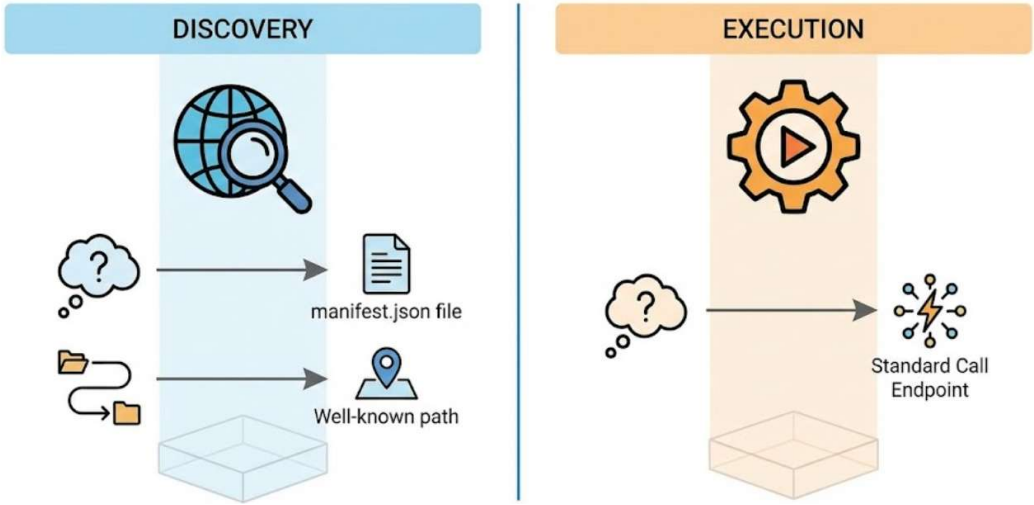


The MCP Lifecycle (How it Works)

We looked at the **definitions** of the **MCP**
We saw the **manifest.json** and the **API endpoints**

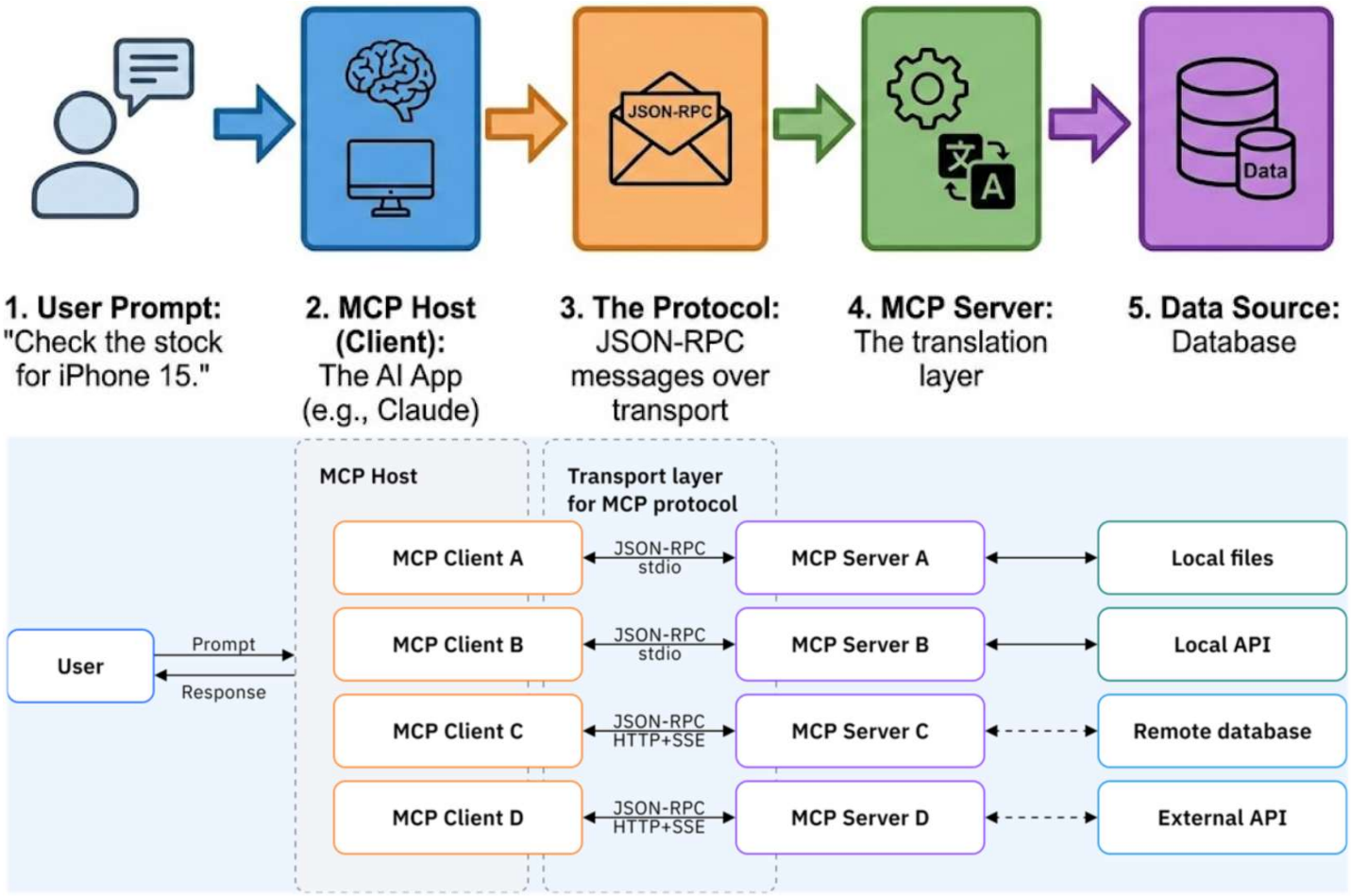
How does this **actually** look in a **live system**? **What is** the **sequence of events** that **happens** when an **AI Agent connects** to your **application**?

Walk through the **4-step lifecycle** that occurs every time an **agent interacts** with your **tools**

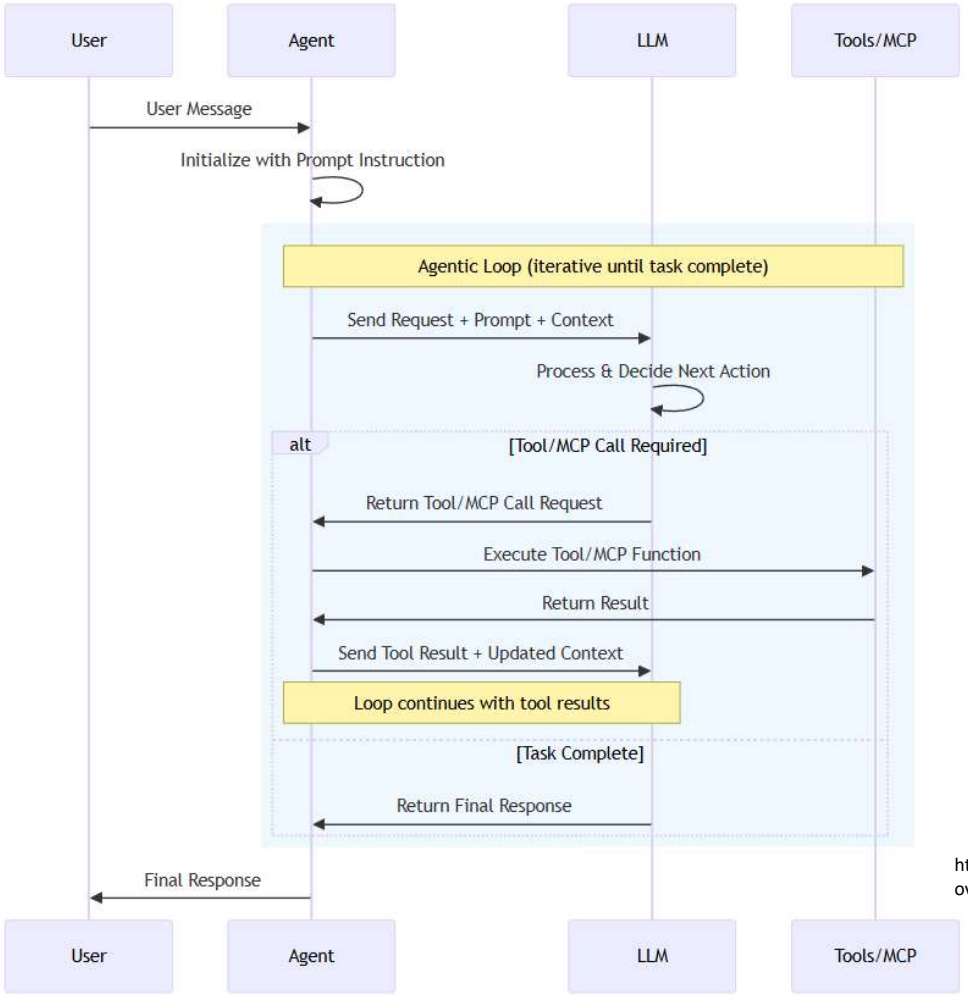




The E2E Flow: From User to Data



Designing Weather Agent – Sequencial Diagram



https://learn.microsoft.com/en-us/agent-framework/overview/agent-framework-overview?toc=%2Fdotnet%2Fai%2Ftoc.json&bc=%2Fdotnet%2Fai%2Ftoc.json

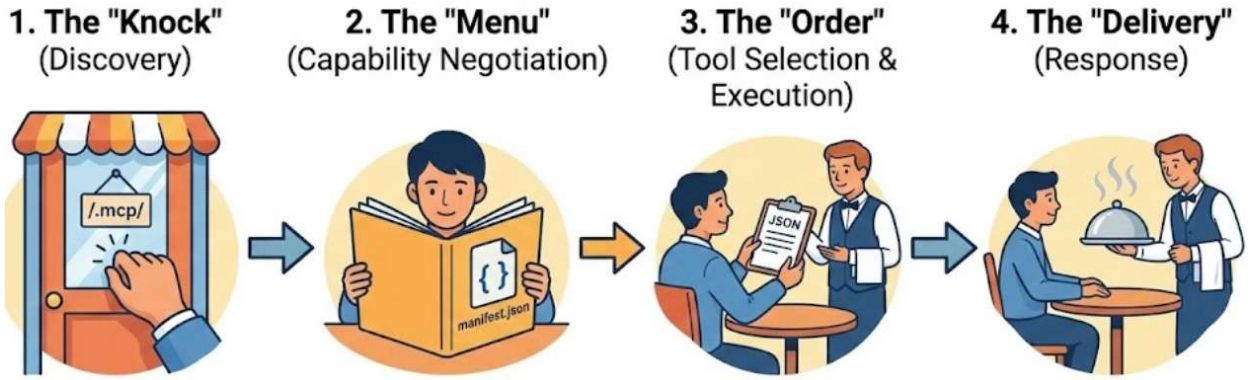
The MCP Lifecycle: A 4-Step Handshake

Step 1: The "Knock" (Discovery)
Agent checks the well-known path

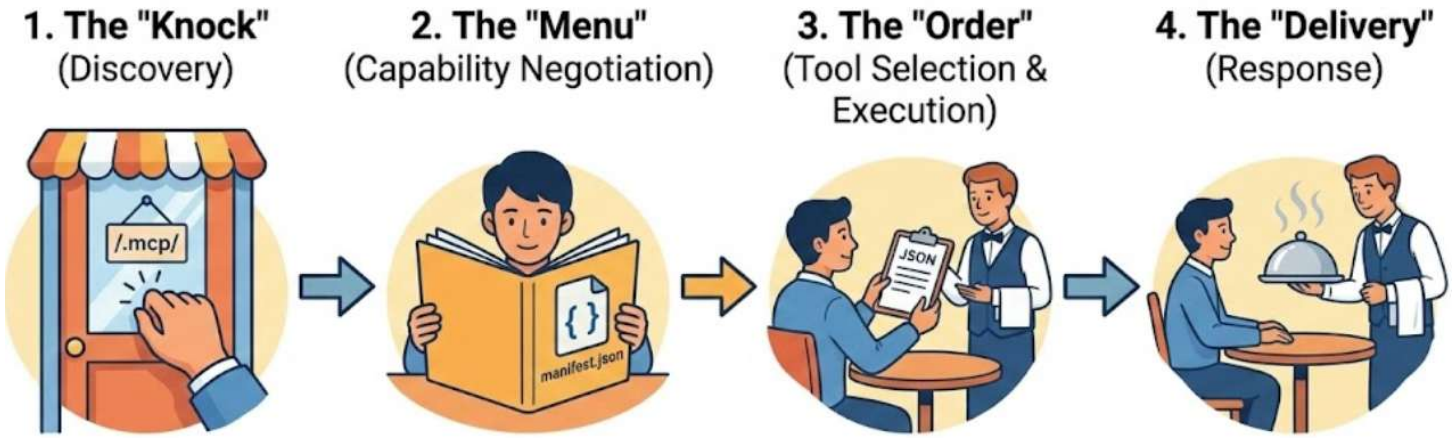
Step 2: The "Menu" (Capability Negotiation)
Agent reads and ingests the Manifest

Step 3: The "Order" (Tool Selection & Execution)
Agent selects a tool and sends a JSON payload

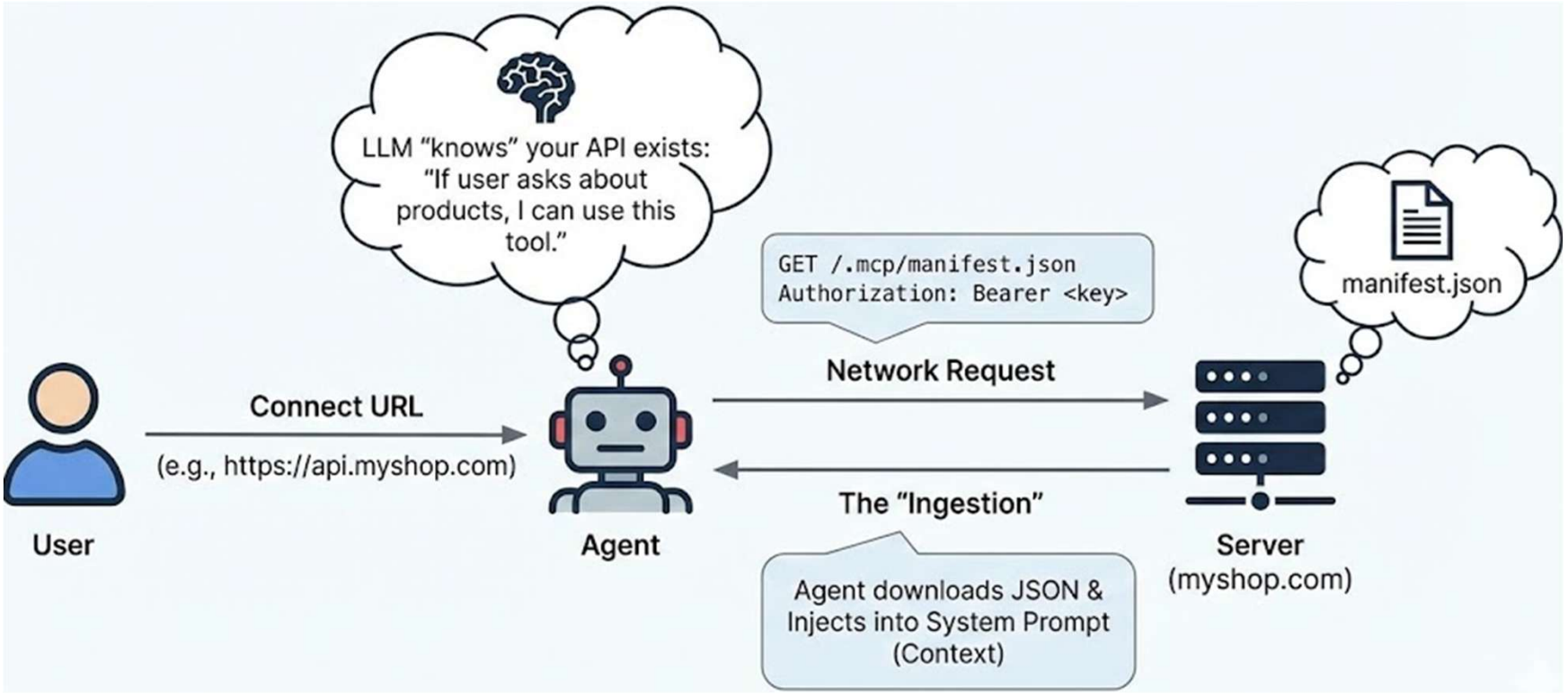
Step 4: The "Delivery" (Response)
Server executes logic and returns raw JSON



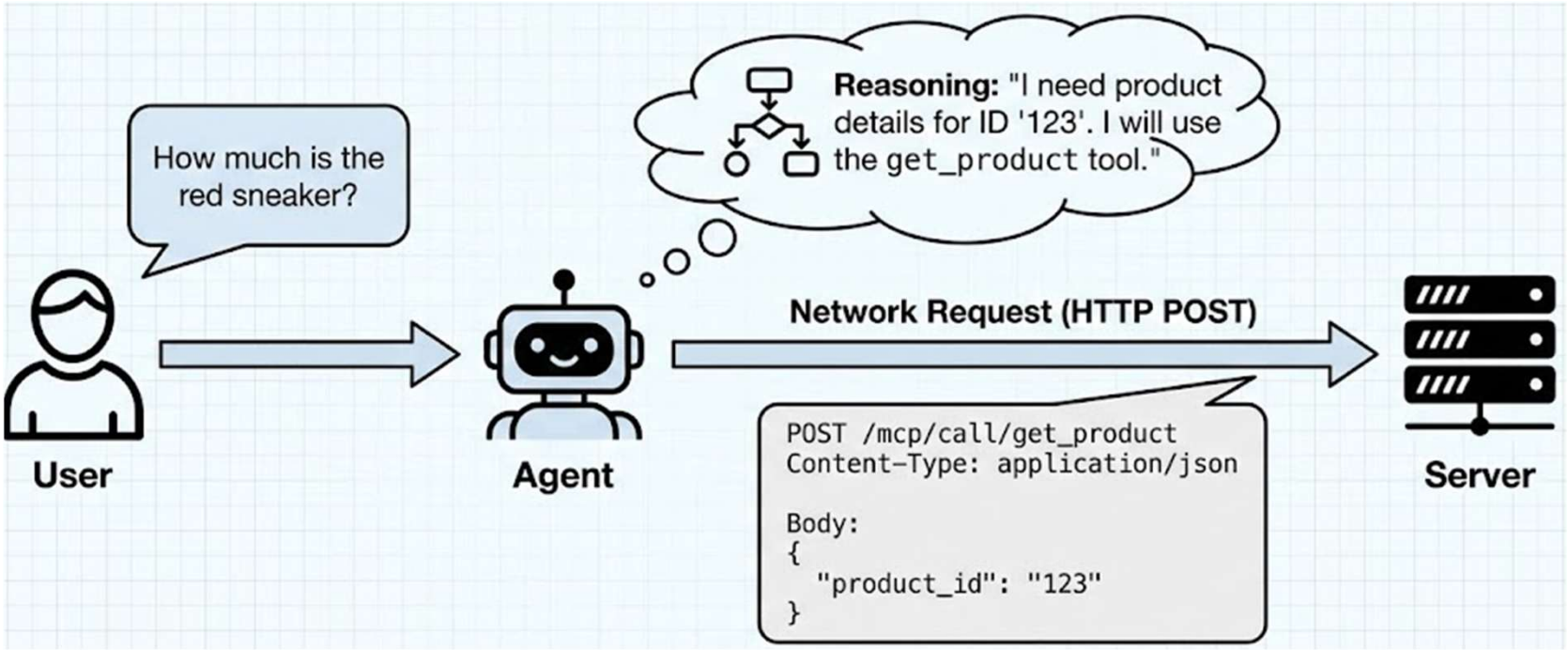
The MCP Lifecycle: A 4-Step Handshake



Step 1 & 2: Discovery & The Menu

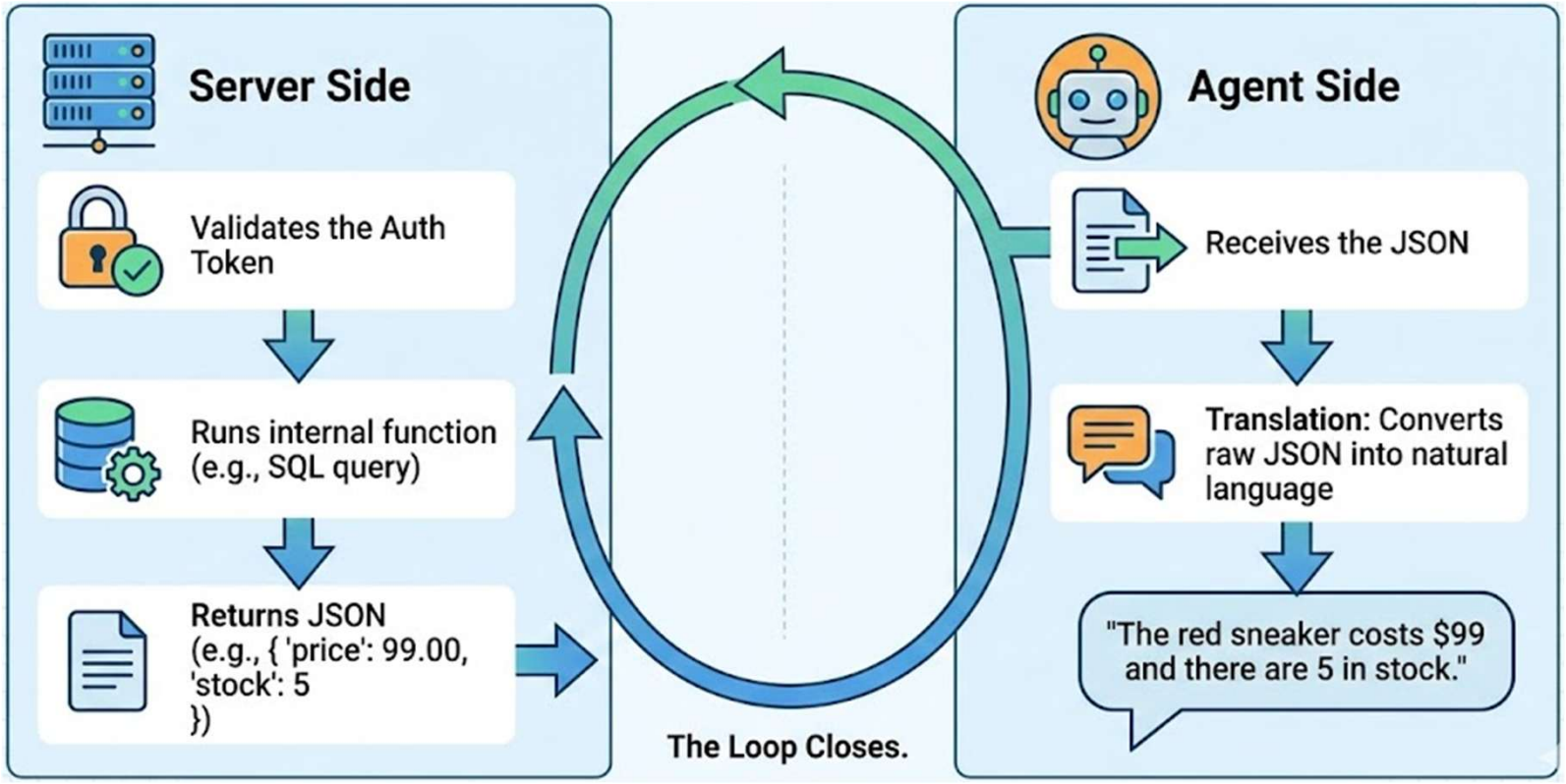


Step 3: The Order (Tool Execution)

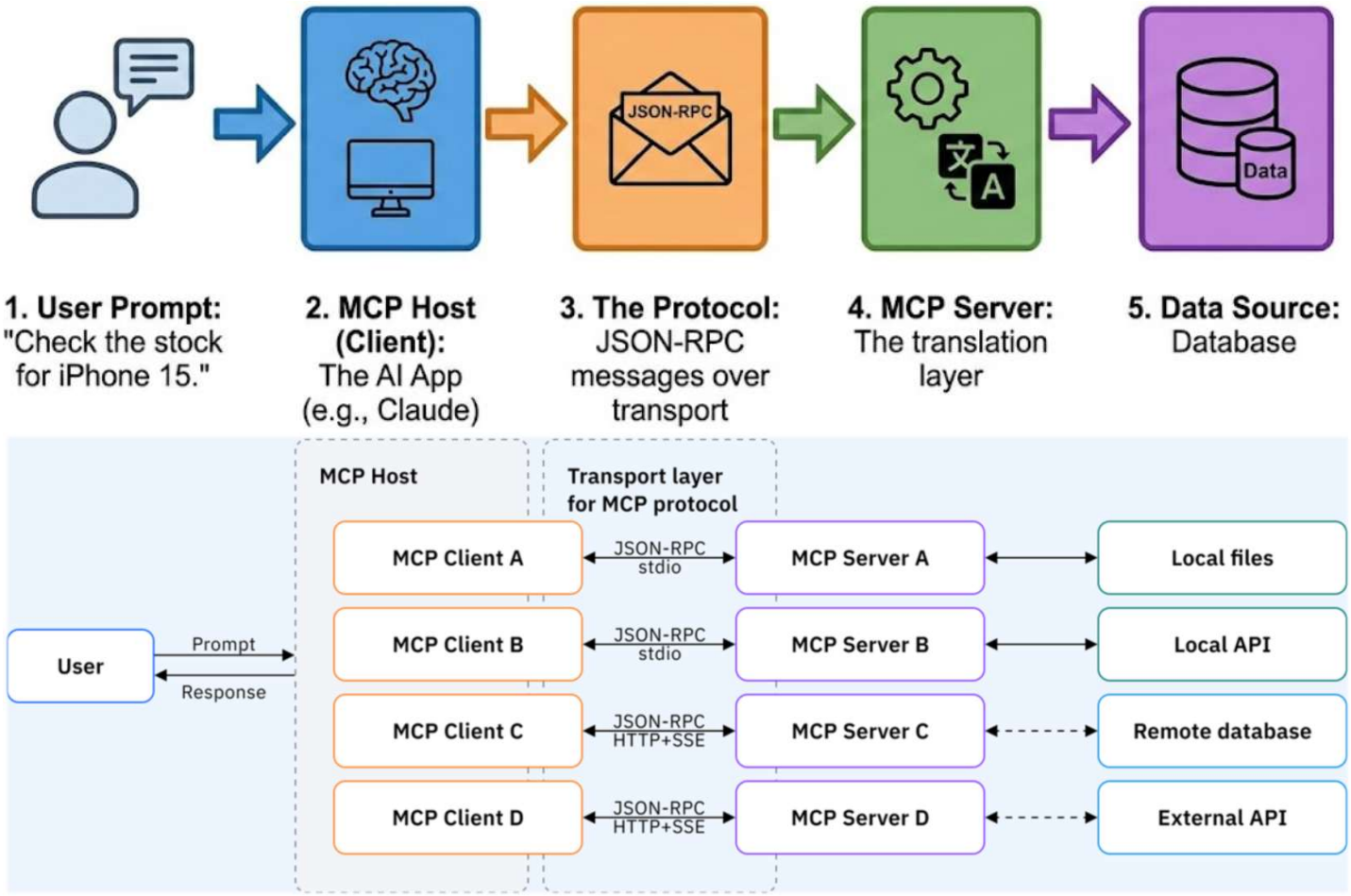


Key: The **Agent writes** the **JSON body automatically based on** your **schema**

Step 4: The Delivery (Response & Loop)



The E2E Flow: From User to Data

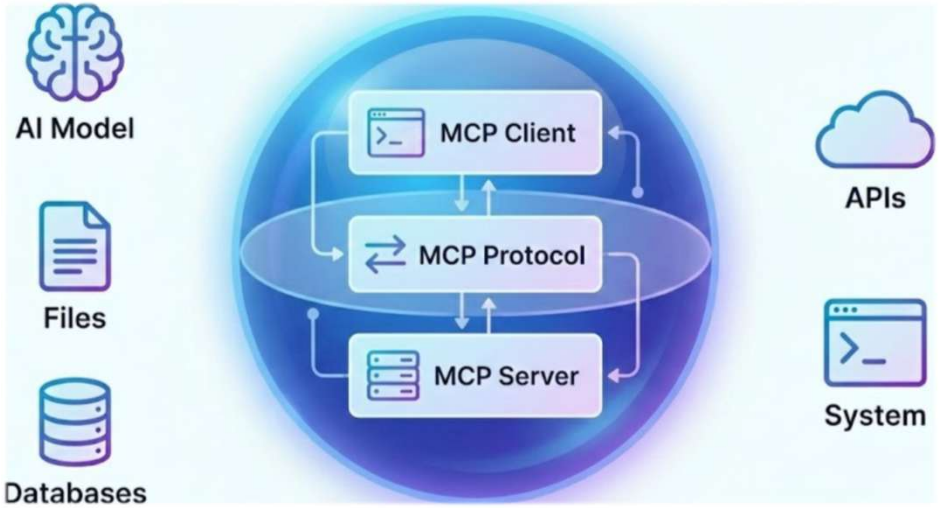


The Architect's View: MCP vs. OpenAPI (The "Why")

Defined the **manifest.json**, the **call endpoint** and **the /.mcp/ path**

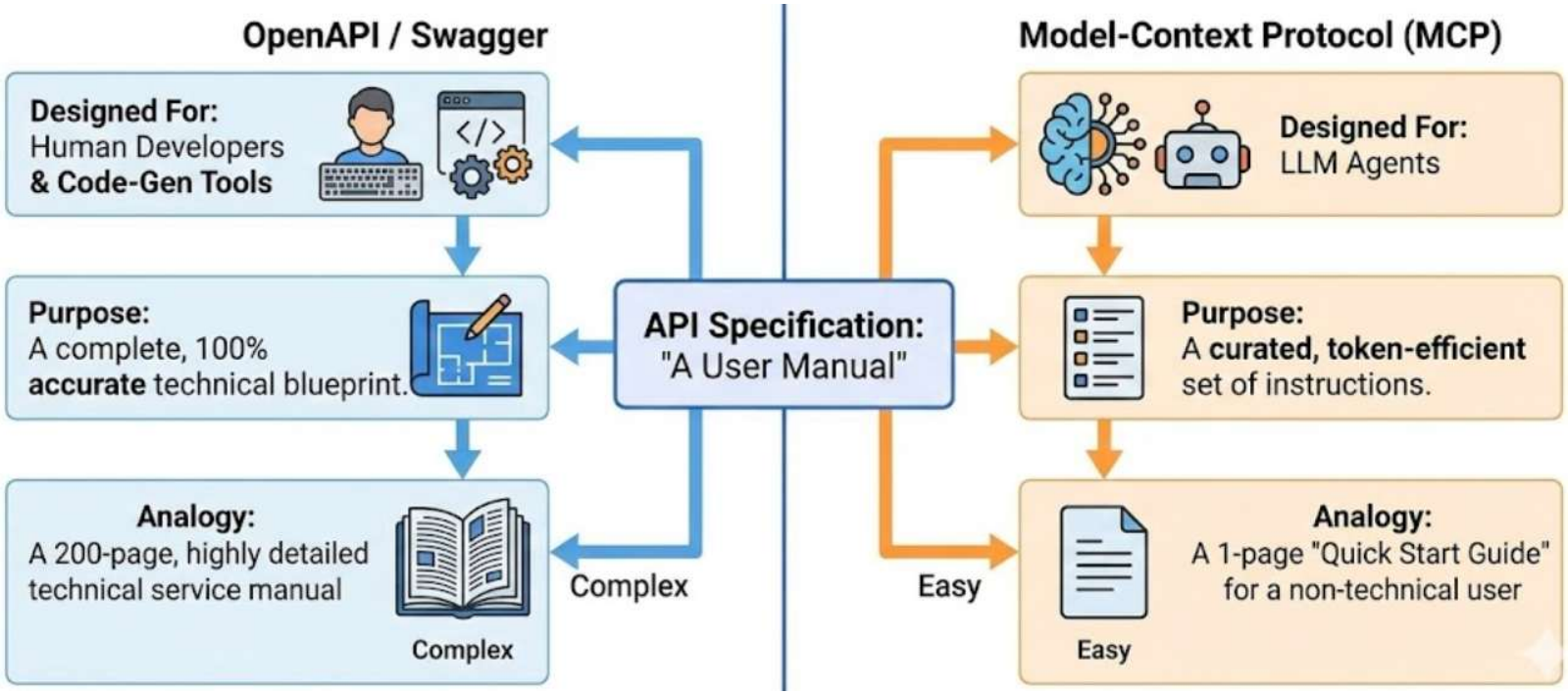
This is just a **simplified OpenAPI** or **Swagger file**. Already have an **OpenAPI spec** for **microservices**, **Why** would I **build** and **maintain** a **second**, **redundant spec?**

It is the **right question** for an **architect** to **ask and** the answer reveals the **entire philosophy** behind **enterprise-ready agentic architecture**



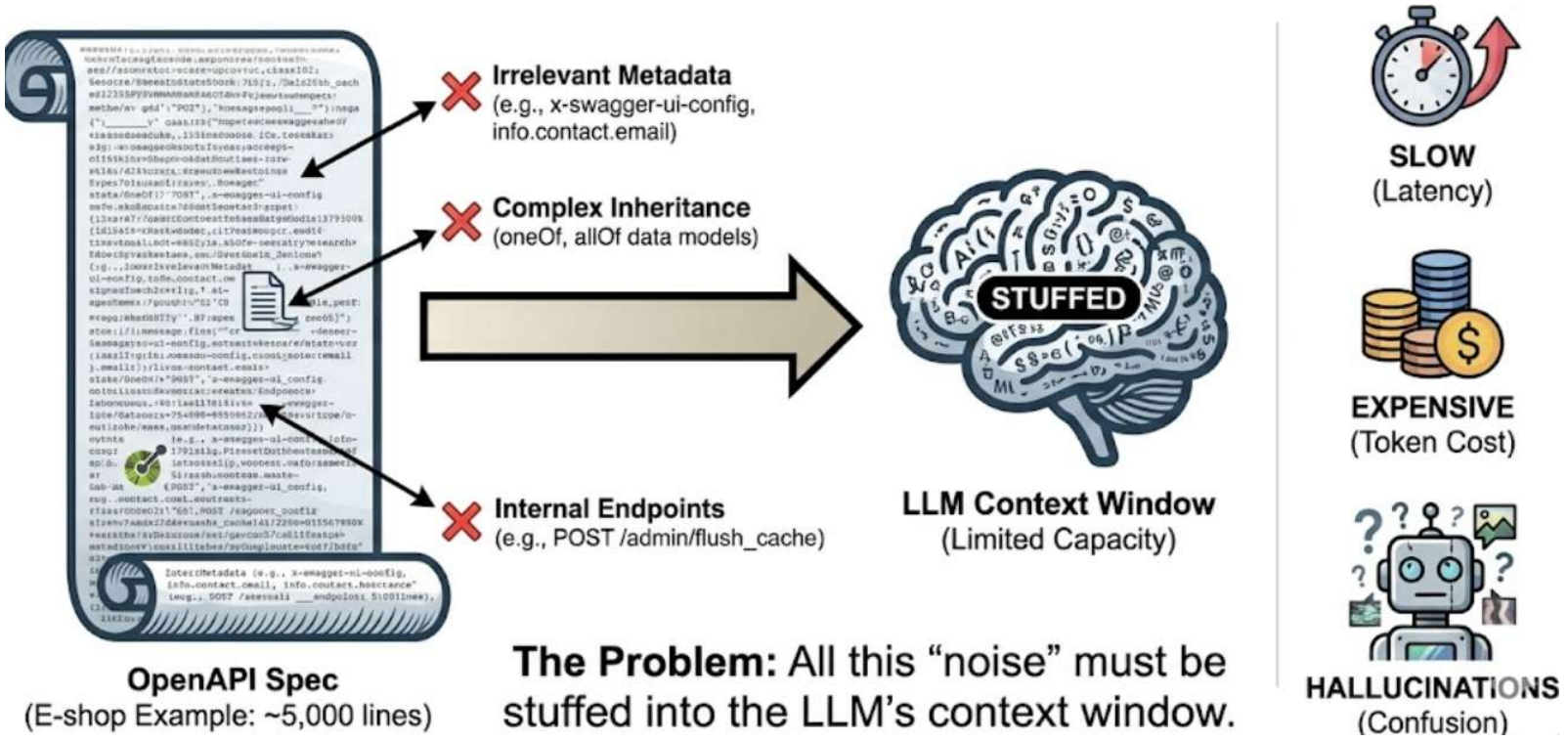
The Core Problem: Designed for Different Consumers

The **question** is: **Who is the user?**
An **API specification** is just a **"user manual"**



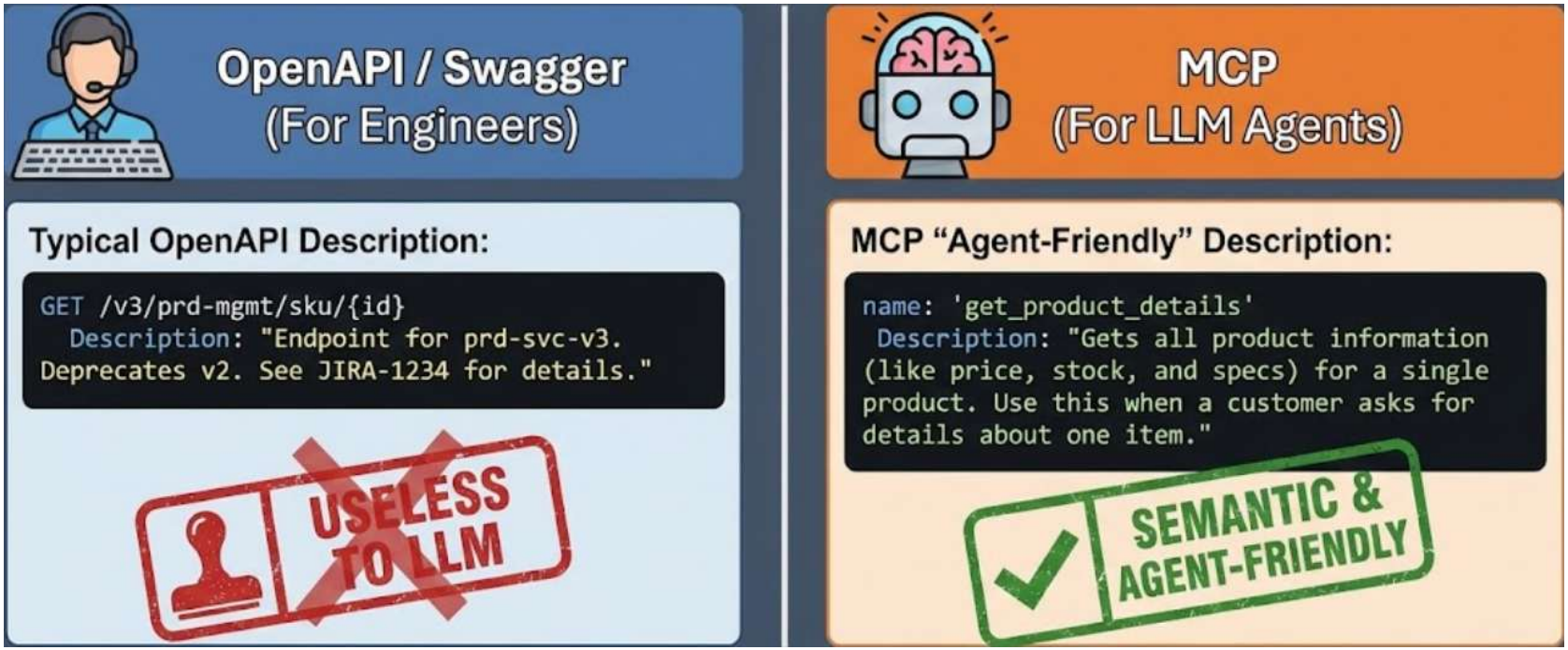
Problem 1: The "Noise" and Token Cost of OpenAPI

Your internal **OpenAPI** spec for an **E-shop** might be **5,000 lines** long



Problem 2: "Developer-Speak" vs. "LLM-Speak"

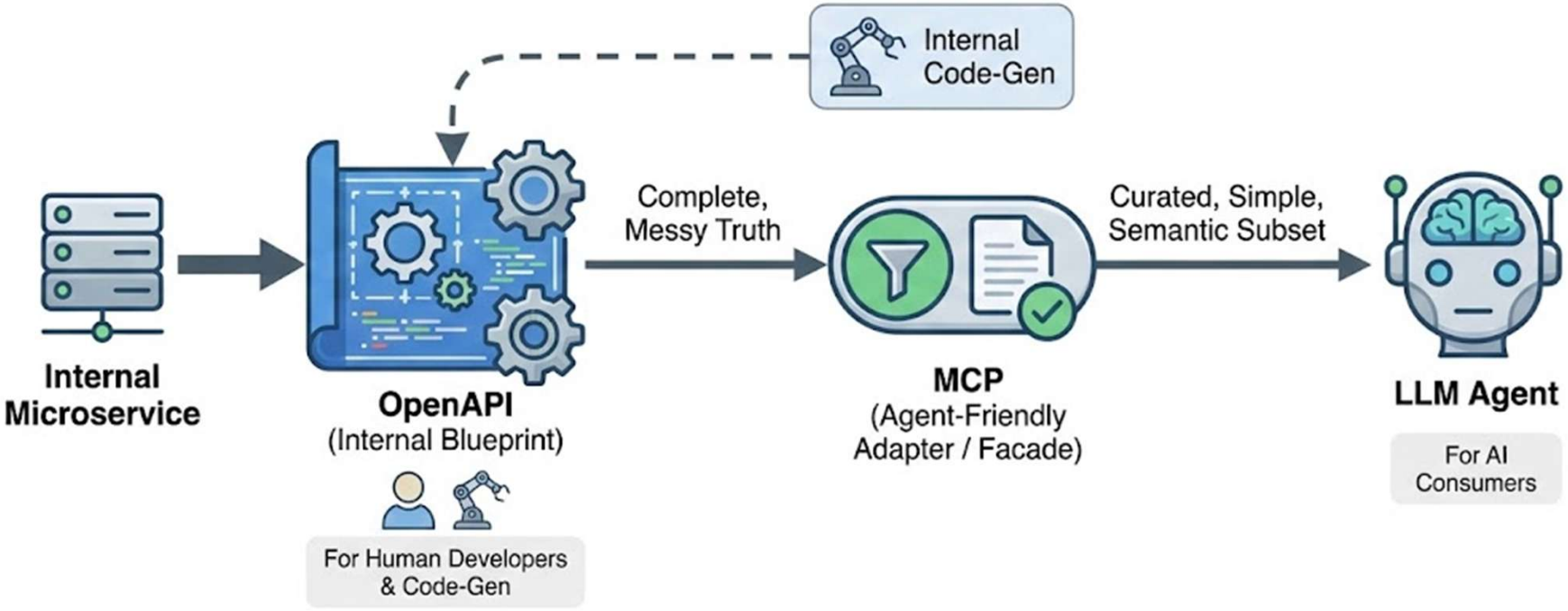
OpenAPI is written for engineers, not LLMs



MCP is a **semantic spec**, **not just a technical one**

The Solution: MCP as the "Agent-Friendly Adapter"

You **do not replace OpenAPI** with **MCP**
MCP is an "**Adapter**" or "**Facade**" Pattern

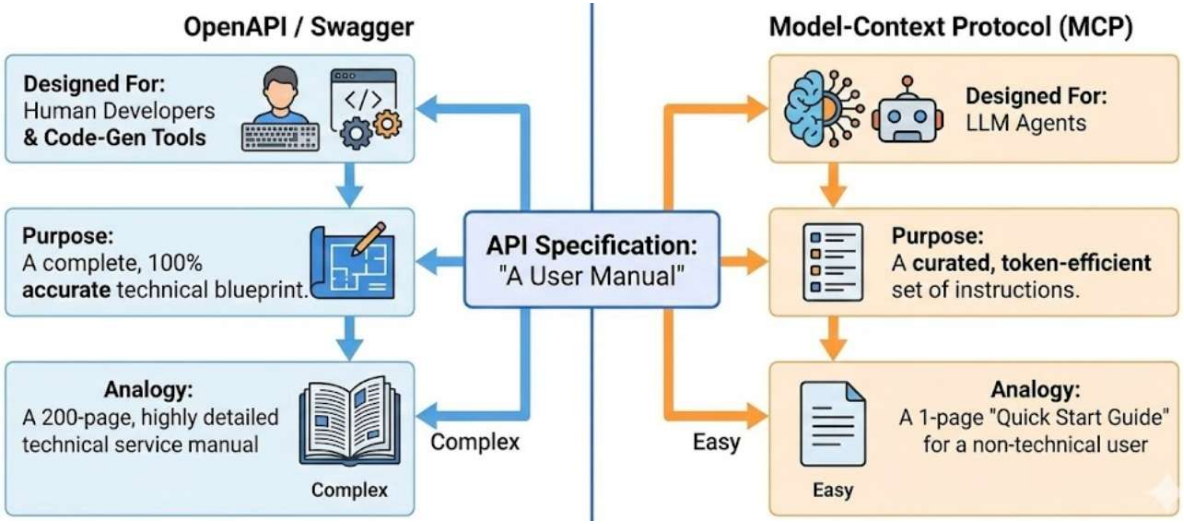


Summary: The Right Tool for the Right Consumer

OpenAPI: The Comprehensive Technical Blueprint. Designed for Humans

MCP: The Curated Semantic "Menu" - Designed for LLMs

Architect's Job: Use MCP to create a safe, efficient and intelligent "facade" in front of your complex internal systems

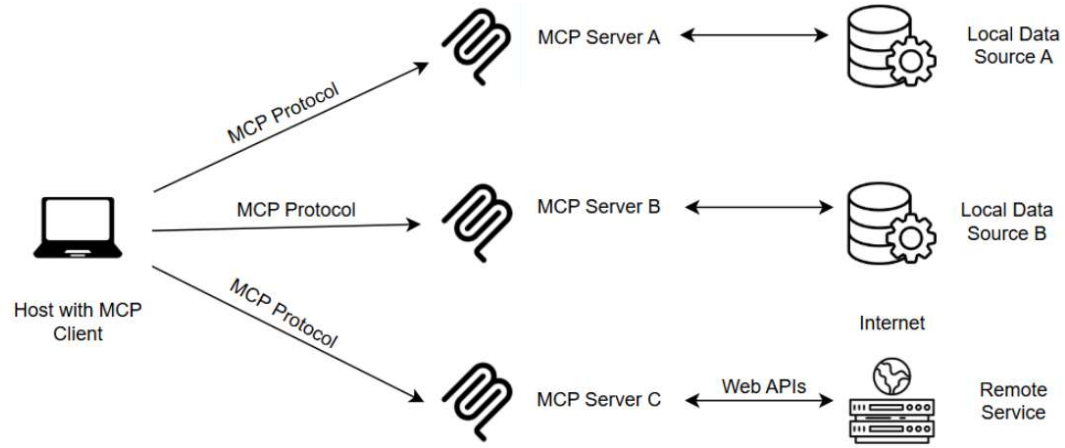


MCP Design Patterns & Case Studies

Defined the **MCP spec**, **justified** its **existence against OpenAPI** and **surveyed** the **tooling** that makes it practical

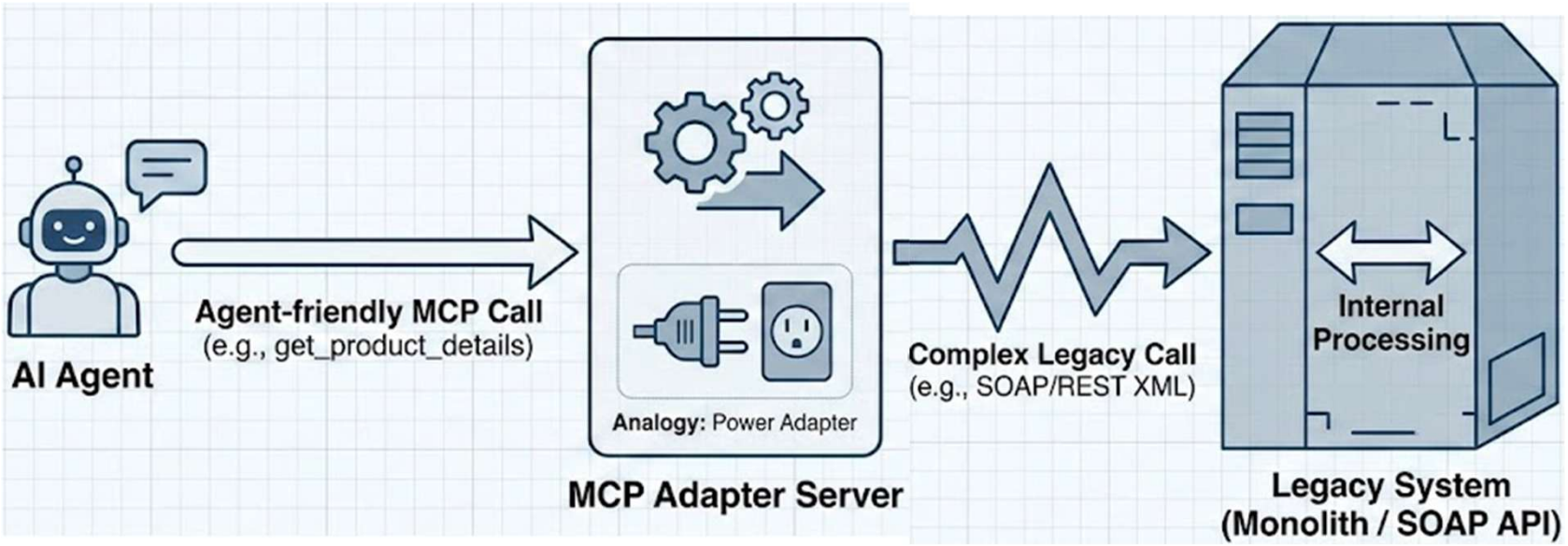
A **protocol** is **just a tool** and a tool is only as good as the **blueprints** we use it with. It's time to ask: **'How do we design with MCP?'**

Explore the **core**, **reusable design patterns** for **integrating MCP** into a **real-world enterprise environment**



Pattern 1: The "Adapter" Pattern (For Legacy Systems)

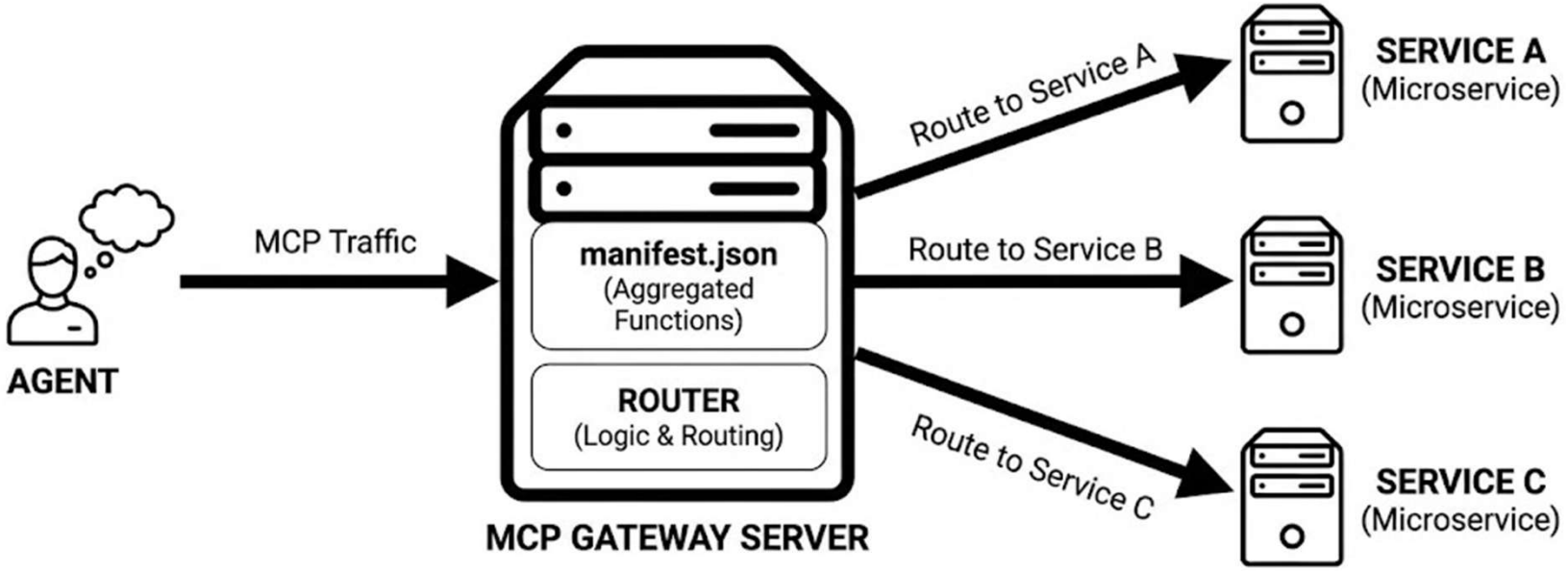
The Problem: You have a single, valuable, but **"non-agent-aware" system**





Pattern 2: The "Gateway" Pattern (For Microservices)

The Problem: You don't have one legacy system; you have fifty modern microservices and you don't want to expose fifty different MCP servers



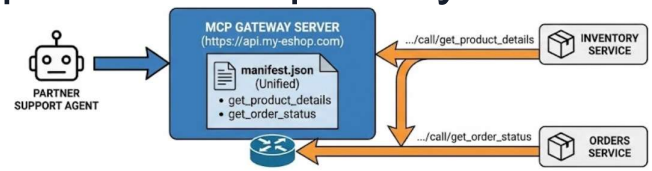
Benefit: A single, secure, and manageable entry point for all your agentic traffic

Case Study: Architecting Our E-Shop with MCP

Our System: A microservice architecture (Inventory, Orders, User Service)

Our Goal: Expose get_product_details (from Inventory) and get_order_status (from Orders) to a partner's Support Agent

The Wrong Way: Give the partner agent two different API specs and hope they integrate both



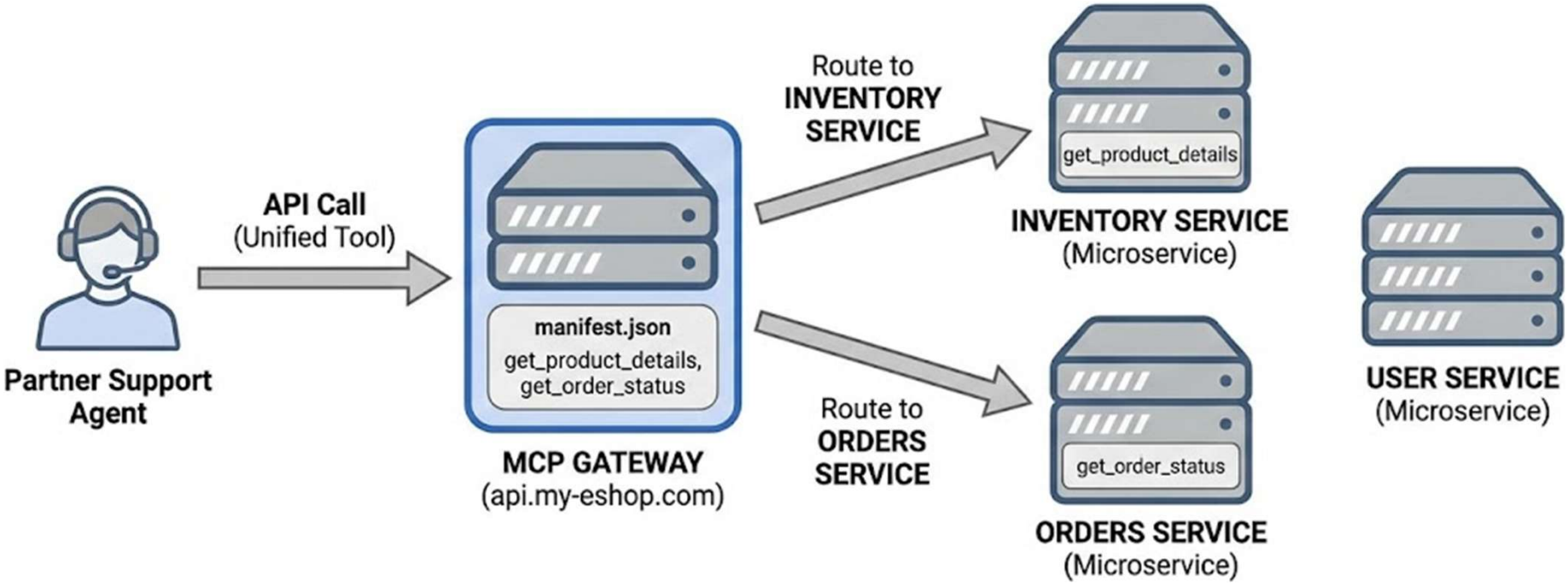
The Right Way (The Gateway Pattern):

1. We build one MCP Gateway server: https://api.my-eshop.com
2. Its manifest.json defines both get_product_details and get_order_status
3. When an agent calls get_product_details, the gateway routes to the Inventory
4. When an agent calls get_order_status, the gateway routes to the Orders Service

The Result: The partner agent has one simple tool, while we maintain full internal control and decoupling

Case Study: Architecting Our E-Shop with MCP

A single, secure and Manageable Entry Point for all Agentic Traffic



Summary: From Theory to Blueprints

We've learned **2 critical, repeatable patterns:**

The Adapter Pattern:

Use When: You have one legacy system to expose

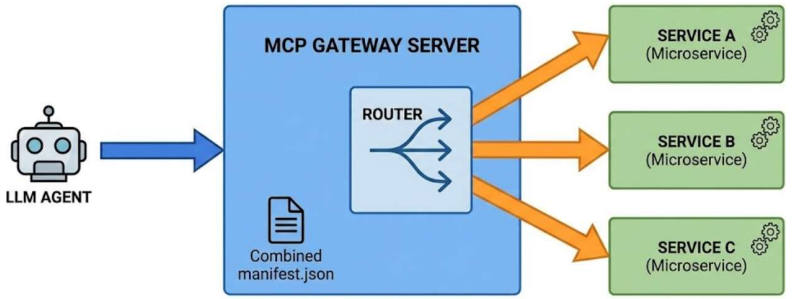
Job: "Translate" calls to one specific system

The Gateway Pattern:

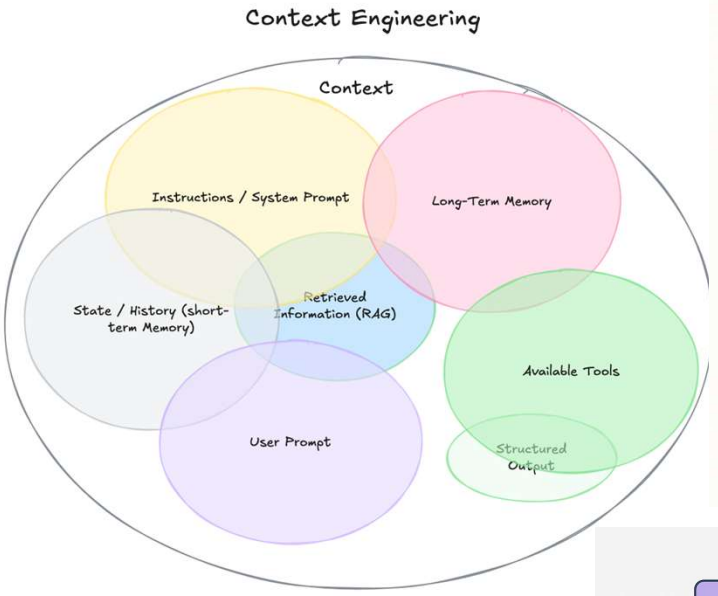
Use When: You have many microservices.

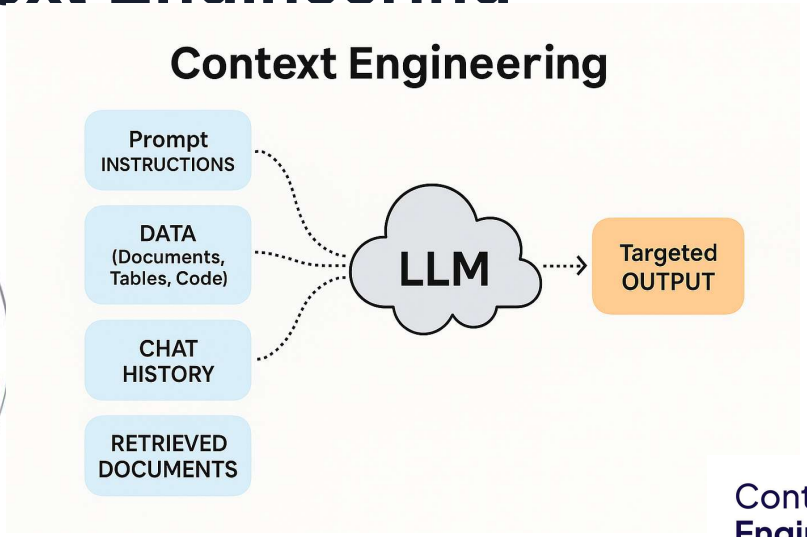
Job: "Route" calls to many different systems.

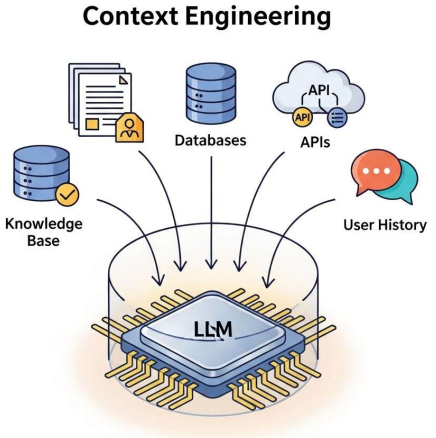


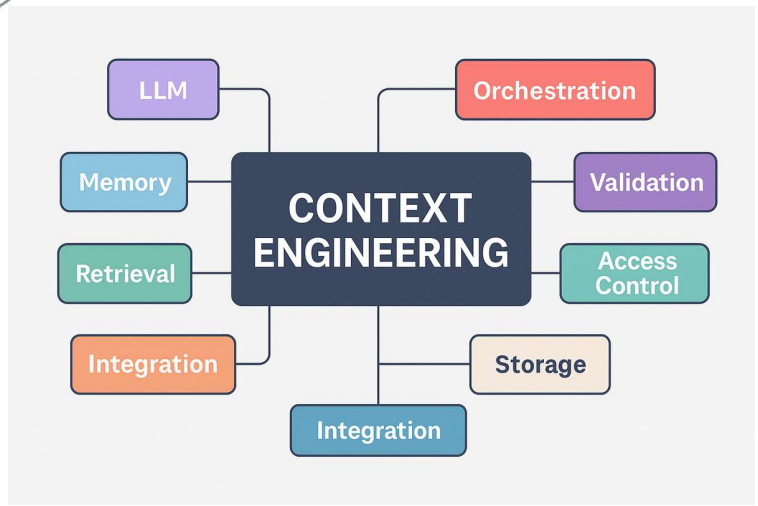


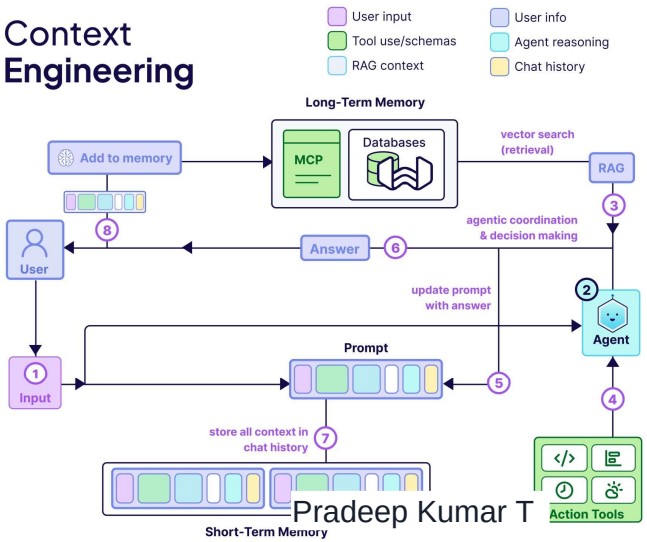
Introduction - Context Engineering











Pradeep Kumar T

Context Engineering: Context Engineering for AI Agents

What is Context Engineering?

Prompt Engineering vs Context Engineering

Strategies for Effective Context Engineering

Example Use Cases of Context Engineering



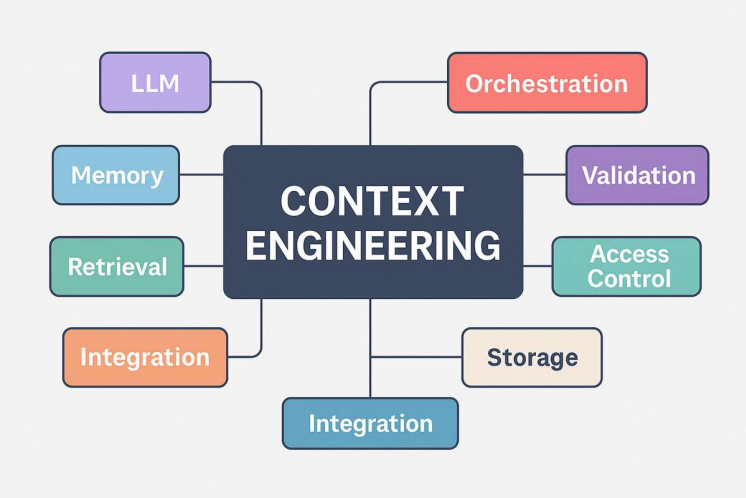
Pradeep Kumar T

Introduction - Context Engineering

Defined the **agent**, built its **components**, explored its **frameworks**, mastered its **design patterns** and standardized its **communication** with protocols like **MCP**

Now, we must ask: **what is the 'fuel' that this engine runs on?**

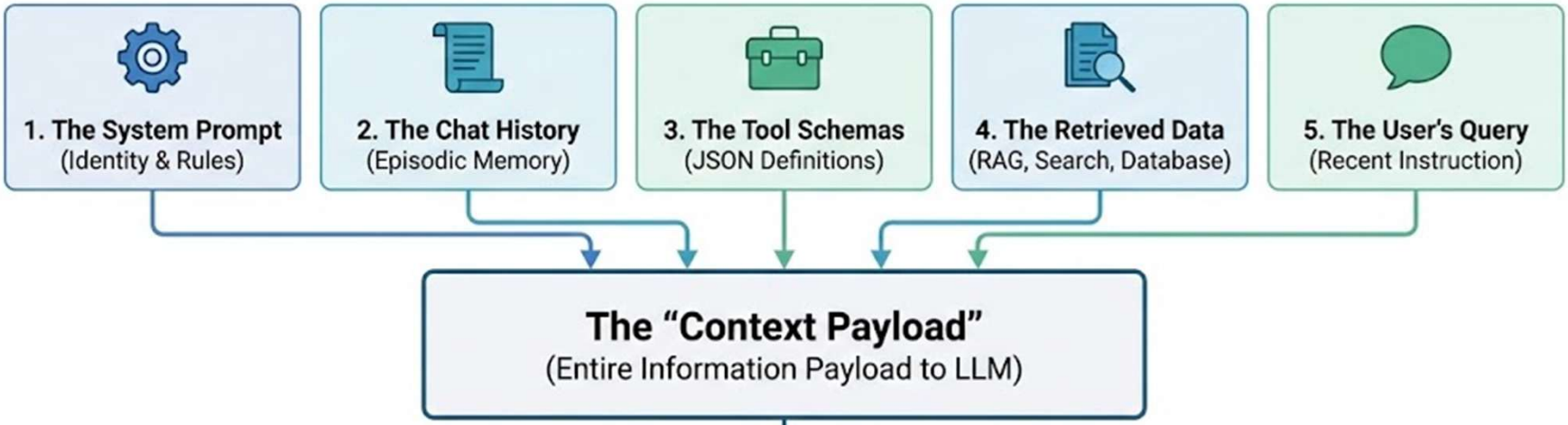
That fuel is **'Context'**
And the discipline of supplying it **is 'Context Engineering'**



What is "Context"? (It's Not Just a Prompt)

An **agent's "context"** is everything in its **limited attention span**

It is the **entire information** payload **sent** to the **LLM** on **every single turn**

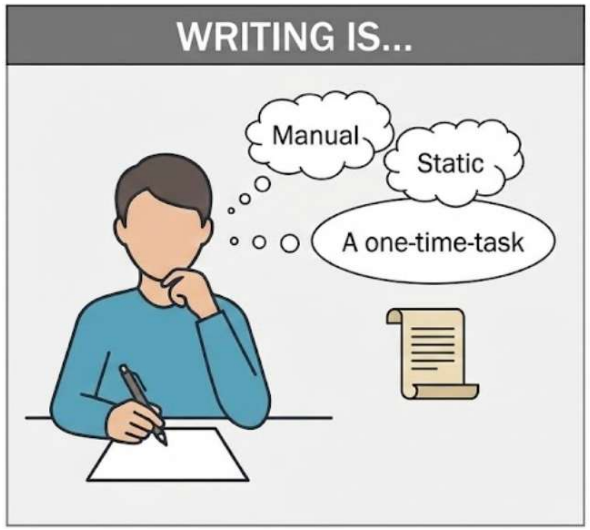


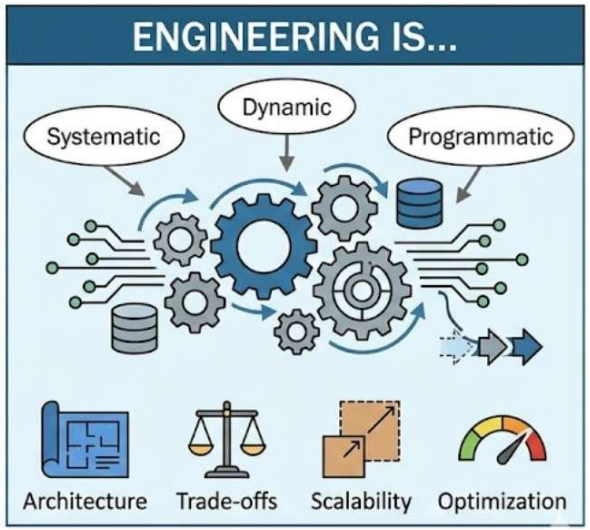
All of these has to be squeezed into **one, finite context window**

What is "Engineering"? (It's Not Just "Writing")

"Writing" is...
Manual
Static
A one-time-task
(writing one good system prompt)

"Engineering" is..
Systematic
Dynamic
Programmatic



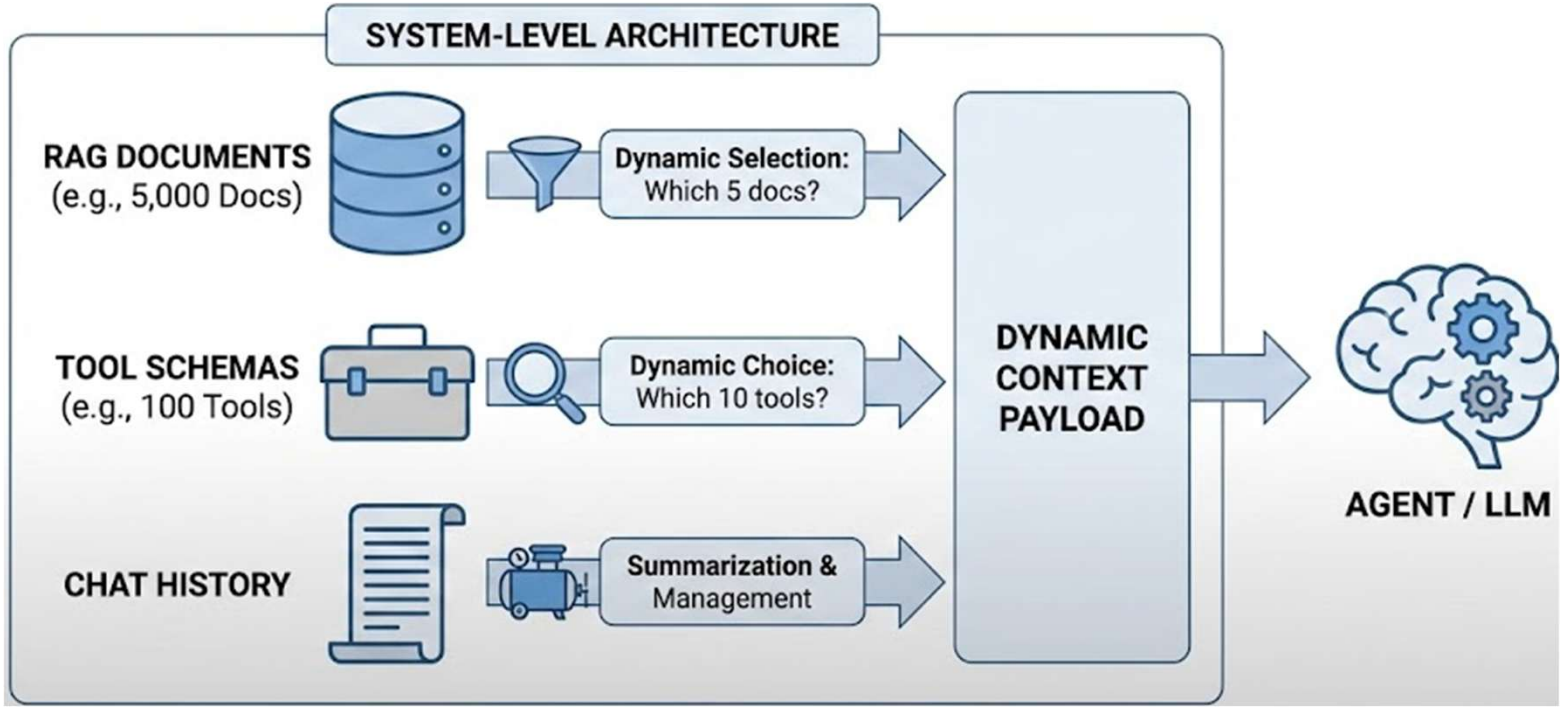


It is the **process** of **designing**, **building** and **managing** a system that runs **continuously** and **efficiently**

It **implies architecture**, **trade-offs**, **scalability** and **optimization**

"Context Engineering": The Architectural Discipline

Architectural **discipline** of **designing**, **building** and **dynamically managing** the entire **information payload** an **agent uses** to **perceive**, **reason** and **act**



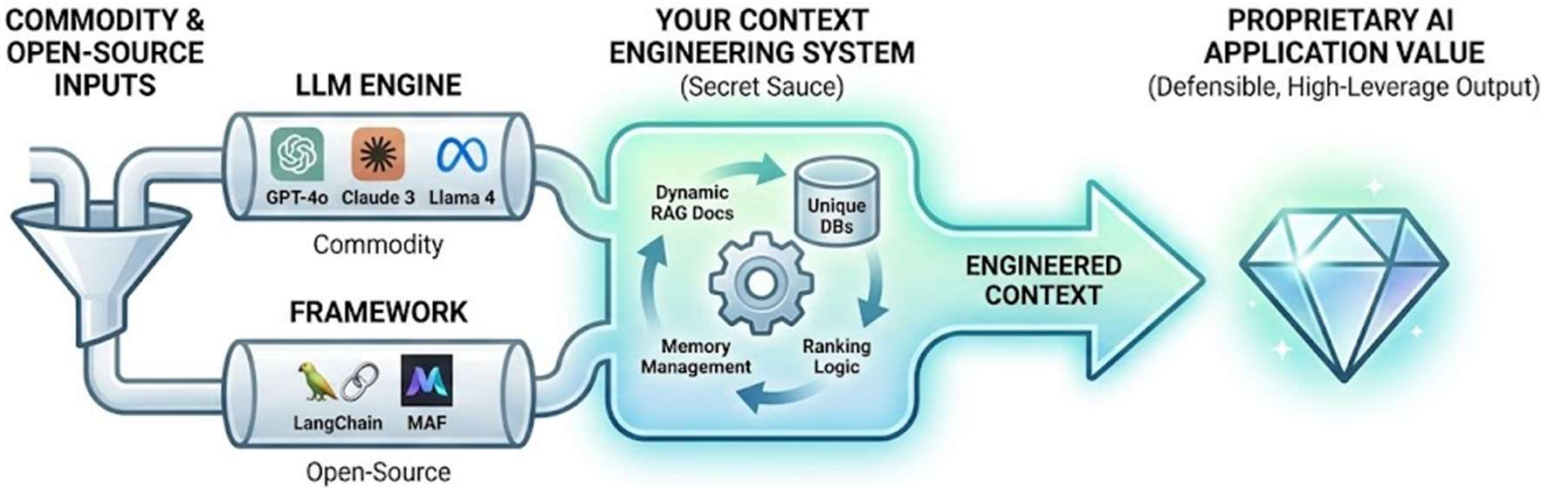
The Architect's Mandate: The AI Architecture of the Future

The **LLM "Engine"** (GPT-4o, Claude 3, Llama 4) is **becoming** a **commodity**

The **"Framework"** (LangChain, MAF) is **open-source**

The **real**, **defensible**, **proprietary value** of your **AI application** is **not** the **LLM**

It is the **system** you **build** to **dynamically source**, **rank** and **engineer** the **unique context** that you **feed into the LLM**

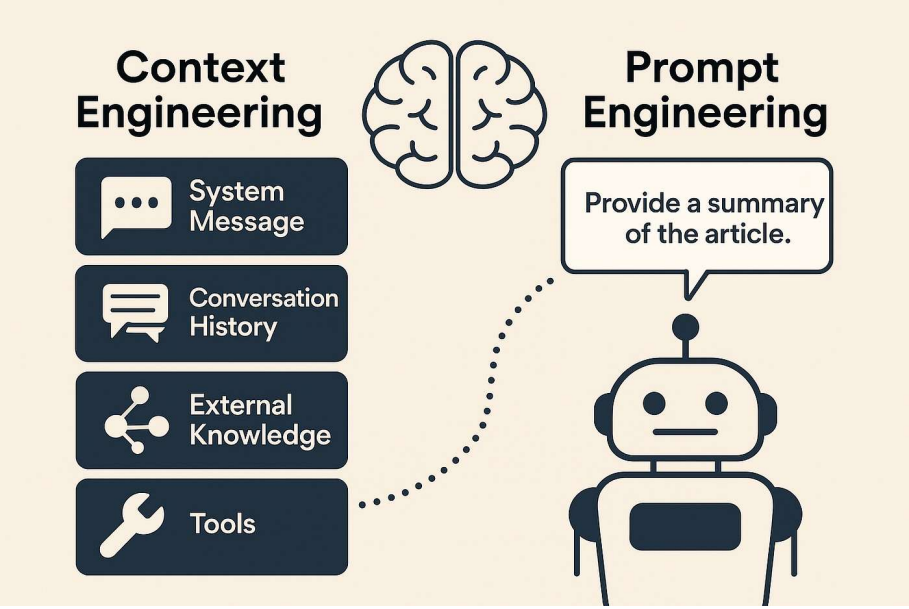


Prompt Engineering vs. Context Engineering

We introduced our **new**, **high-level discipline**: **Context Engineering**

Why it demands an architectural mindset—we must compare it directly to the skill that got us here: **Prompt Engineering**

Fundamentally shift our perspective from **'writing prompts'** to **'architecting information flows'**



What is "Prompt Engineering"? (The "Old Way")

Definition: The art and science of manually crafting a clear, effective, and precise instruction (prompt) for an LLM

Core Focus: The instruction itself

Process: Manual, iterative, and static. You "write" the prompt, test it, and save the "magic words" in your code

A "master wordsmith" carefully writing a **single**, **perfect**, **static instruction** for a **passive**, **stateless tool**



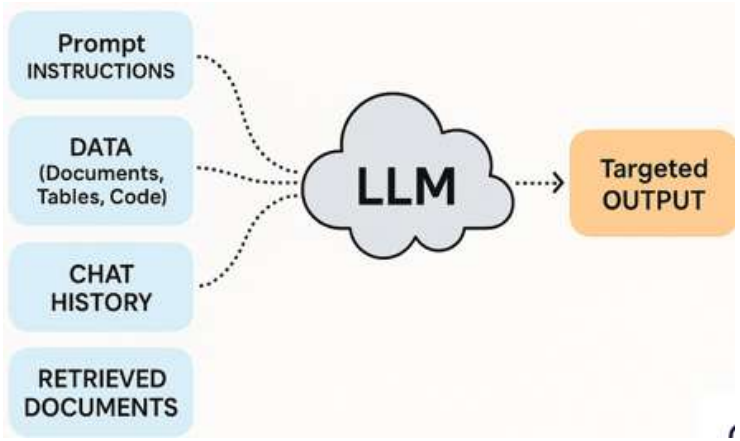
What is "Context Engineering"? (The "New Way")

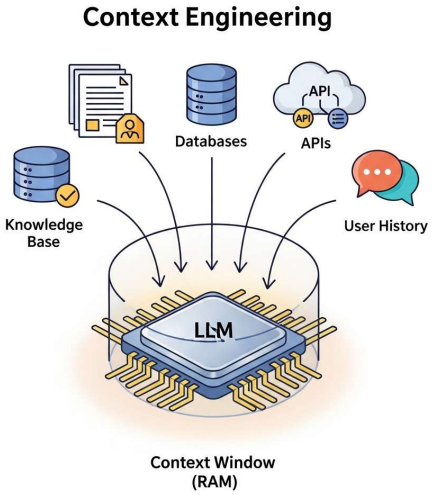
Definition: The architectural discipline of programmatically designing a system that dynamically assembles a complete information window for an agent

Focus: The system that builds the context

Process: Programmatic, dynamic, and stateful. You "build" a system that assembles prompts, RAG docs, tool schemas, and memory in real-time

An "**architect" designing** the entire **information dashboard** for an active, stateful employee (the agent)

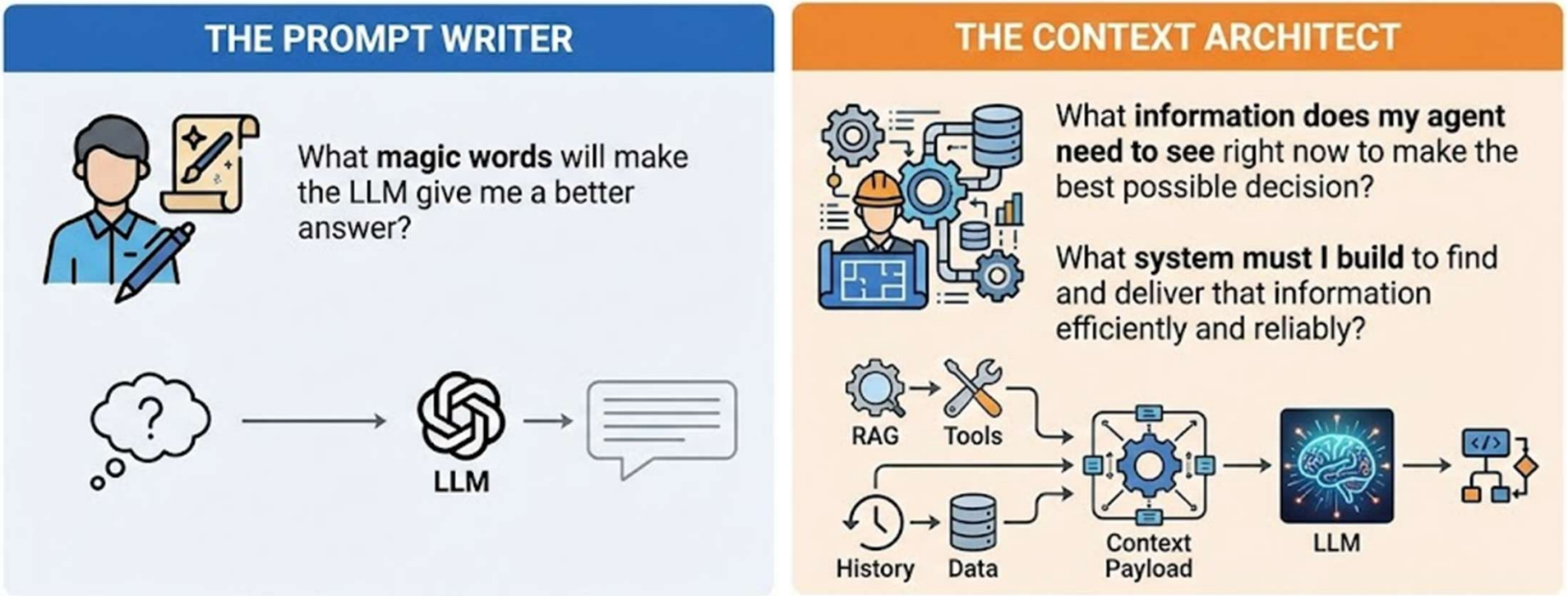




The Critical Shift: Prompt Engineering vs. Context Engineering

Feature	Prompt Engineering	Context Engineering
Artifact	A static string	A dynamic system
Focus	The Instruction (The "magic words")	The Information Flow (The "data")
Process	Manual & Static	Programmatic & Dynamic
Scope	A single prompt	The entire context window (Prompt + RAG + Tools + Memory)
Target	Stateless LLM Call (Generative AI)	Stateful Agentic Loop (Agentic AI)
Role	Prompt Writer	Information Architect

From "Writer" to "Architect"



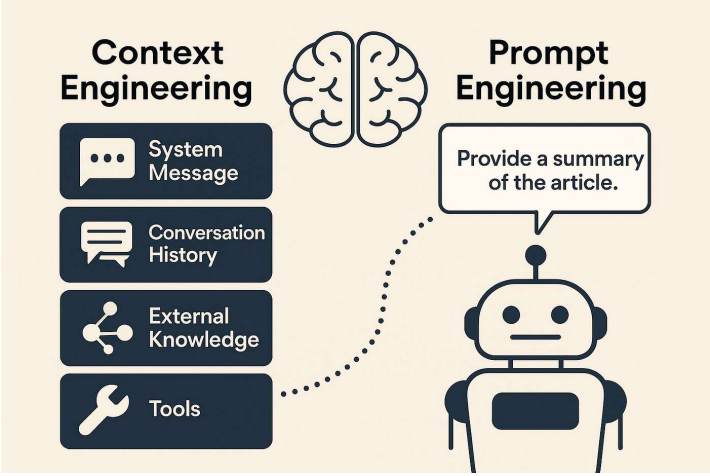
Fundamental shift from **writing prompts** to **architecting information flows**

The Anatomy of Context (Types of Context)

We've shifted our mindset from **'writing a static prompt'** to **'architecting a dynamic information flow'**

Now that we are **'Context Architects'** our first job is to **understand** the **'materials'** we are working with

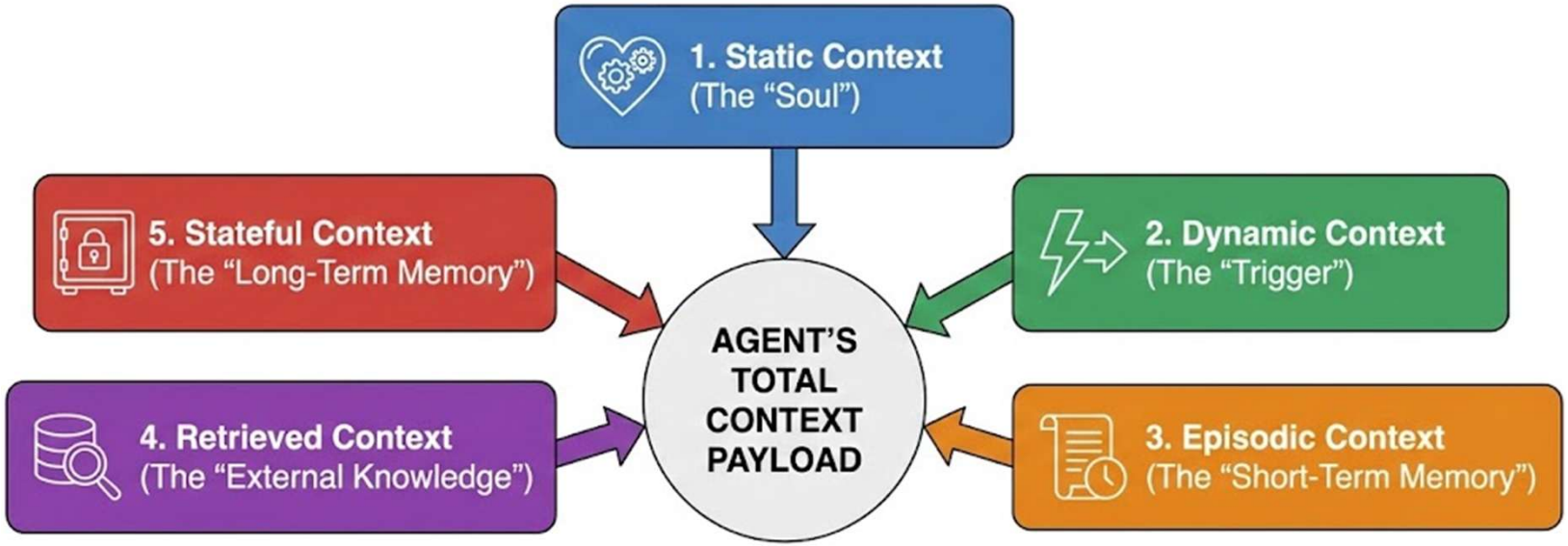
We will perform an **'anatomy'** of the context window. We'll create a formal framework by **categorizing** the **different types** of **context** we must learn to manage



The Architect's Framework: 5 Types of Context

As **architects**, we **can't just 'dump' information** into the **prompt**

Must be **systematic**. **Classify** all **agentic context** into **5 distinct categories**

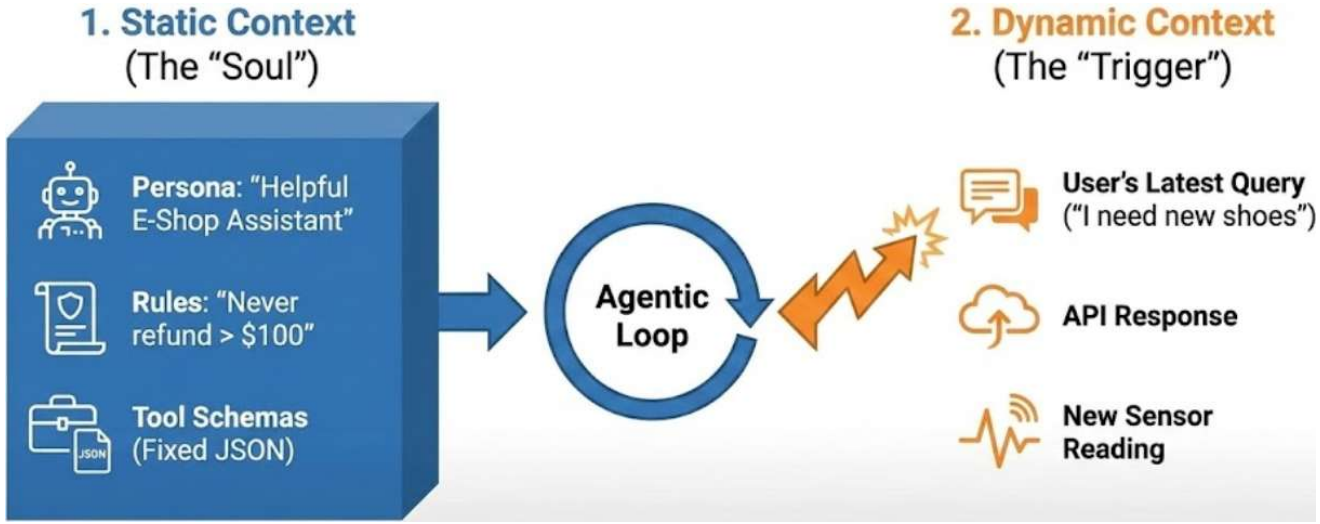


Understanding these types allows us to engineer the flow deliberately

Type 1 & 2: Static and Dynamic Context

1. Static Context (The "Soul"):
Information that is fixed and rarely changes
The agent's persona, core rules and the tool schemas

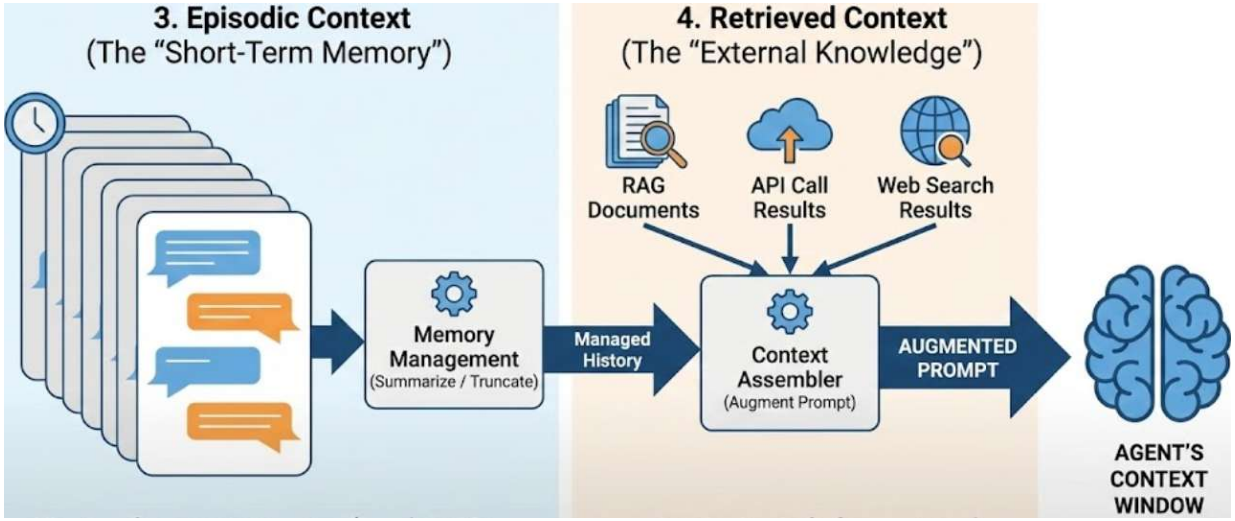
2. Dynamic Context (The "Trigger"):
The new, immediate piece of information that initiates the agentic loop
The user's latest query, an API response or a new sensor reading



Type 3 & 4: Episodic and Retrieved Context

3. Episodic Context (The "Short-Term Memory"):
The history of the current interaction
The last 10 messages in the chat history
This context grows and must be managed to avoid overflowing the window

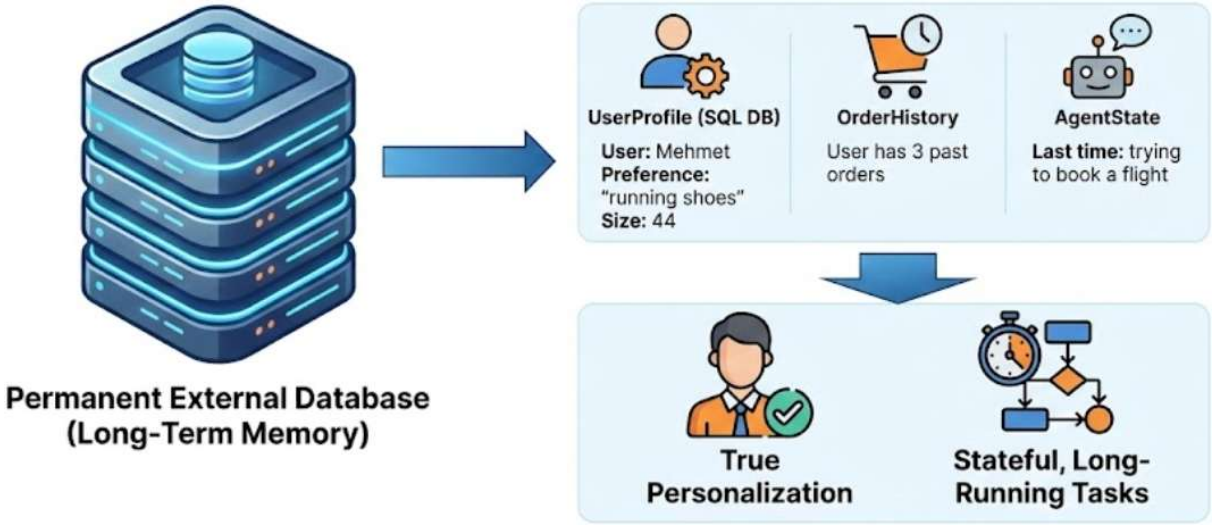
4. Retrieved Context (The "External Knowledge"):
Just-in-time information pulled from external sources
The RAG documents, API call results or web search results



Type 5: Stateful Context (The "Long-Term Memory")

This is the **most powerful** and **most "engineered" type** of **context**

Definition: Information **pulled from** a **permanent**, **external database** about the user or state



This is what enables **true personalization** and **stateful**, **long-running tasks**

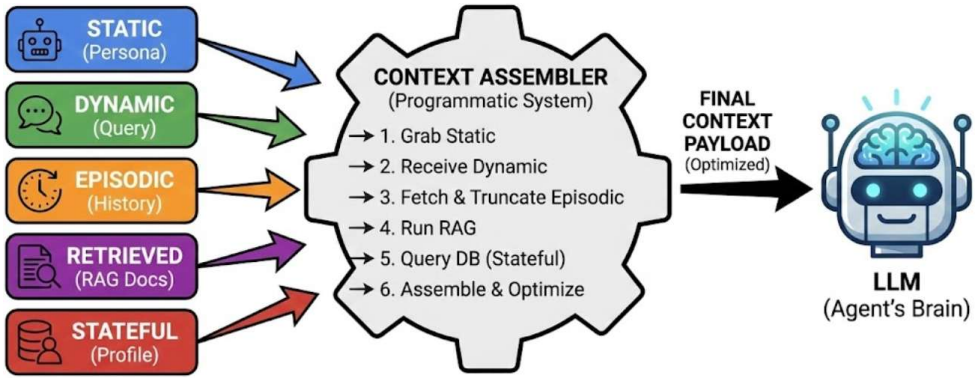
The Architect's Job: Context Assembler

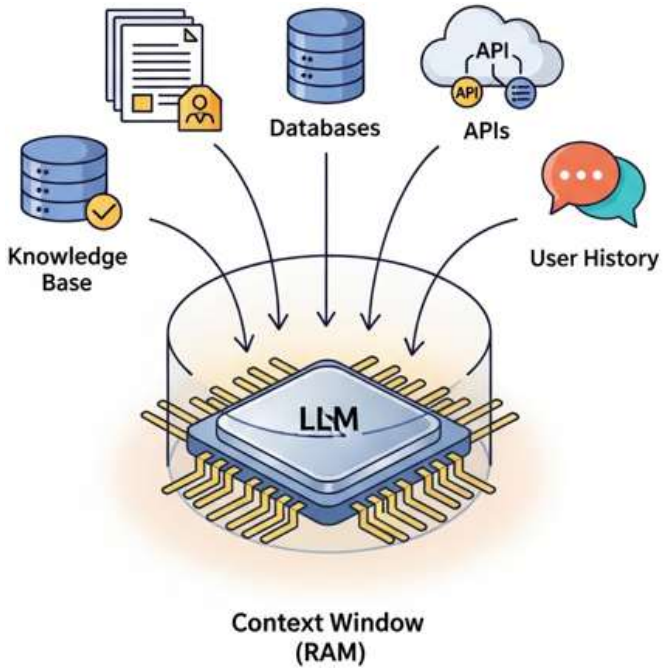
Our job as a **Context Engineer** is to build the "**Context Assembler**"

This is the code that, for every single loop:

1. Grabs the Static context (persona)
2. Receives the Dynamic context (query)
3. Fetches & truncates the Episodic context (history)
4. Runs RAG to get the Retrieved context (docs)
5. Queries the DB to get the Stateful context (profile)

Assembles all 5 into the **final, optimized** payload for **LLM**



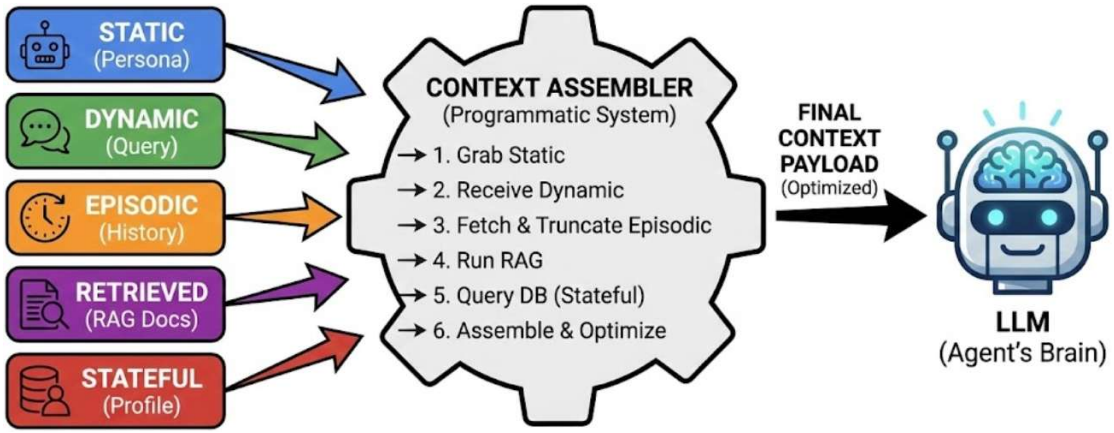


Strategies for Effective Context Engineering

The **5 distinct types** of context our agent must deal with:
Static, Dynamic, Episodic, Retrieved and Stateful

Challenge: Agent has a **potentially infinite amount** of **information** it could use, but it **has to fit all** of it into a **finite context window**

We are **not just 'stuffing' data** into a **prompt**
We are **performing** a **complex**, **real-time optimization problem**



The Engineer's Constraint: The Finite Context Window

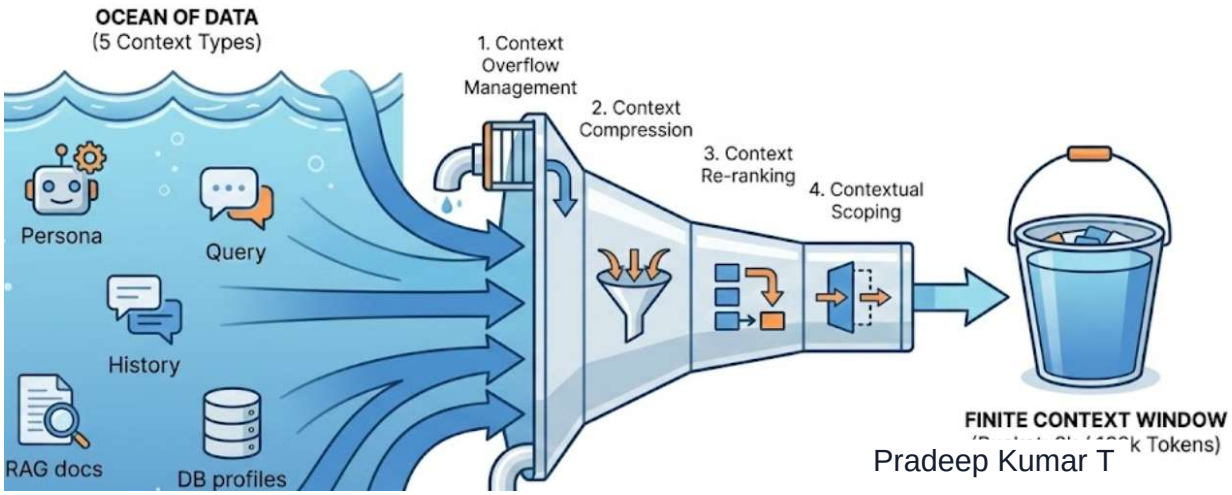
The Problem: We have an ocean of data from our 5 context types (Persona, Query, History, RAG docs, DB profiles)

The Constraint: We have a bucket (the context window: 8k, 128k, etc.)

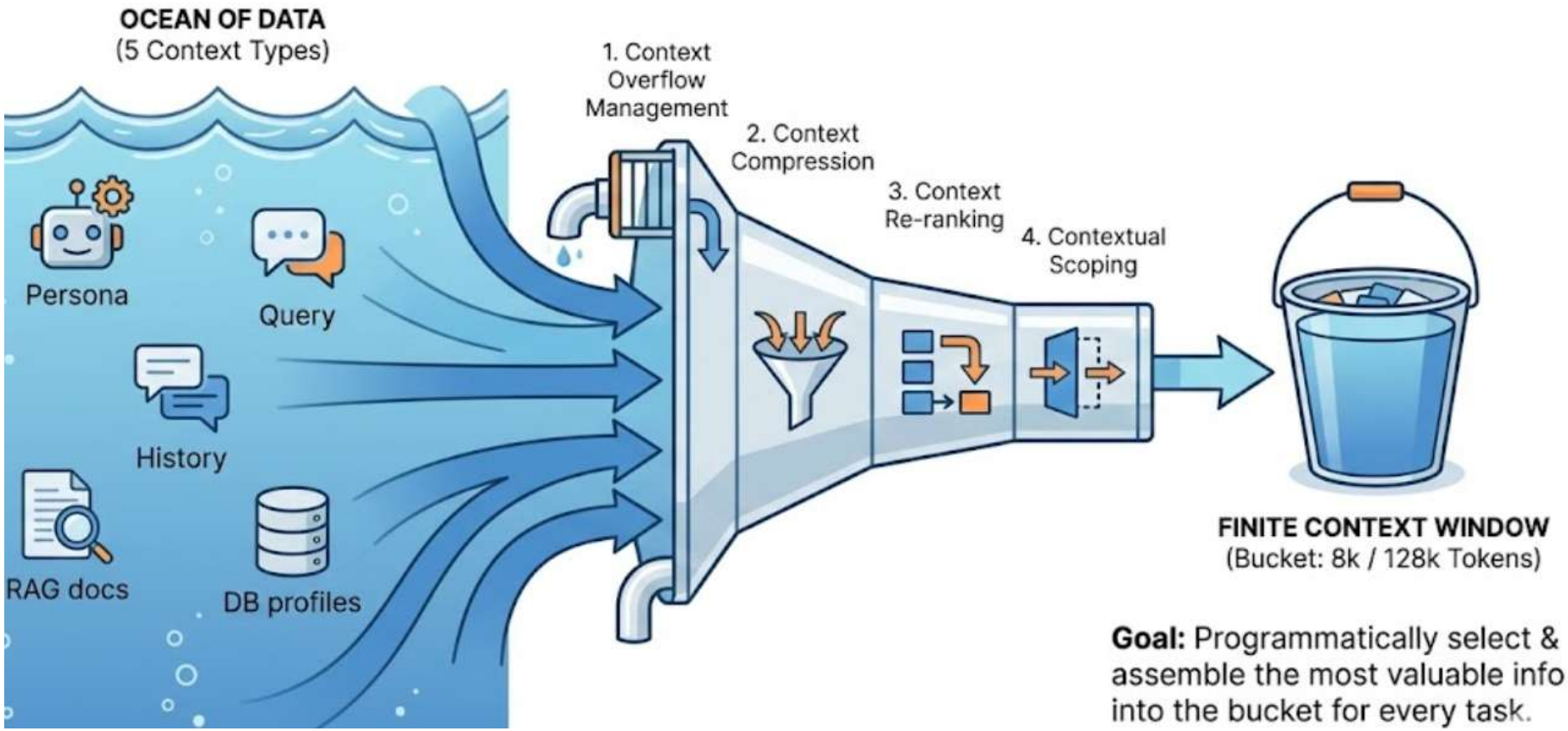
The Goal: To programmatically select and assemble the most valuable information into that bucket for every single task

Our Toolkit: We have four primary strategies to achieve this

1. Context Overflow Management
2. Context Compression
3. Context Re-ranking
4. Contextual Scoping

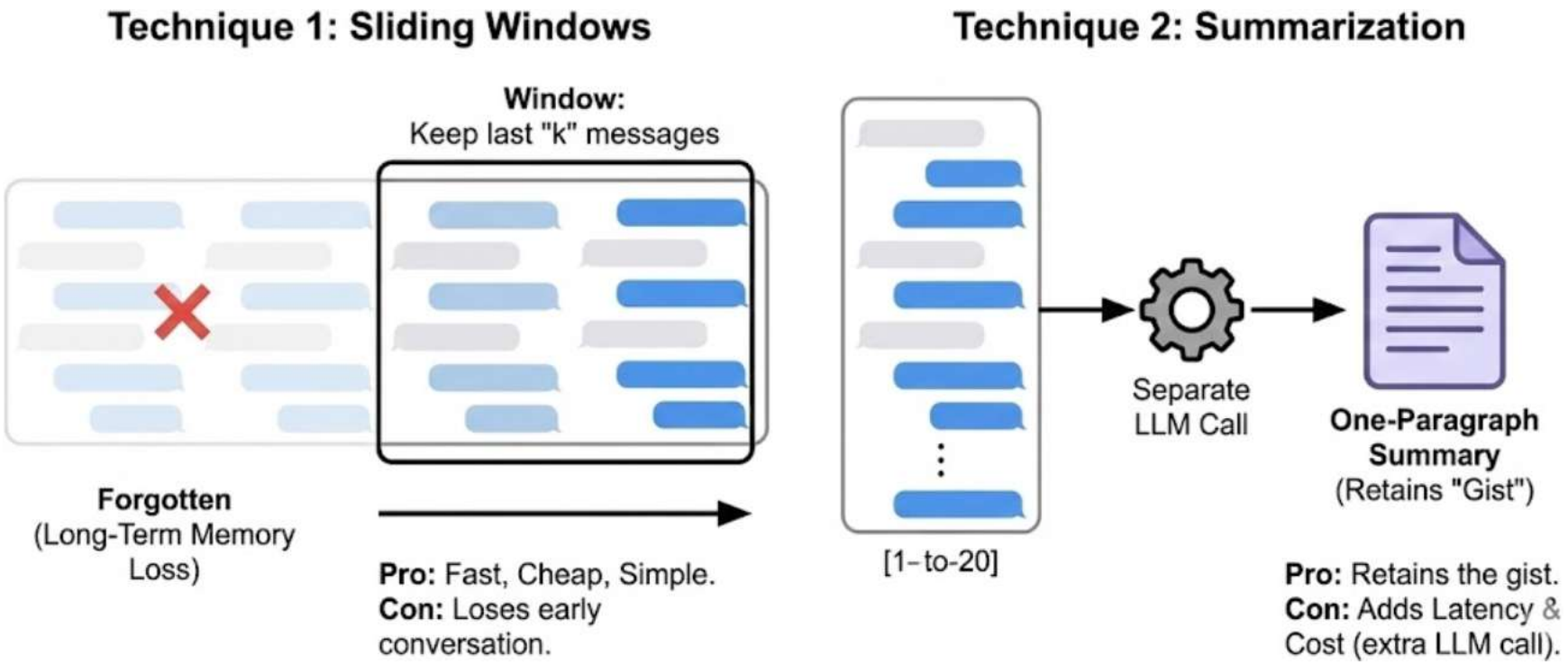


The Engineer's Constraint: The Finite Context Window



Strategy 1: Context Overflow Management

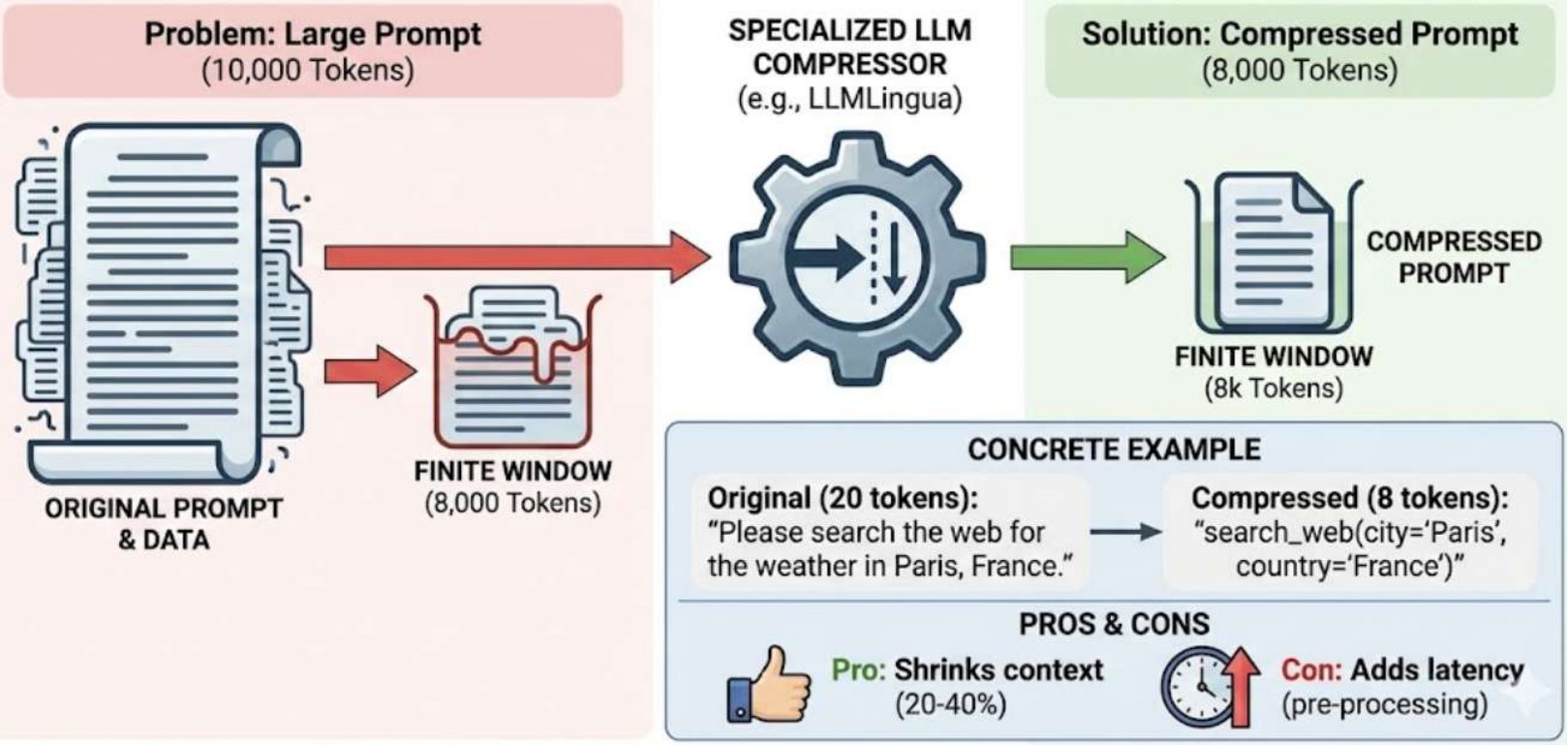
Problem: What do you do when the "Episodic Context" (chat history) gets too long?



Strategy 2: Context Compression

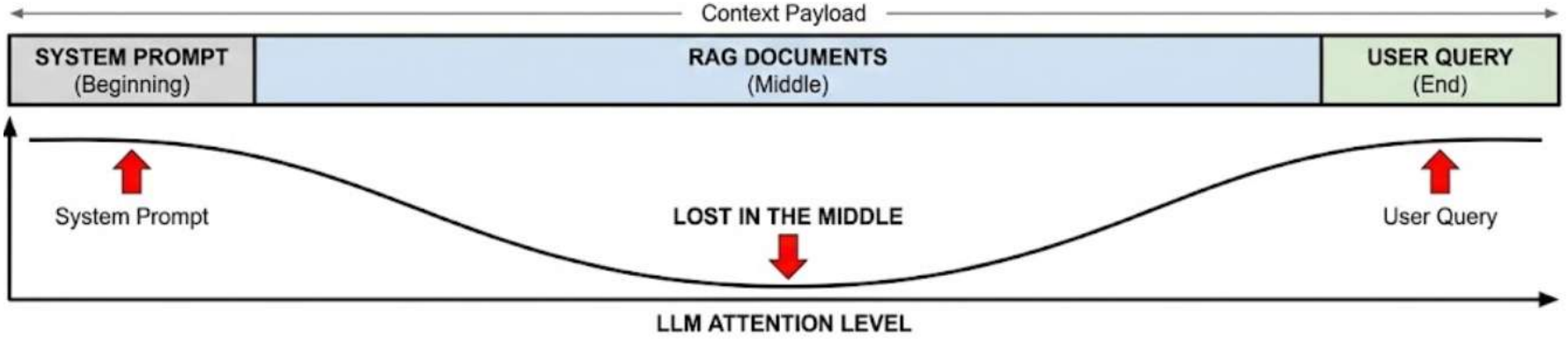
The Problem: We have a 10,000-token prompt, but an 8,000-token window

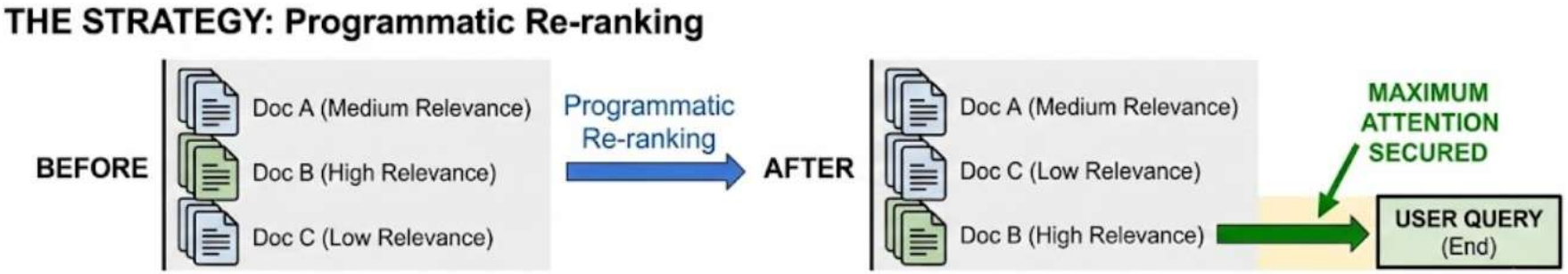
Technique: Use a specialized, small LLM to "compress" the data



Strategy 3: Context Re-ranking

The Problem: LLMs suffer from the "**Lost-in-the-Middle**" problem

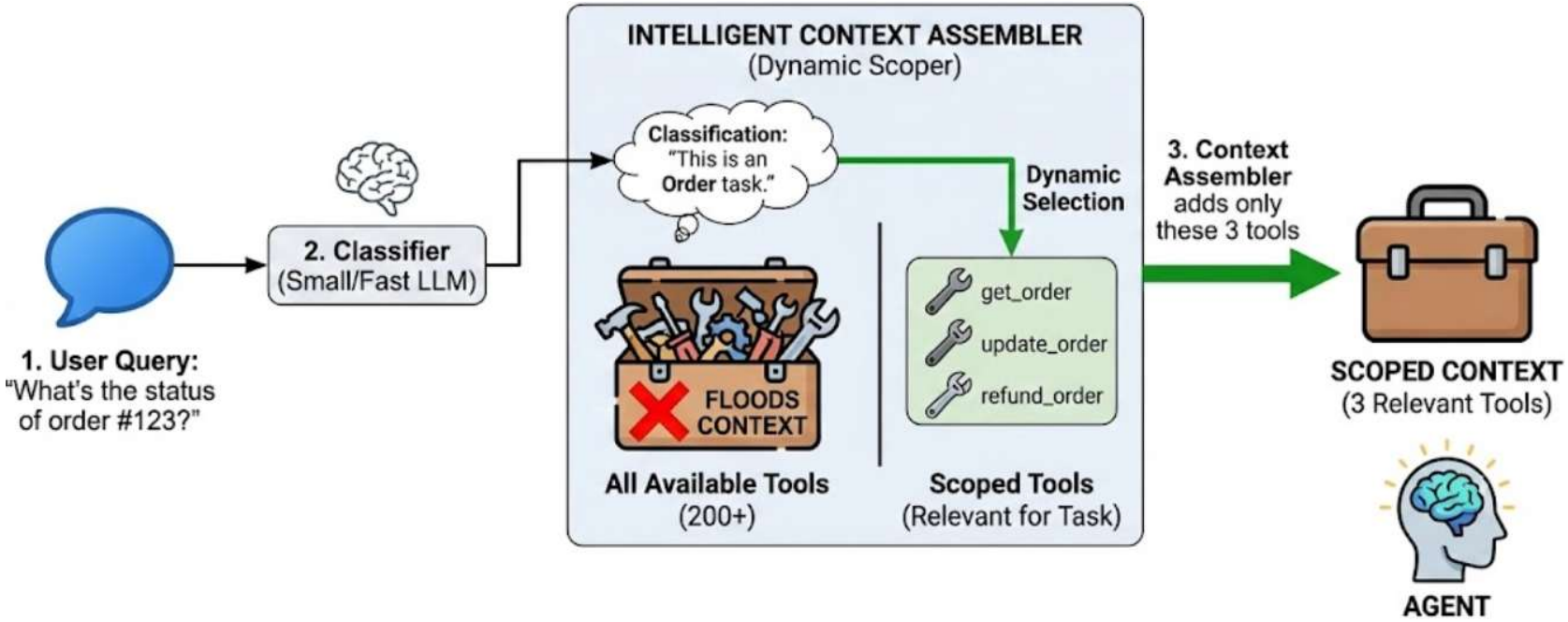




Programmatically **move the most relevant** docs to the **very end of the context**, just **before the user's query**

Strategy 4: Contextual Scoping

Problem: An enterprise agent might have 200 tools, but only needs 3 for a given task. Showing all 200 tool schemas would flood the context



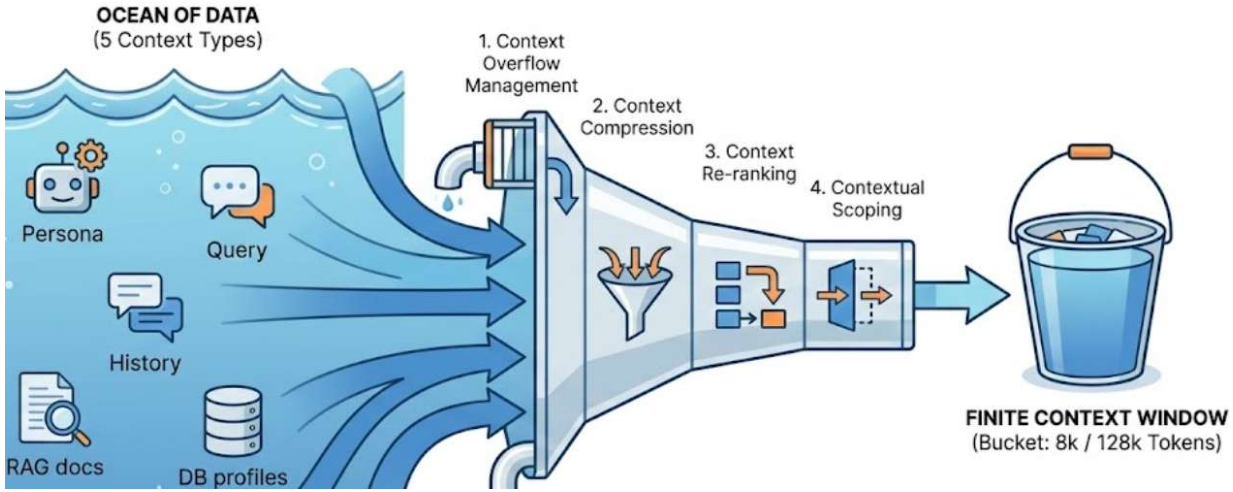
This is the most "agentic" strategy: Dynamically selecting only the tools needed for the task

Summary: The Context Engineer's Toolkit

Graduated from simple 'prompting' to true engineering

Our Toolkit Includes:

1. **Overflow Management:** Sliding windows & summarization (for history)
2. **Compression:** Shrinking context with tools like LLMLingua
3. **Re-ranking:** Fighting "Lost-in-the-Middle" for RAG
4. **Scoping:** Dynamically selecting what to show (like tools)

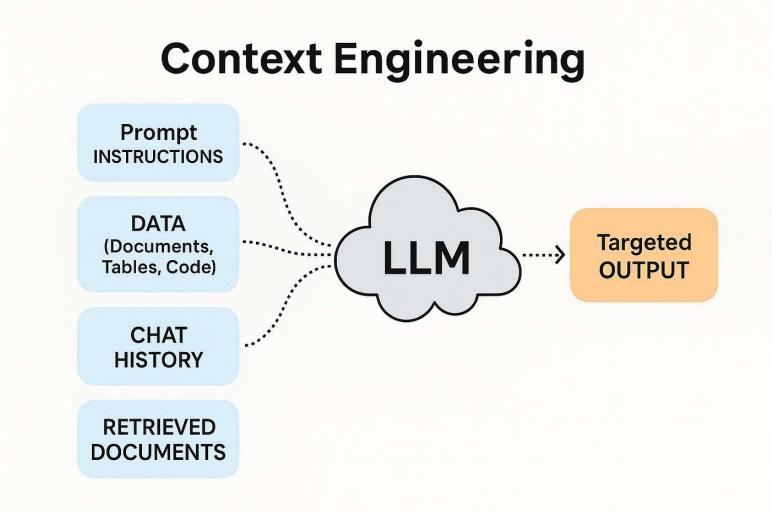


Example Use Cases of Context Engineering

Defined the 'what'—the **5 types of context** and defined the 'how'—the **4 engineering strategies** to manage them

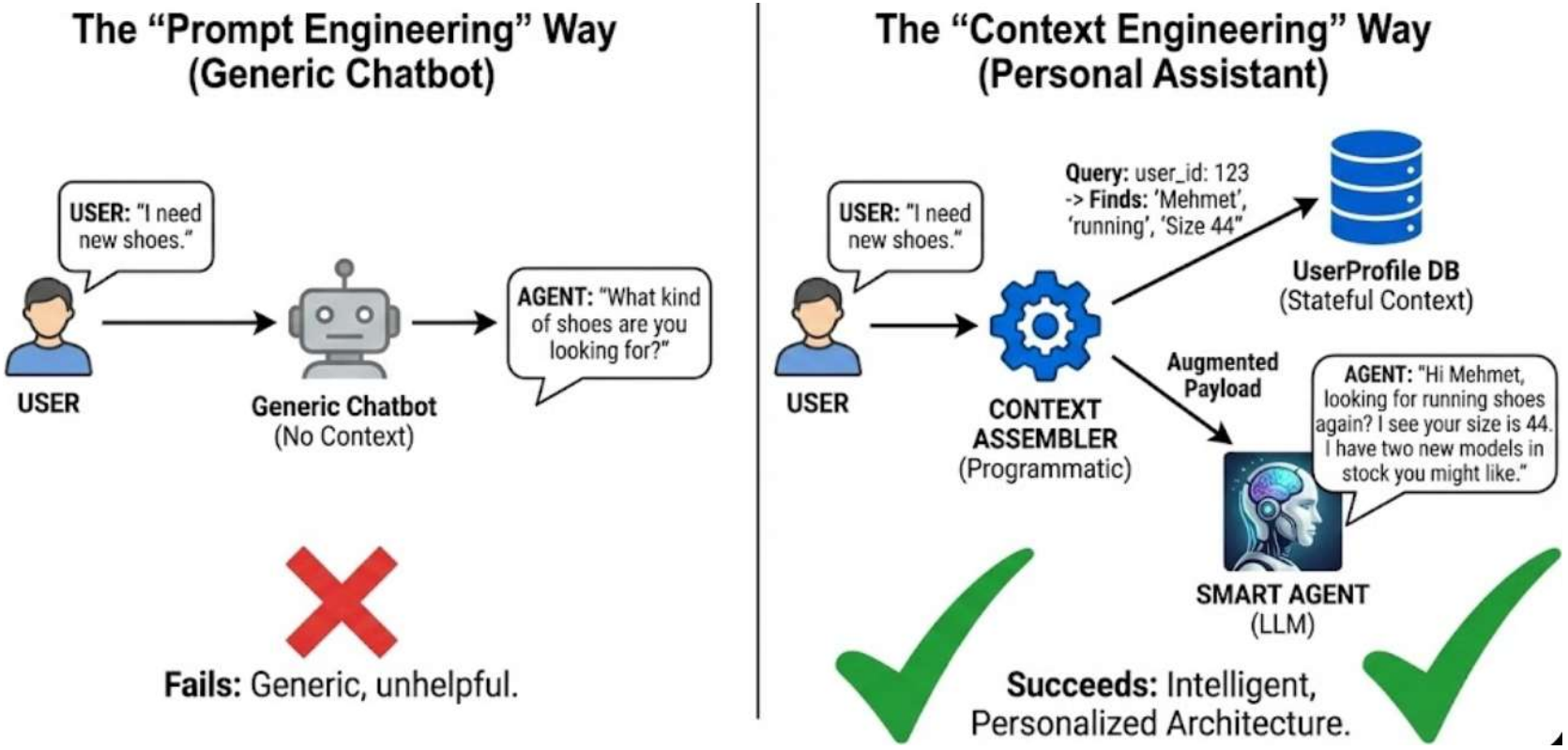
Why do we **go through** all this **extra effort**?
Why build a **'Context Assembler'** instead of just writing a better prompt?

We will explore the **2 most powerful use cases**: **Hyper-Personalization** and the **Dynamic 'Expert' Agent**



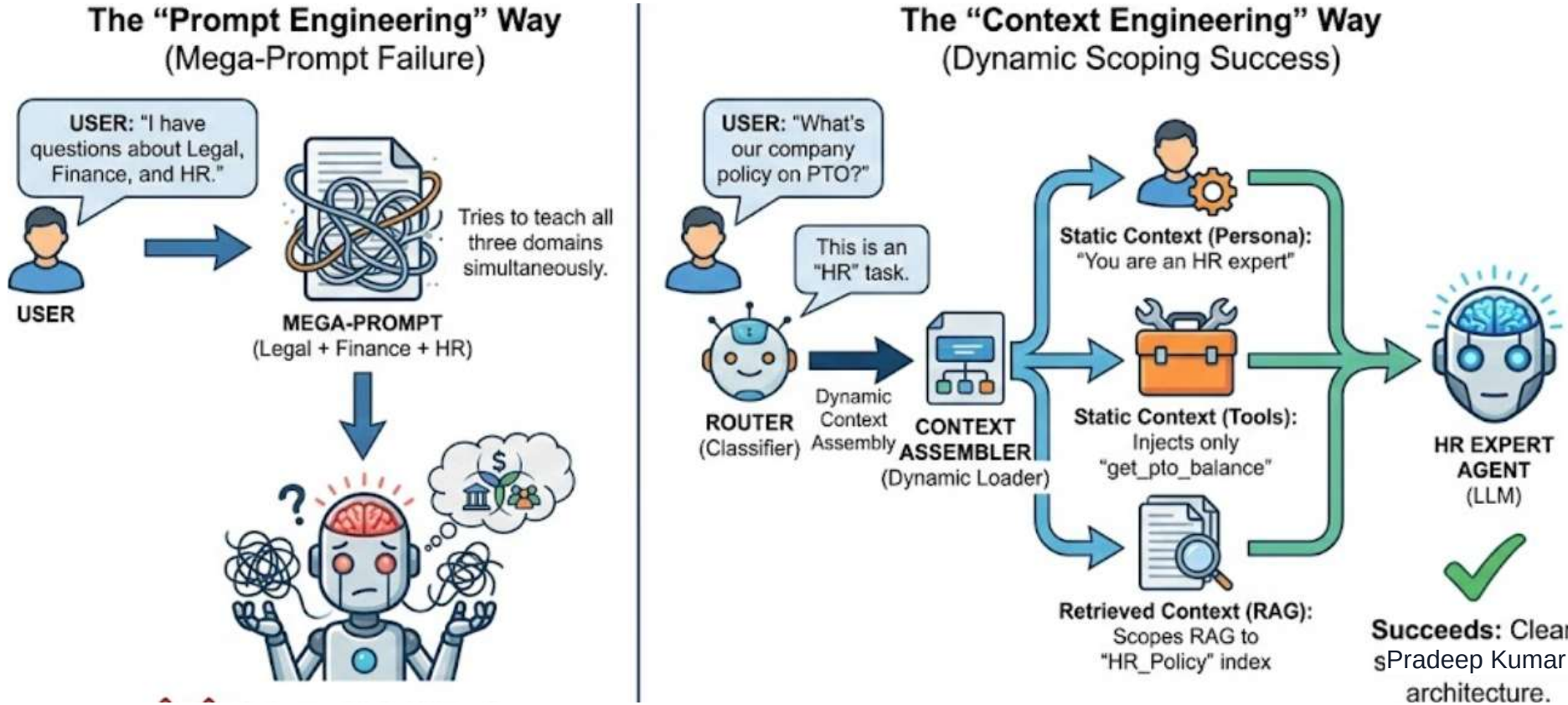
Use Case 1: Hyper-Personalization

The Goal: An agent that knows the user
It moves from a generic "chatbot" to a true "personal assistant"



Use Case 2: The Dynamic "Expert" Agent

The Goal: A single, central agent that can handle multiple, distinct, expert domains (Legal, Finance, HR)



Summary: Architecture > Prompting

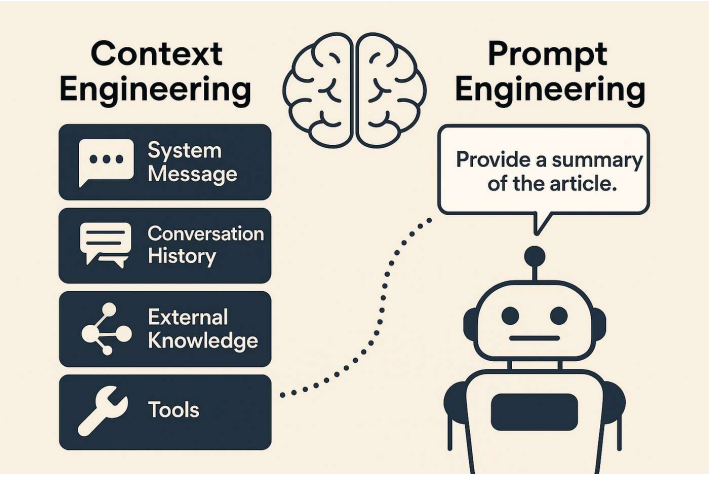
These use cases are **not** the result of "**magic prompts**"

They are the **result** of a **deliberate information architecture**

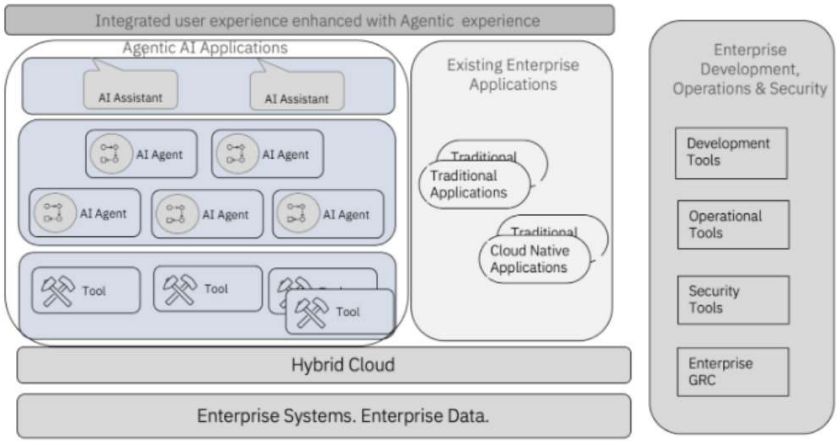
Hyper-Personalization: Achieved by engineering Stateful Context (user profiles)

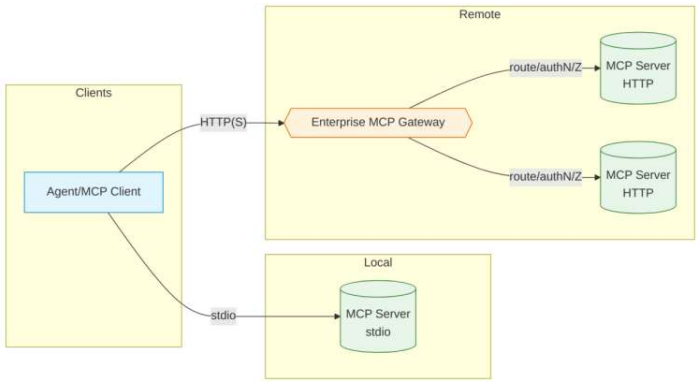
Dynamic "Experts": Achieved by engineering Contextual Scoping (swapping personas, tools and RAG indexes)

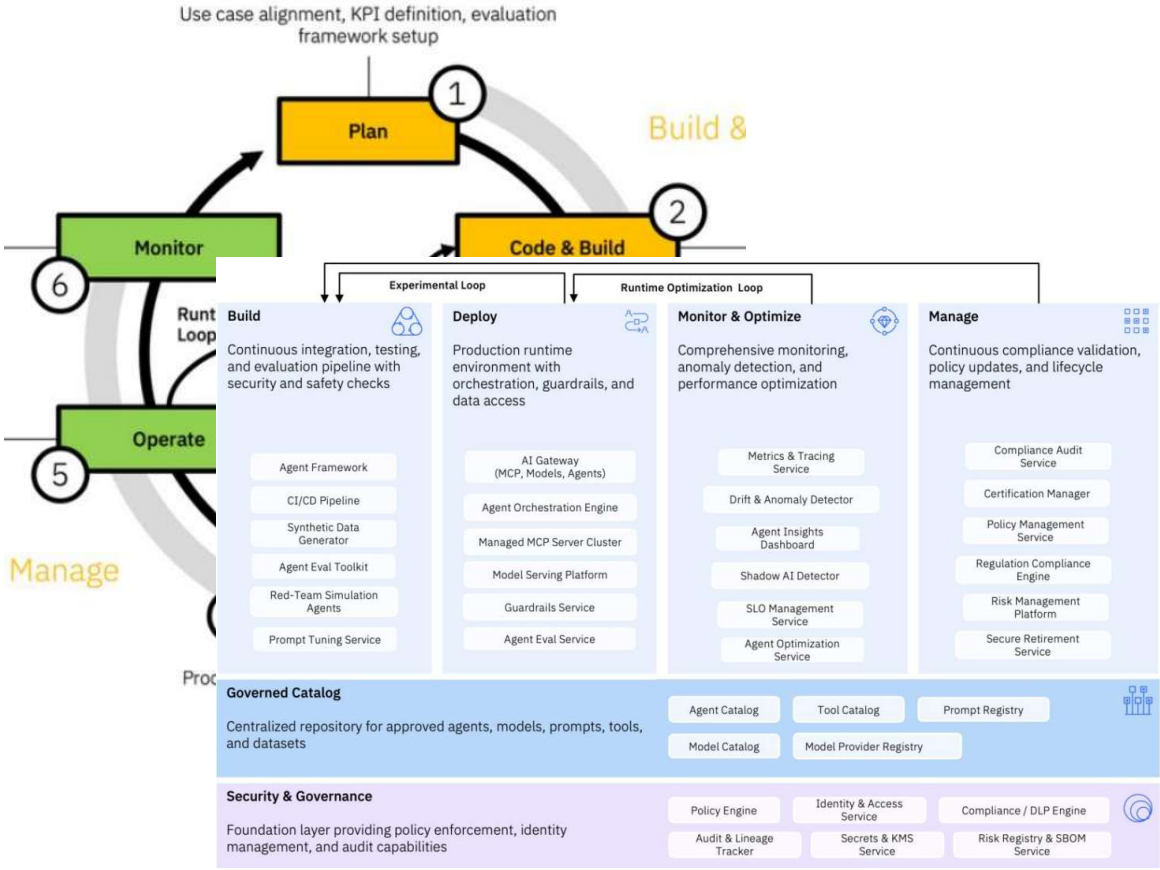
This is the **true power** and **purpose** of **Context Engineering**



Introduction - Context Engineering







The Agent Development Lifecycle (ADLC) w/ Enterprise Agent Development

The Agent Development Lifecycle (ADLC): A 7-Step Guide

Deep Dive: The Challenge of "Evaluation"

Deep Dive: "AgentOps" (Observability & Monitoring)

LLM Guardrails for Data Leakage, Prompt Injection, Security

Reference Architecture for an Agentic AI Platform

Pradeep Kumar T

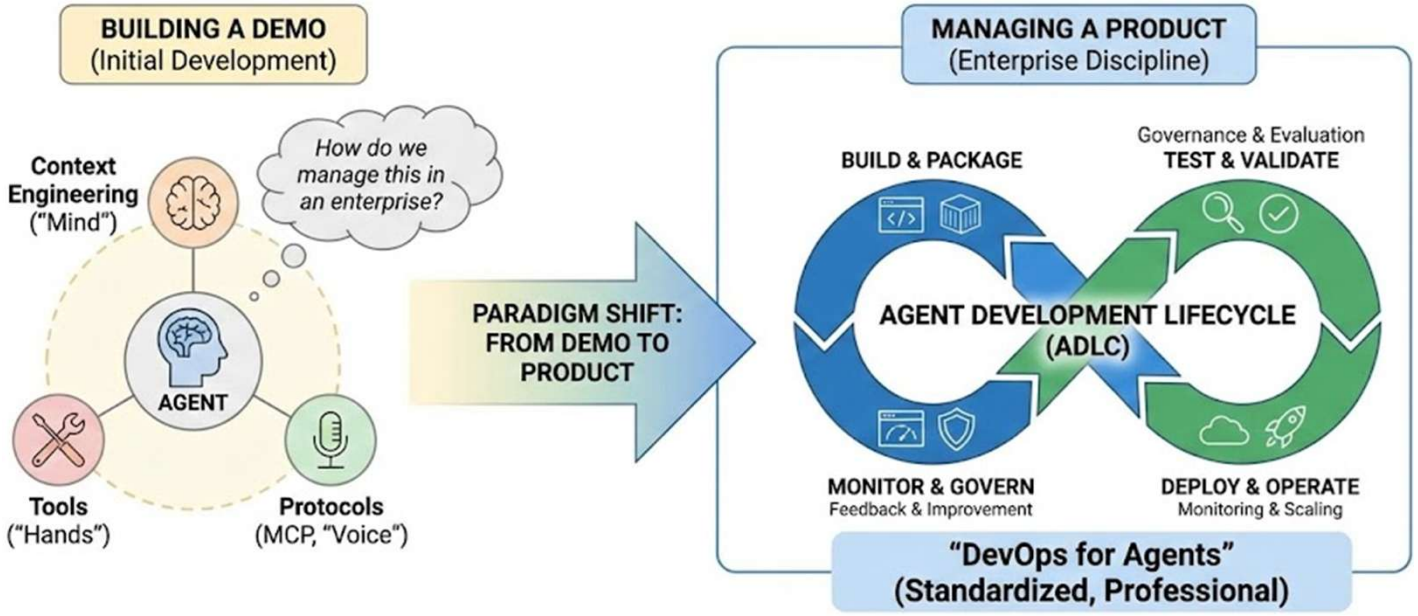


Introduction – Enterprise Disipline of Agent Development

Defined the **agent**, built its 'mind' with **Context Engineering**, given it 'hands' with **tools** and standardized its 'voice' with **protocols** like **MCP**

Ask the most important question: **How do we manage this?**

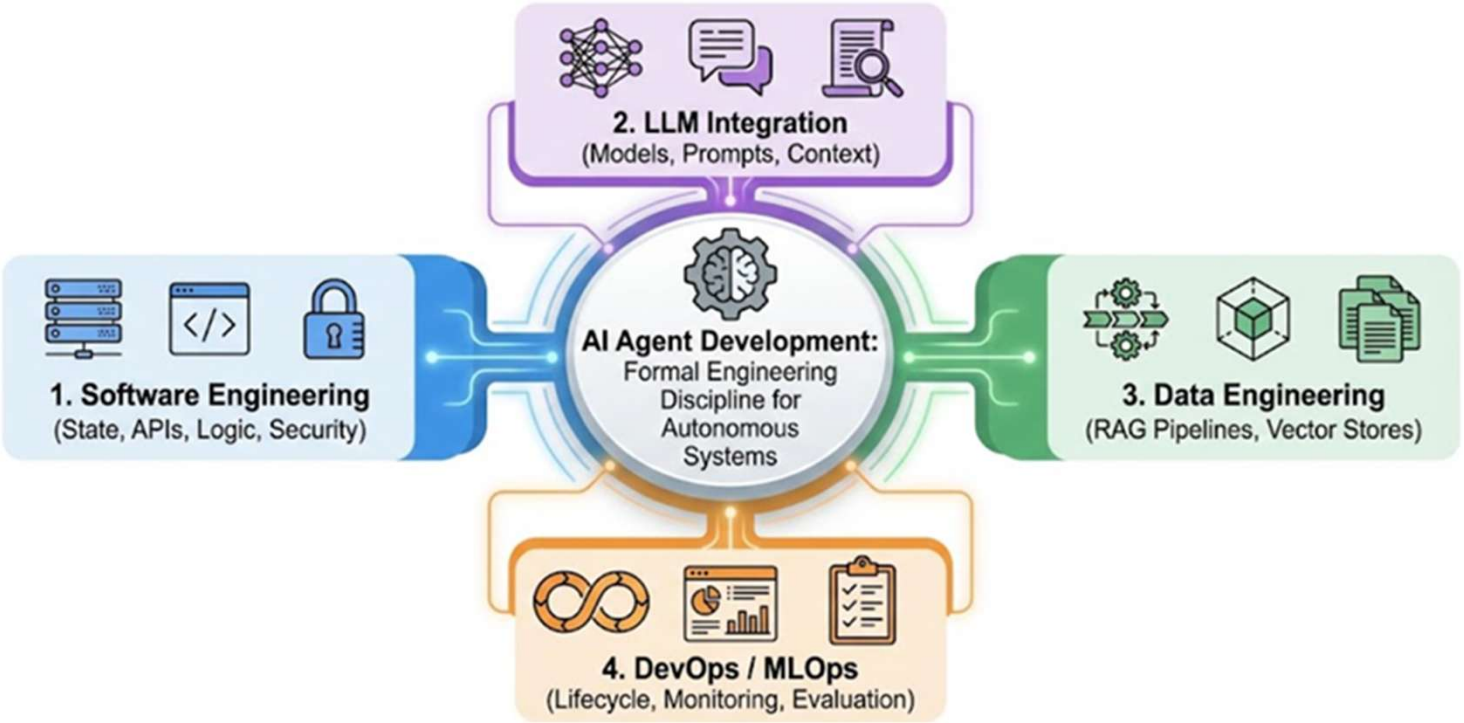
How do we **build**, **test**, **deploy** and **govern** this **new class** of **software** in a real, **professional**, **enterprise** environment?



What is AI Agent Development?

It is not just "**prompt engineering**" or "**model training**"

It is a formal **engineering discipline** for **building**, **testing**, **governing** and managing **autonomous**, **goal-driven software** systems



The Core Challenge: The Probabilistic Shift

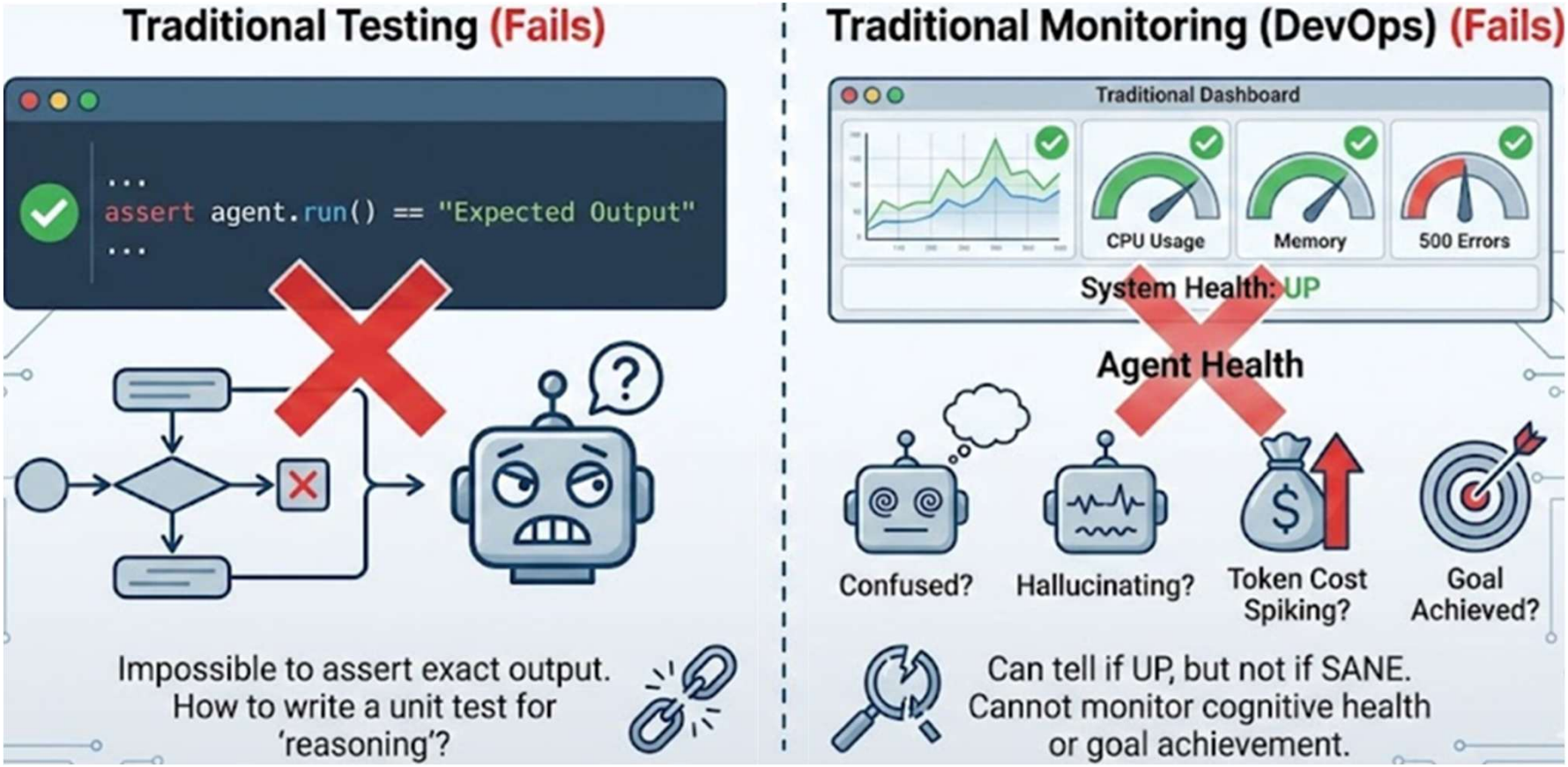
The "Old" Way: Deterministic Software (SDLC)
Input X -> Function f(x) -> Output Y
F(5) always returns 10
If it returns 11, it is a bug
We test for binary correctness (Pass/Fail)



The "New" Way: Probabilistic Software (ADLC)
Goal -> Agent(Goal + Context) -> Result
Agent(Goal) might return Result A or Result B... and both could be valid
It is non-deterministic by design

This **changes everything** about **development** and **QA**

Why Traditional QA & DevOps Fails

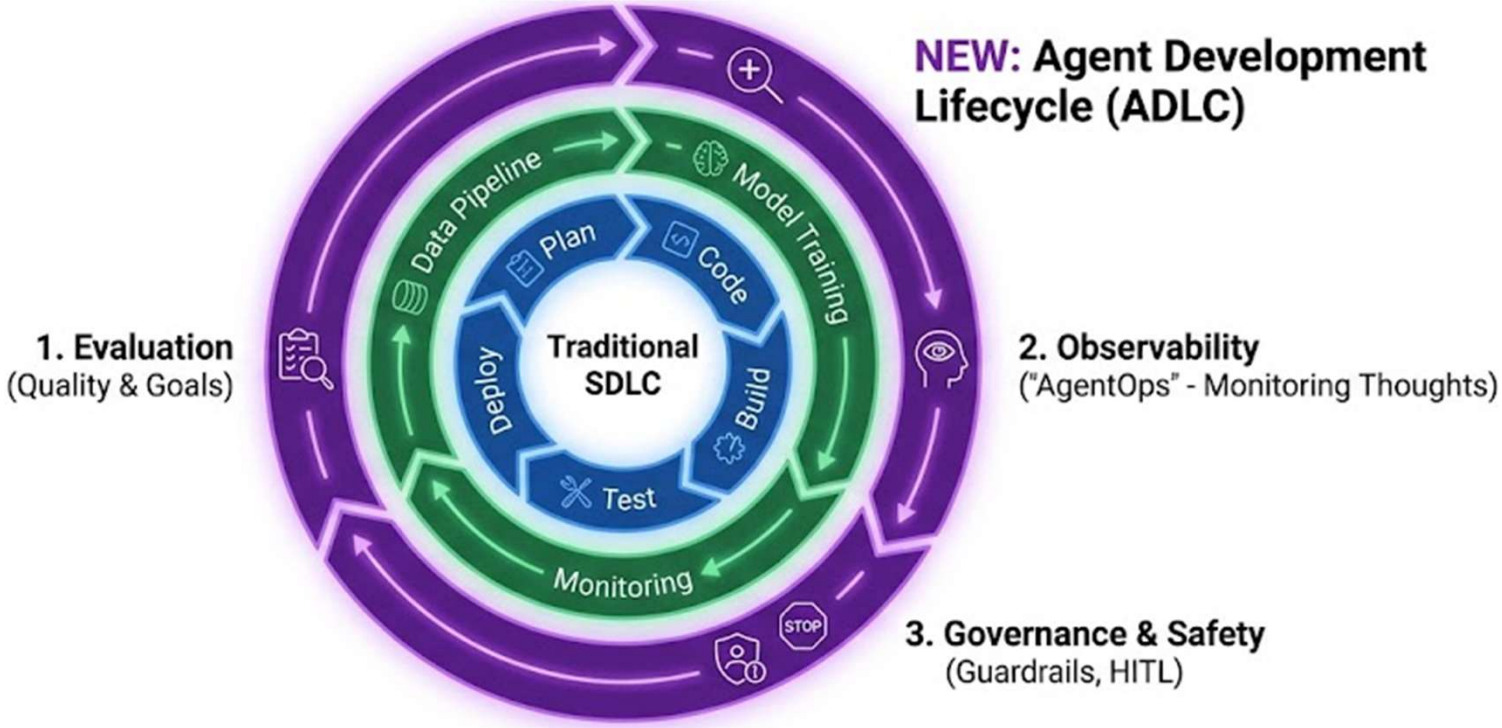


The Solution: The Agent Development Lifecycle (ADLC)

We must evolve our **SDLC**

ADLC = SDLC + MLOps + New "Cognitive" Practices

It is a new, formal process built to manage probabilistic software



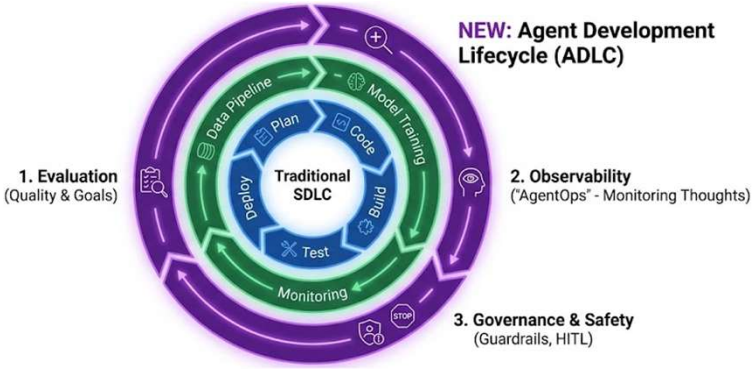
The Agent Development Lifecycle (ADLC): A 7-Step Guide

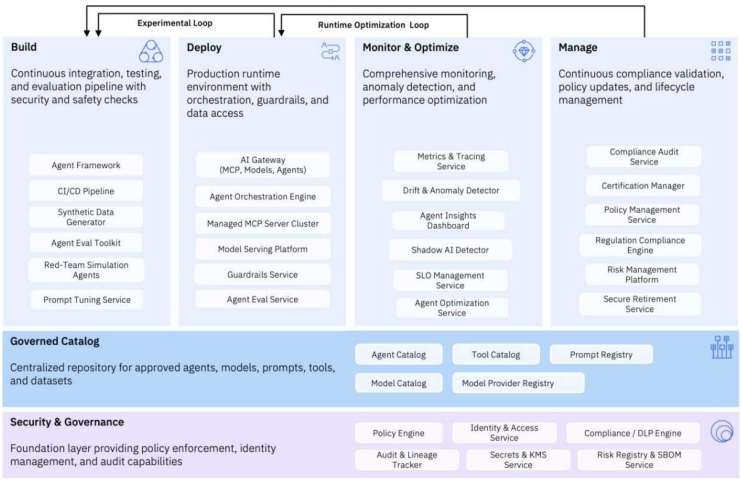
Defined the **'Probabilistic Shift'** and **why** our **traditional SDLC** is **no longer sufficient** for **managing AI agents**

The **problem** is that **traditional tools can't test** for **'reasoning'** or **monitor** for **'confusion**'

The solution is the **Agent Development Lifecycle or ADLC.**

Walk through its **7 key phases from** the **initial idea** to a **fully monitored, enterprise-grade product**



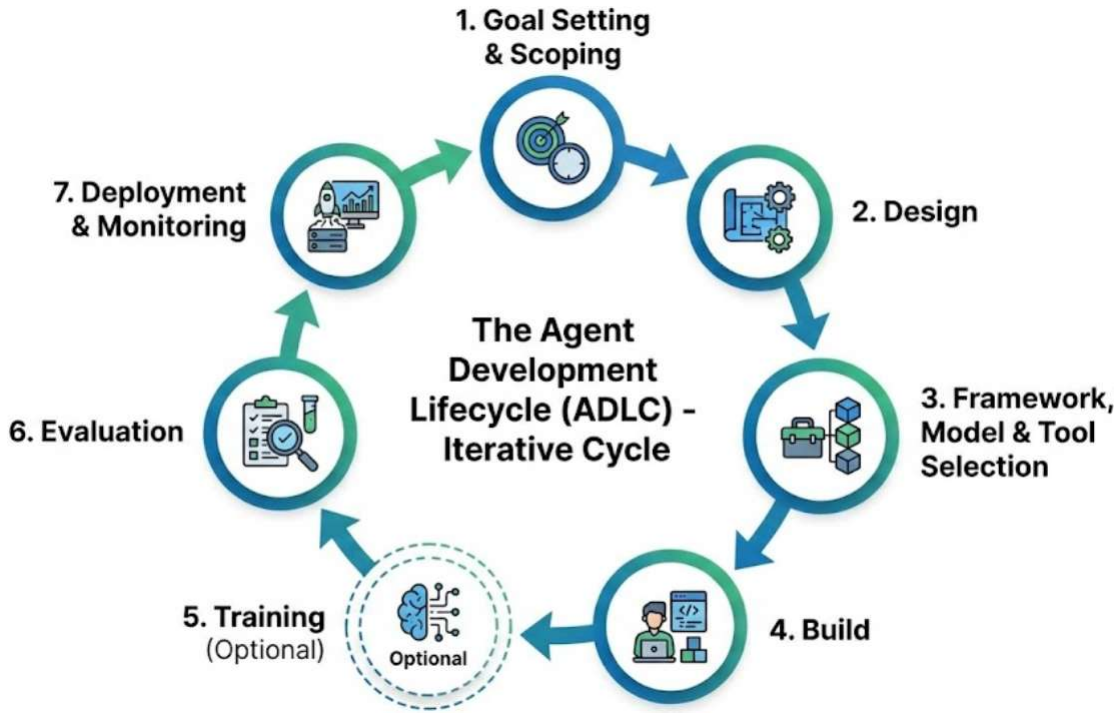


The Agent Development Lifecycle (ADLC) Overview

An iterative cycle, similar to a modern SDLC or MLOps loop

The 7 Phases:

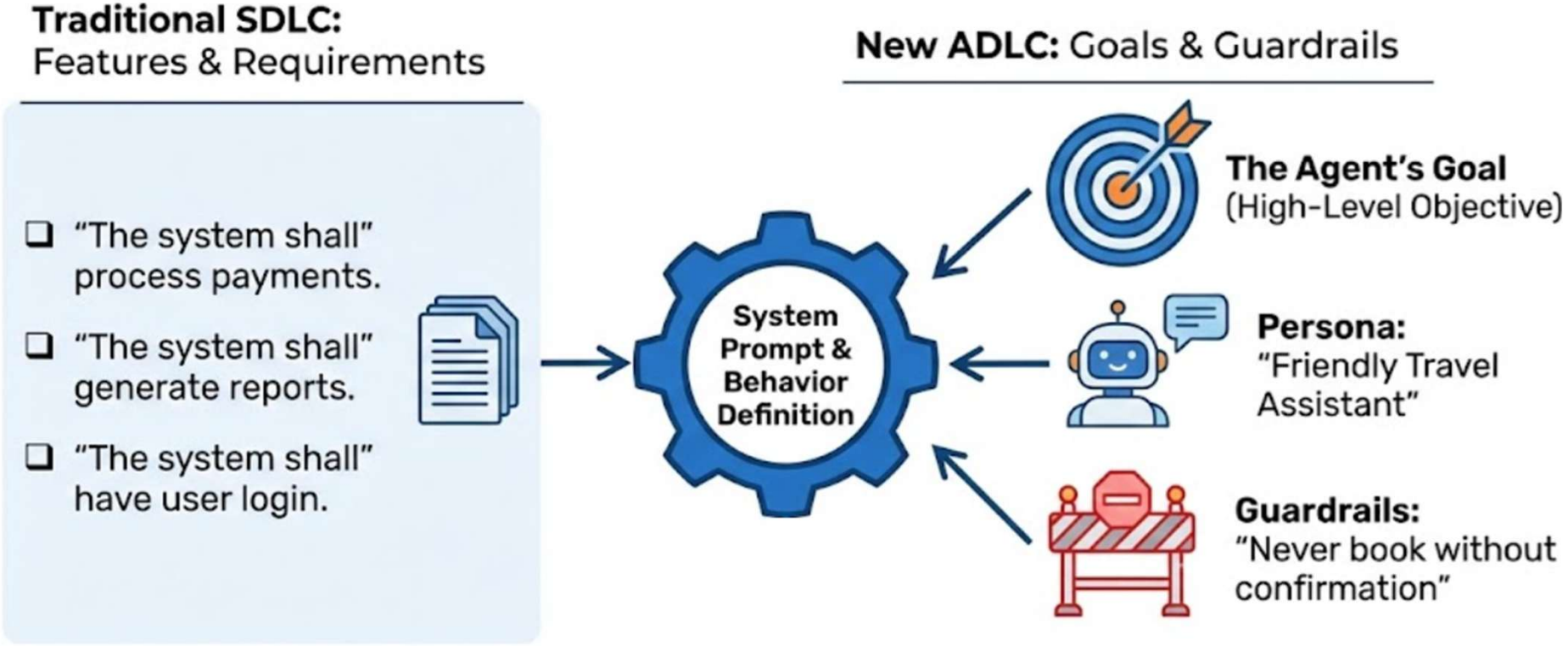
1. Goal Setting & Scoping
2. Design
3. Framework, Model & Tool Selection
4. Build
5. Training (Optional)
6. Evaluation
7. Deployment & Monitoring



We will now look at each one in detail

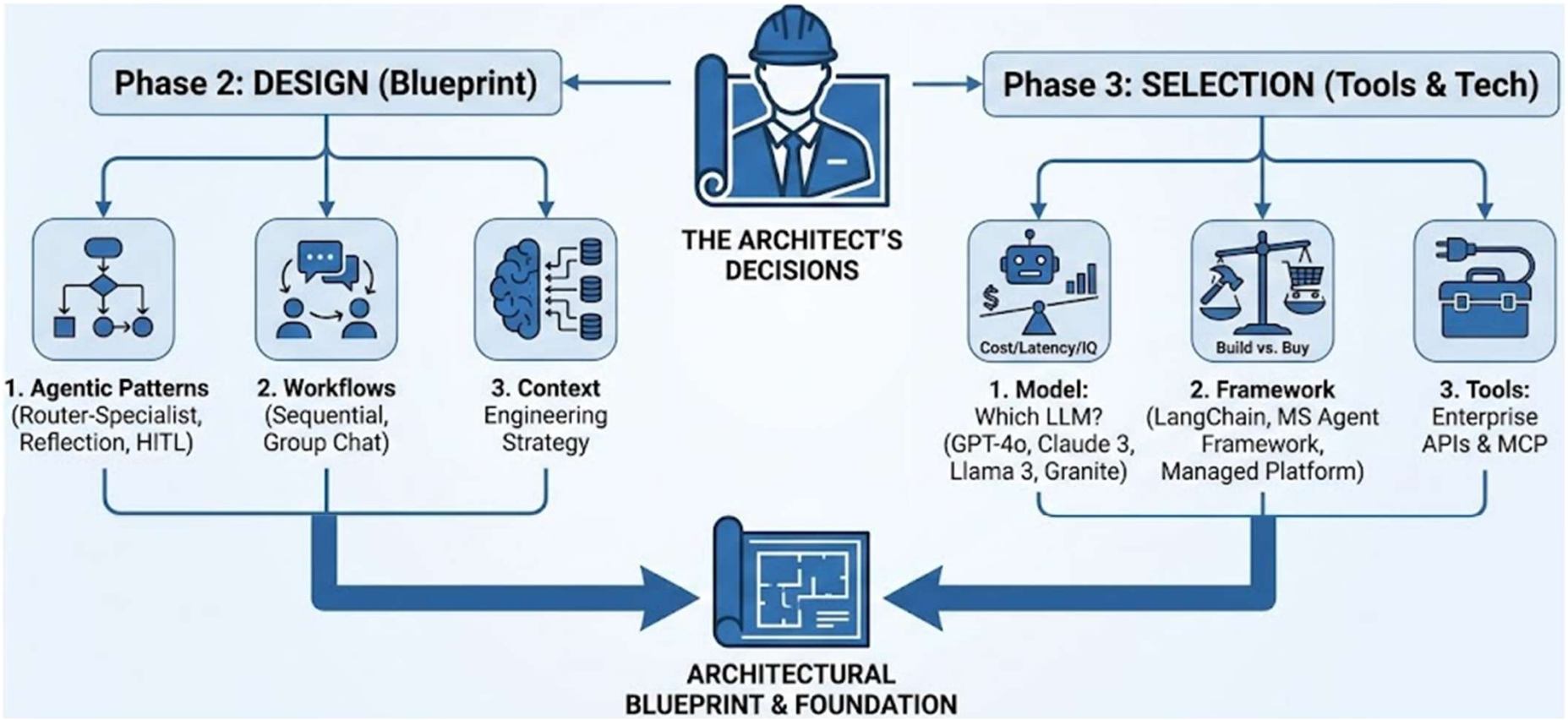
Phase 1: Goal Setting & Scoping

Task: Define what the agent is supposed to do and what it is not

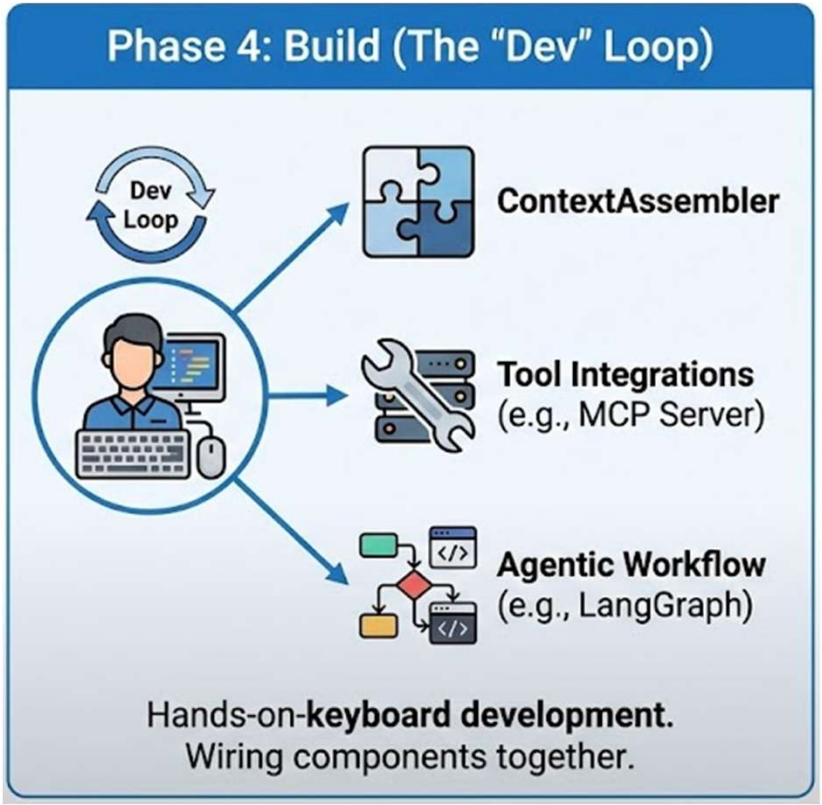


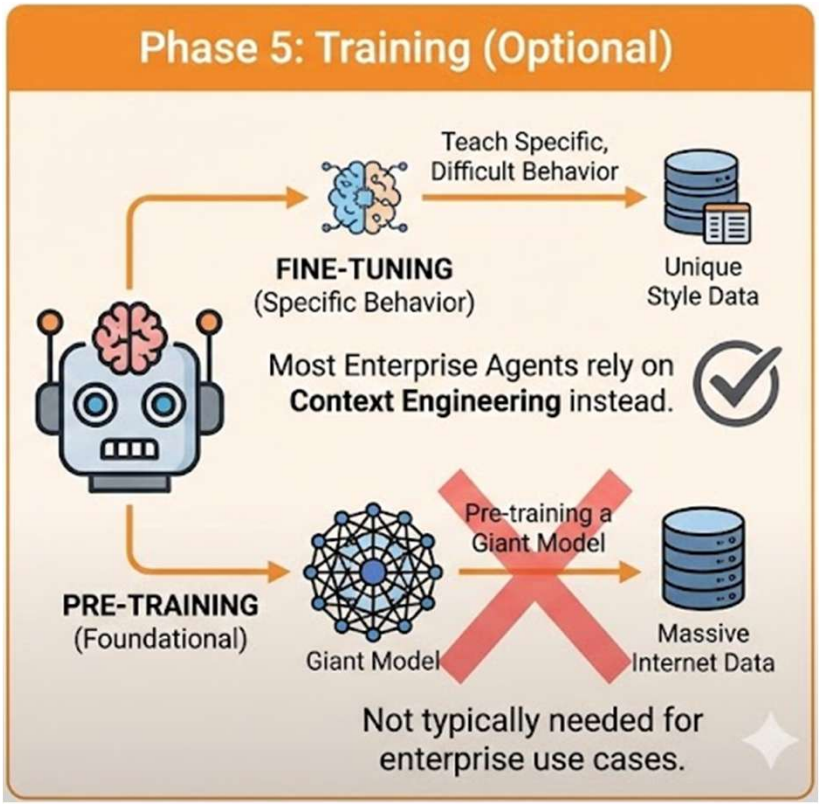
The "goal" becomes a technical part of the system prompt

Phase 2 & 3: Design, Framework & Model Selection

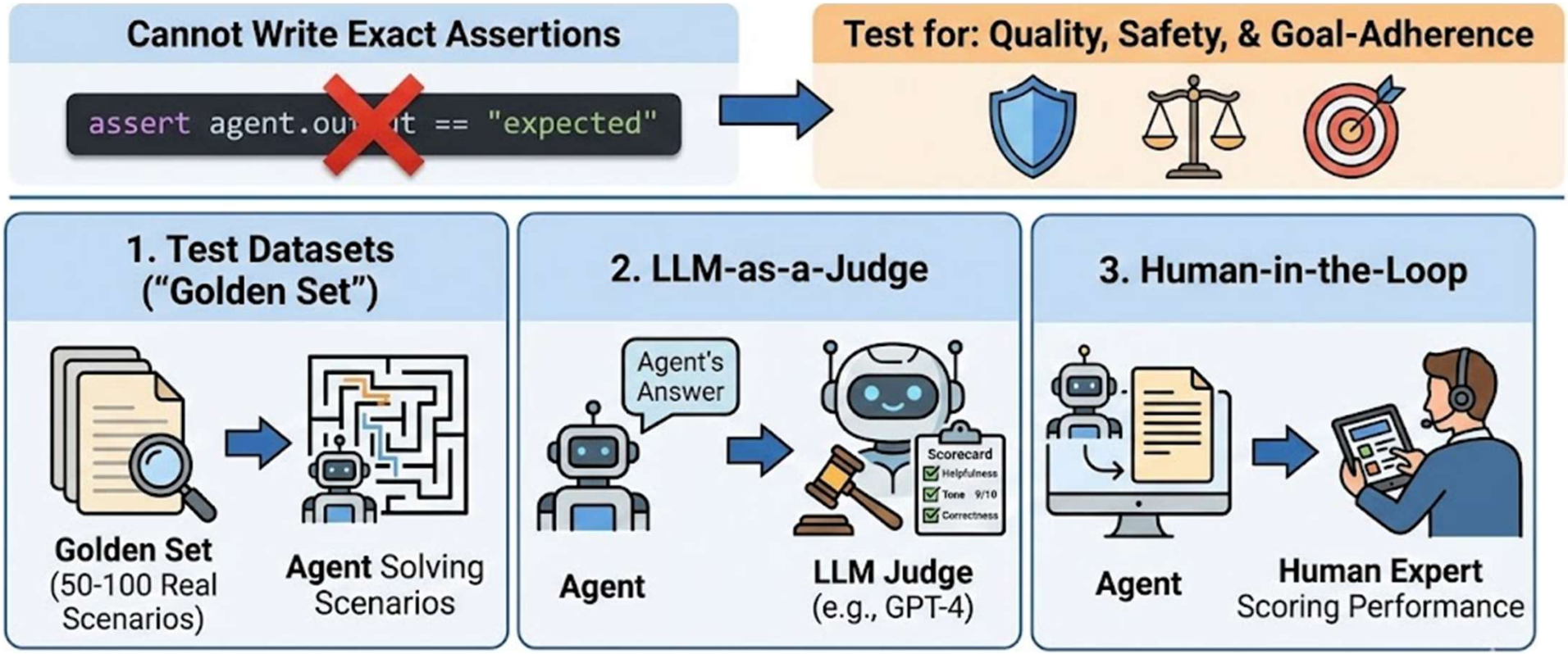


Phase 4 & 5: Build & Training

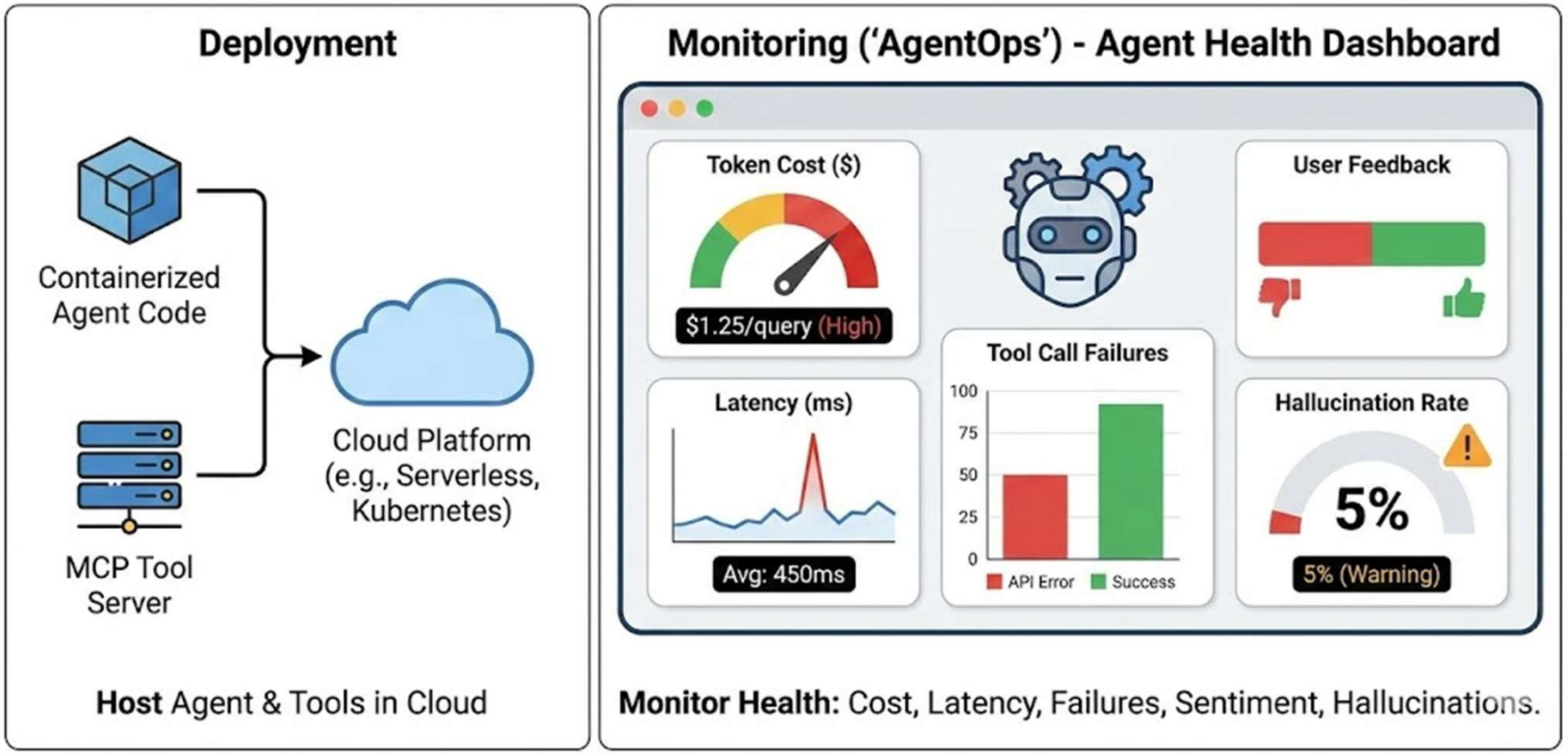




Phase 6: Evaluation (The New "Testing")



Phase 7: Deployment & Monitoring ("AgentOps")

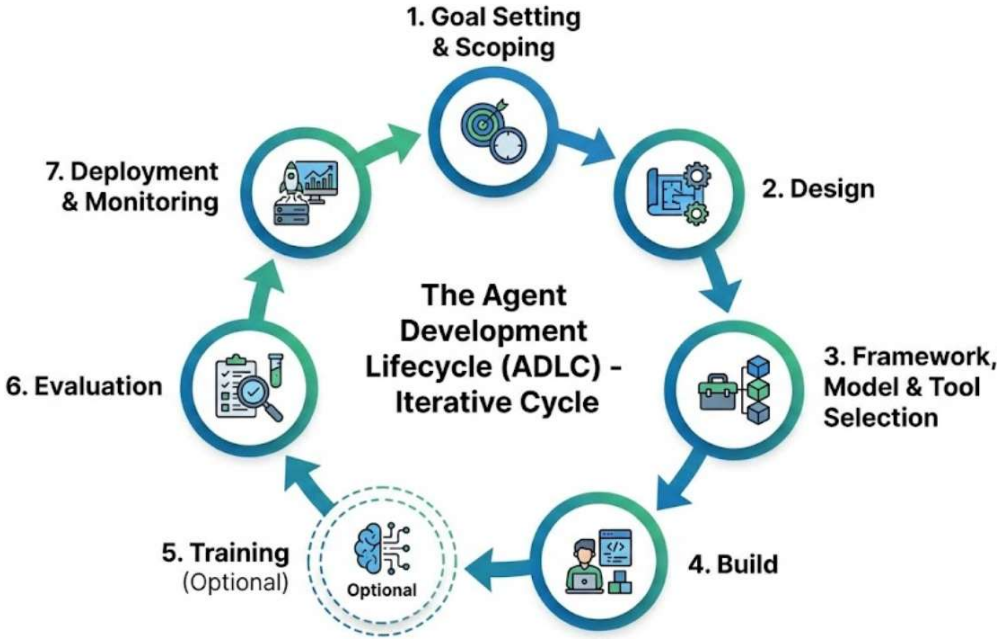


Summary & Next Up: Deep Dives

The ADLC is a **7-step iterative process** for managing probabilistic software

The Hardest Parts: Are the new phases
Phase 6: Evaluation (How do we test it?)
Phase 7: AgentOps (How do we monitor it?)

Next Lectures:
We will do a deep dive on these
2 critical new topics

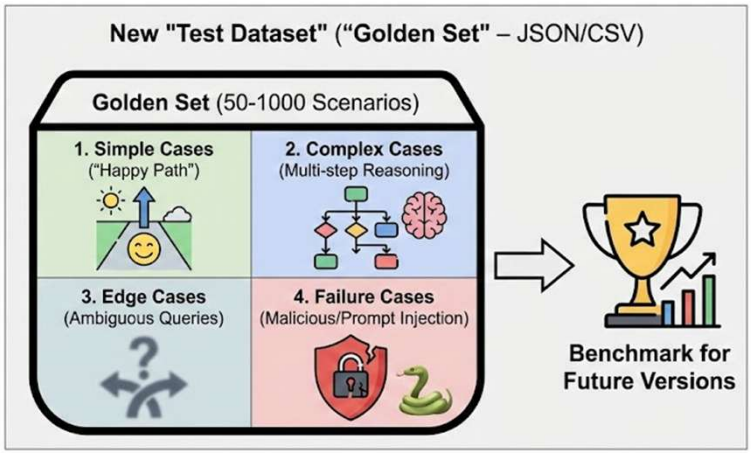


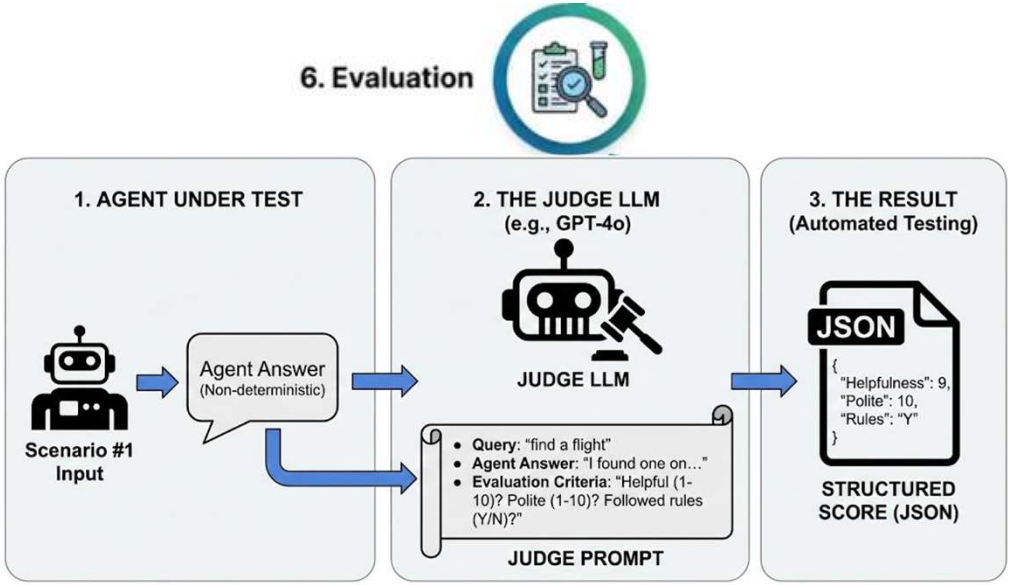
Deep Dive: The Challenge of "Evaluation"

We identified **'Evaluation'** and **'Monitoring'** as the 2 most critical new phases

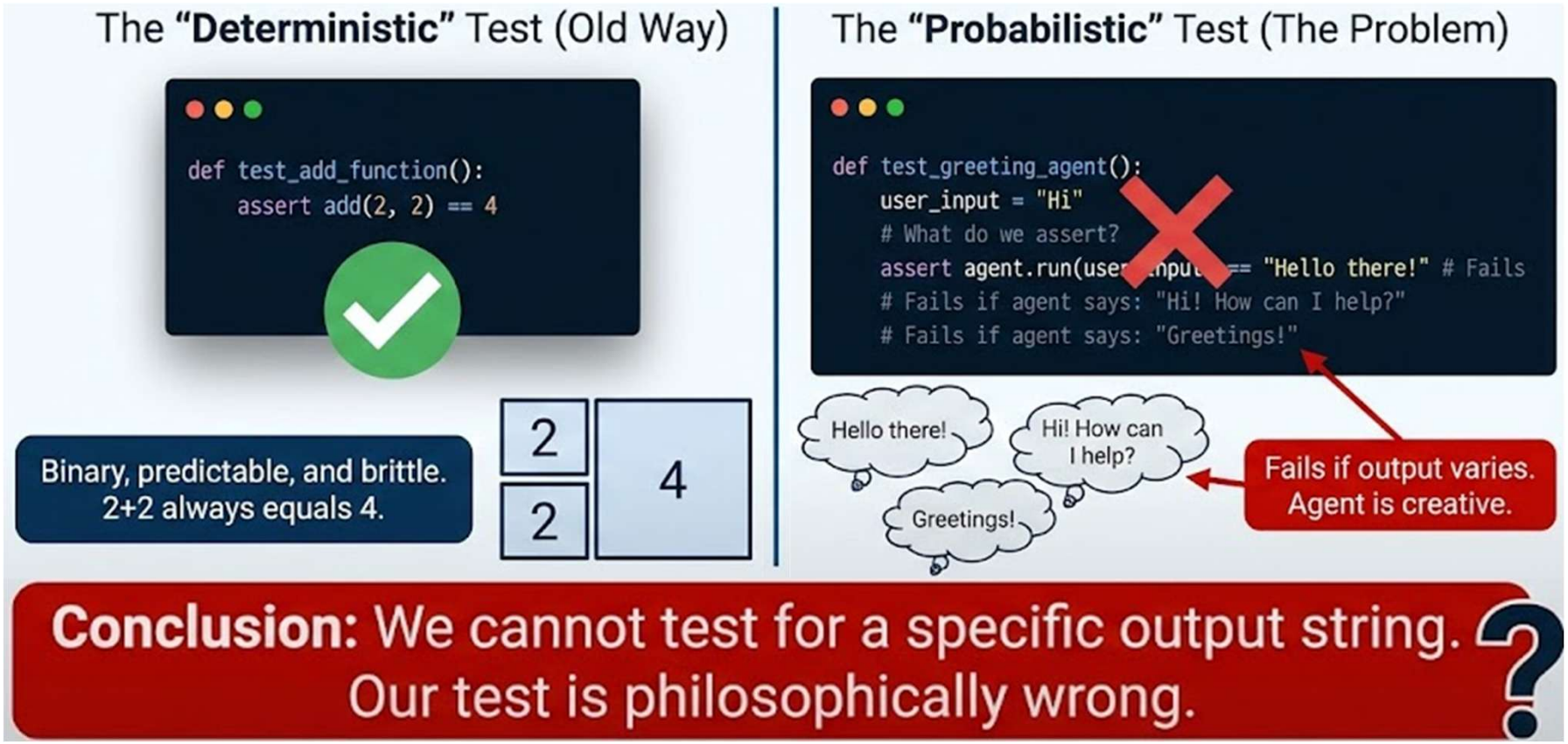
Why? Because as architects and engineers, our **instinct** is to **write tests**

How do we actually test a non-deterministic agent?
How do we build a CI/CD pipeline when the output isn't predictable?

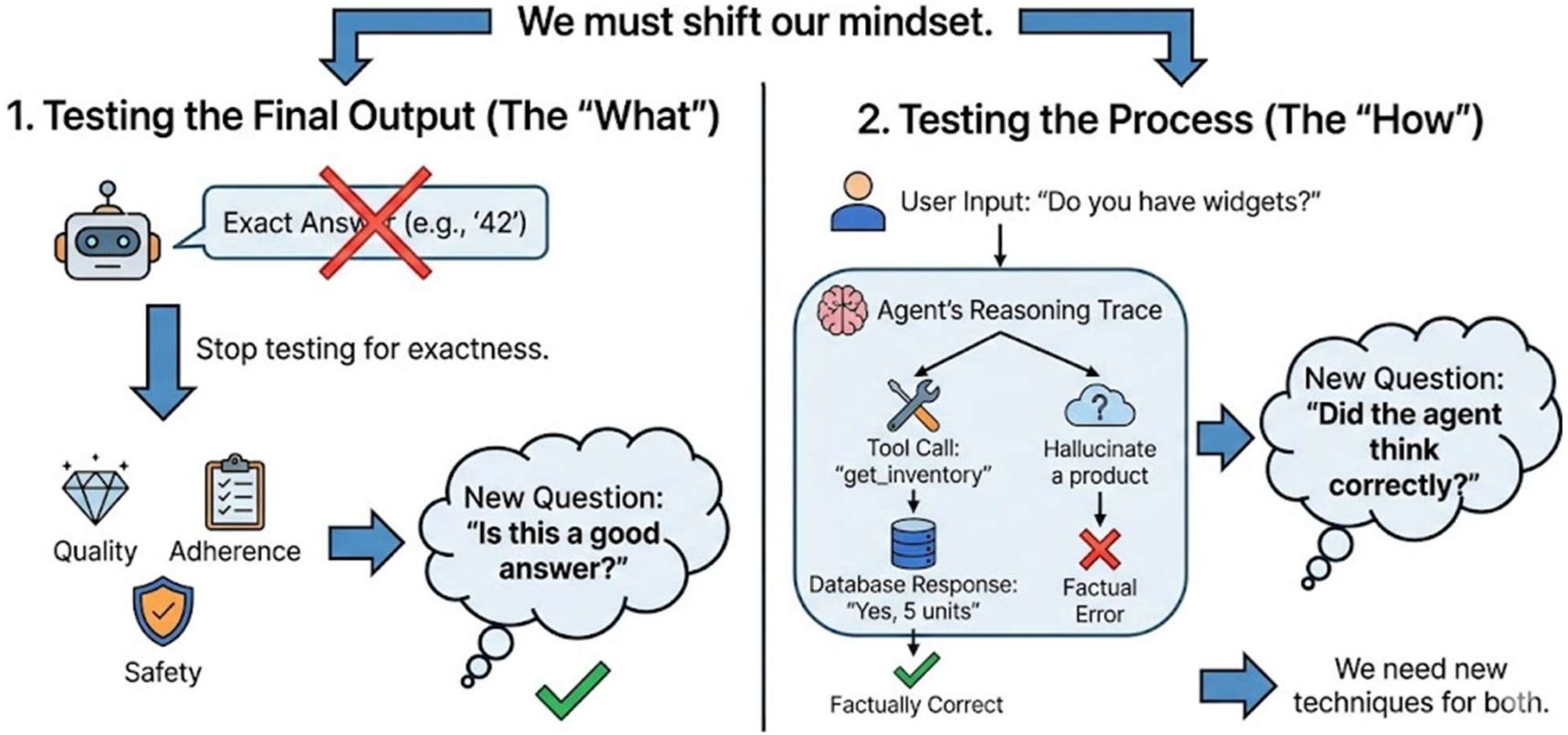




Why Traditional Unit Tests Fail ?



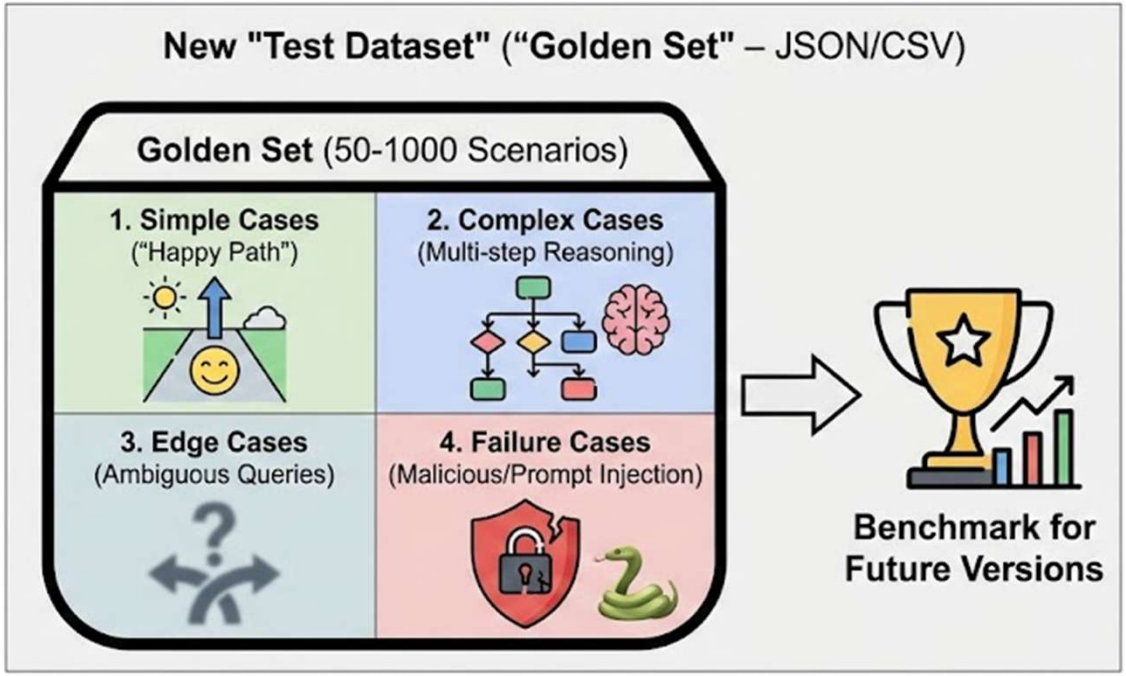
The New Paradigm: Testing "Process" vs. "Output"



Technique 1: Creating a "Test Dataset"

Your new "**Test Suite**" is no longer a .py file, It's a "**golden set**" of scenarios

What it is: A curated set of 50, 100 or 1000 real-world scenarios for the agent

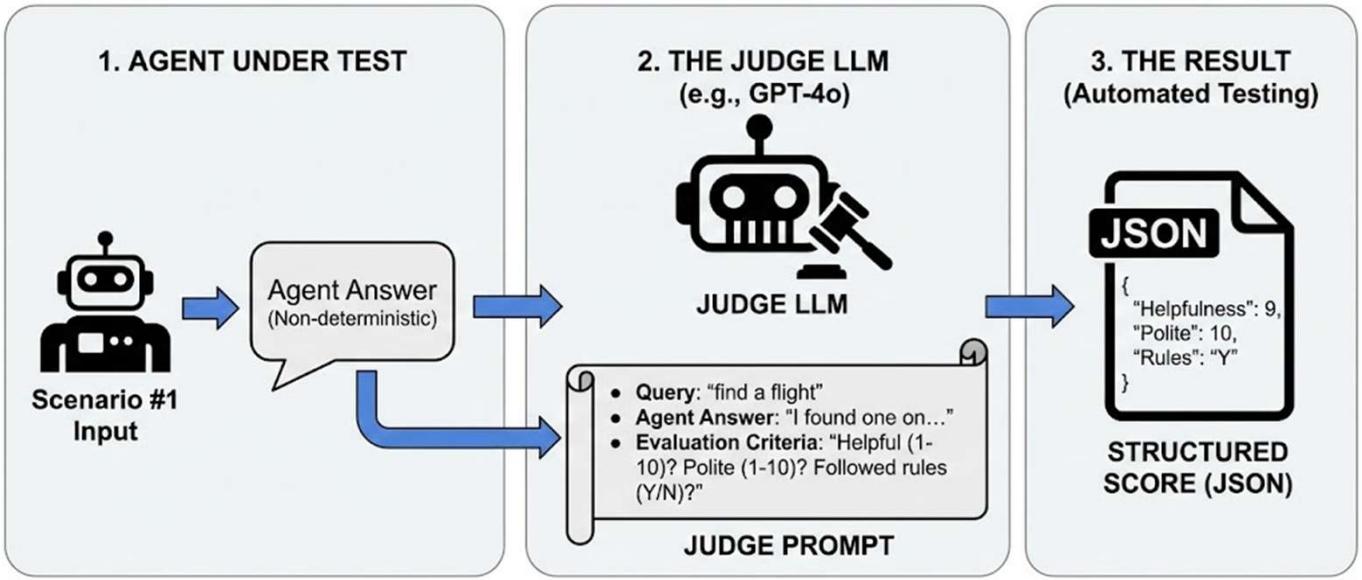


This **dataset becomes** the **benchmark** for all future versions of your a

Technique 2: "LLM-as-a-Judge" (The Evaluator)

The Problem:100 scenarios, but who grades the 100 non-deterministic answers?

The Solution: We use another, more **powerful LLM** as the "**Judge**"



This is **how** you **automate testing** for "**quality**"

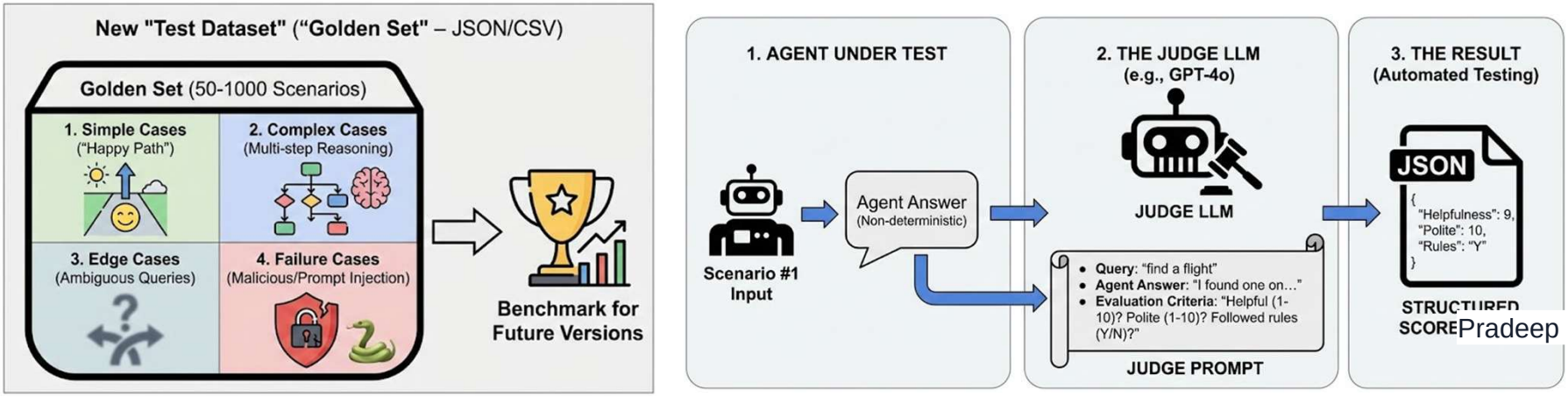
Summary: The New QA Pipeline

1. Unit Tests (Deterministic):
For your tools. (Your add(2, 2) function must still have a unit test)

2. Evaluation Pipeline (Probabilistic):
Test Dataset: A "Golden Set" of scenarios
Agent Run: The agent runs against the full dataset
LLM-as-a-Judge: The "Judge" grades the agent's output for quality.
Trace Analysis: We "white-box" test the process to ensure correct tool use



This is the **new CI/CD pipeline** for **agents**



Deep Dive: "AgentOps" (Observability & Monitoring)

Built a new **'QA' process** called **Evaluation**, which lets us **test** our **agent before deployment**

Must answer the next **question**: **What happens after we deploy?**

How do we monitor it?
How do we know if it's healthy, if it's confused, or if it's costing us a fortune?

This is a **new, specialized field** that is **emerging**, and we call it **'AgentOps'**

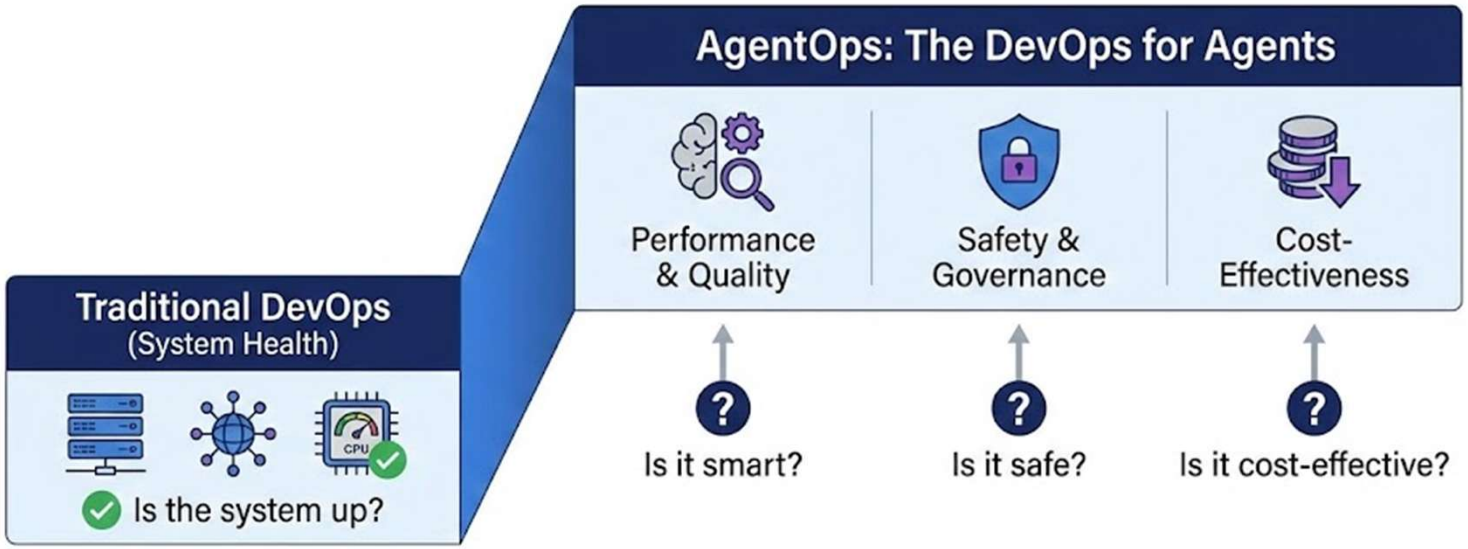


What is AgentOps?

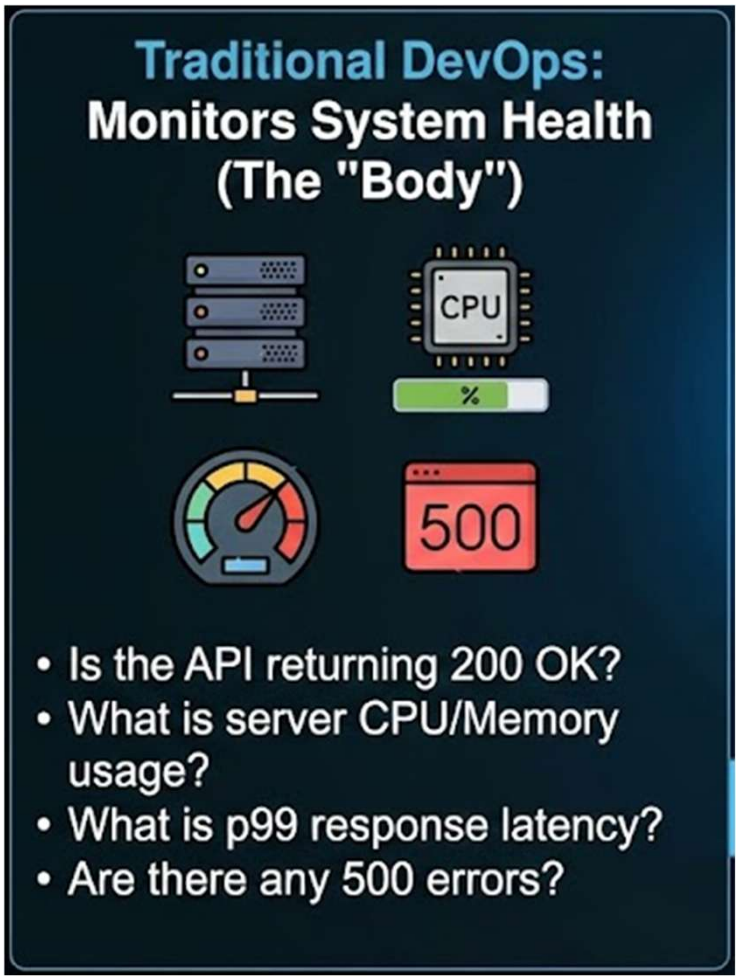
It is the "**DevOps for Agents**"

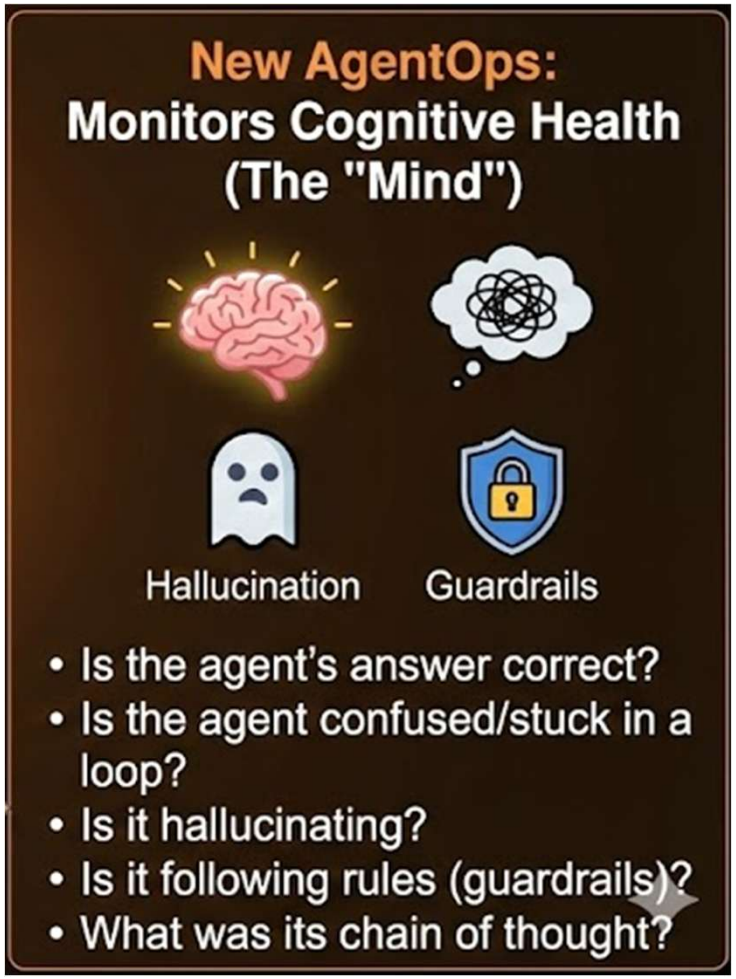
Set of **practices** and **tools** for **managing**, **monitoring** and ensuring the **performance**, **quality** and **cost of AI agents** in a **live production** environment

It **extends traditional DevOps** (for system health) to include new practices for **monitoring agent health**



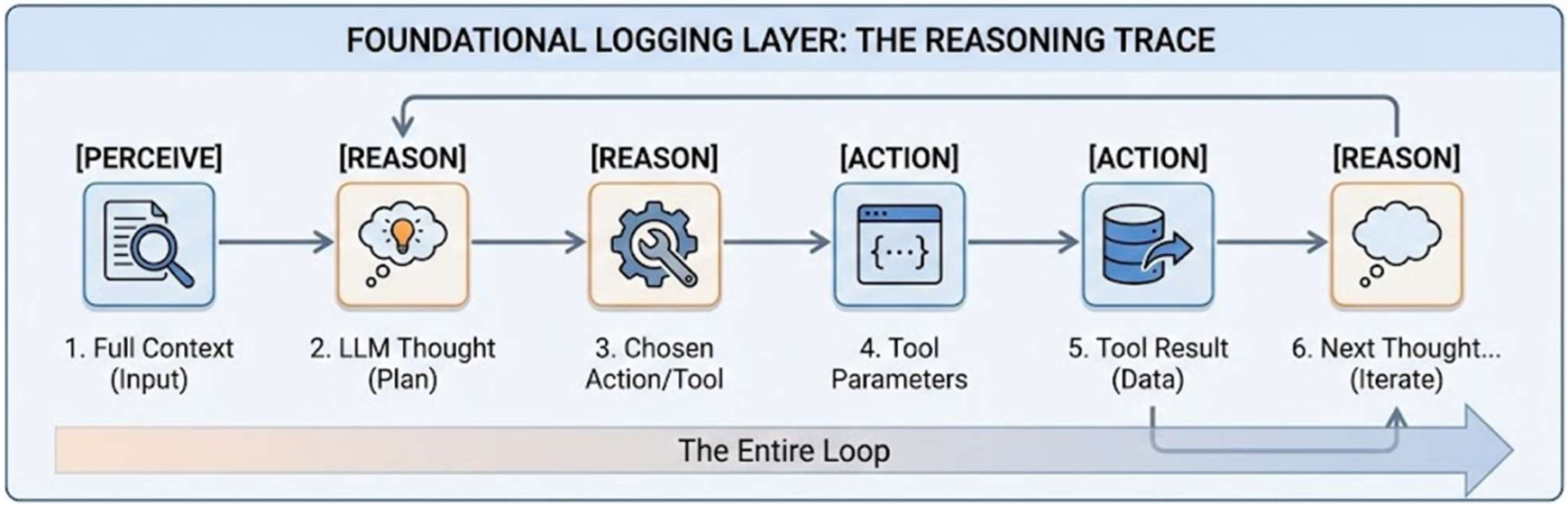
DevOps vs. AgentOps: System Health vs. Cognitive Health





The Foundation of AgentOps: Traceability

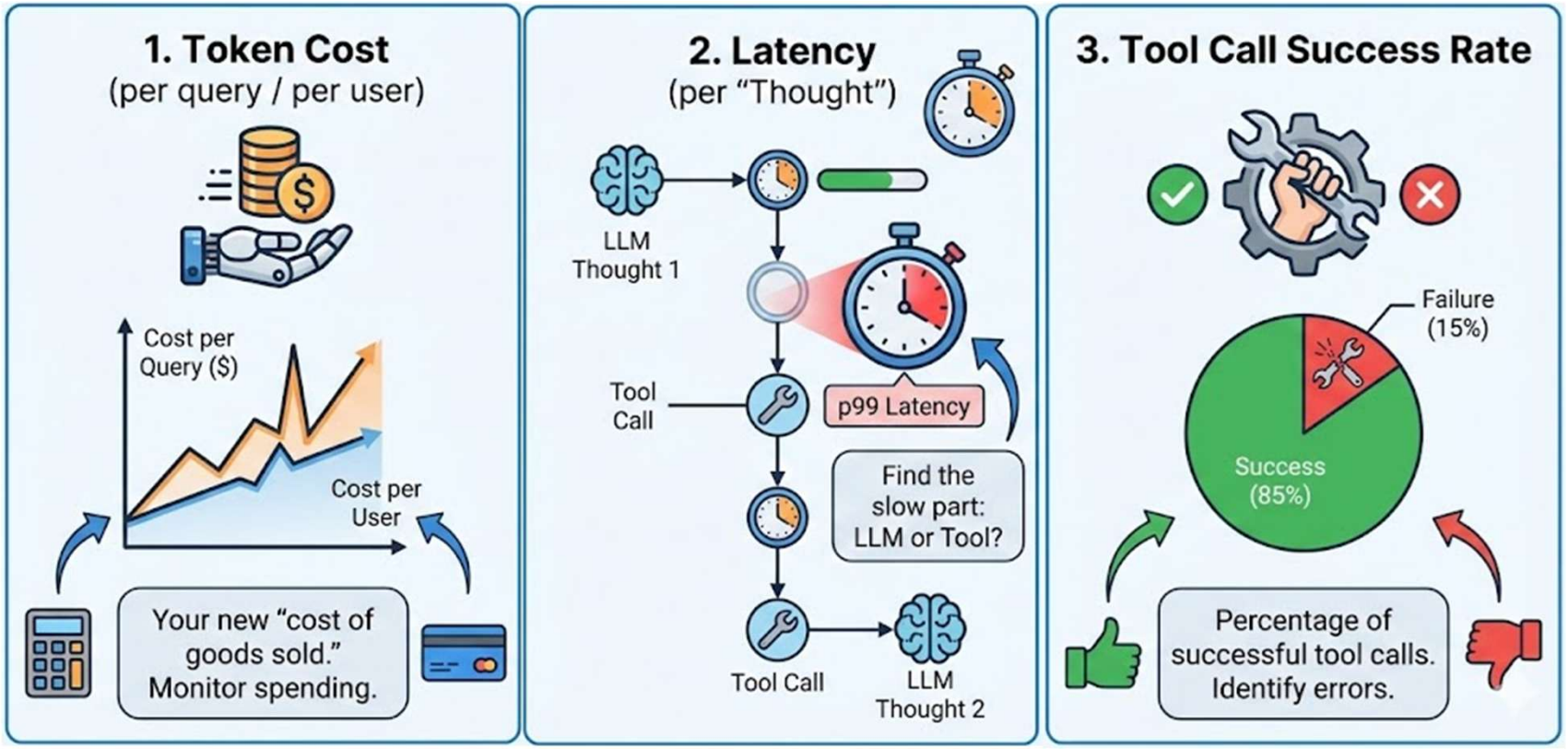
The Problem: A "500 error" is easy to log. How do you log "confusion"?
The Solution: You must log the agent's entire reasoning trace



Without this, **all other metrics** are **impossible** to **debug**

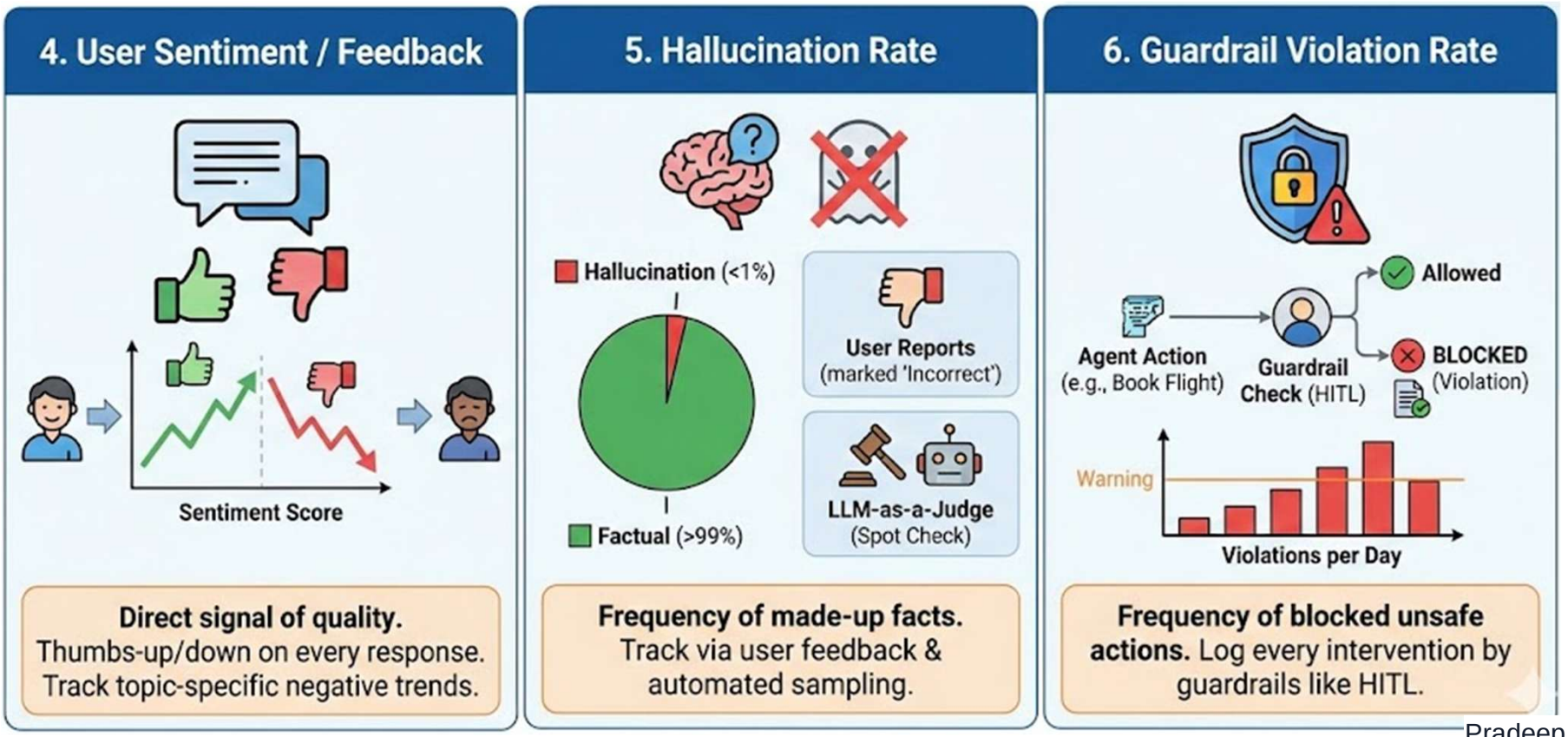
Key AgentOps Metrics (Part 1): Performance & Cost

These are the new **"hard" metrics** you add to your dashboard

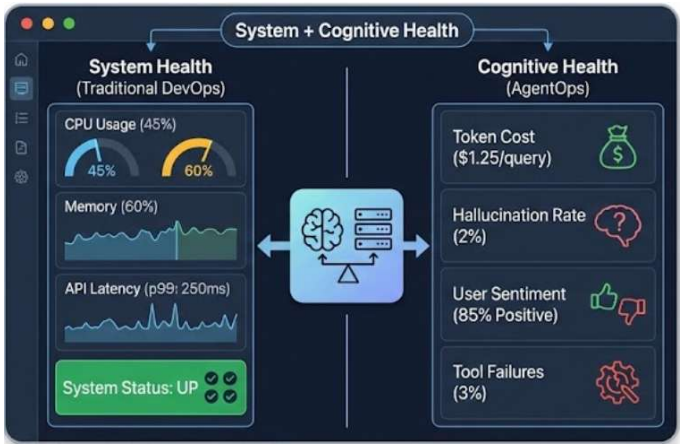


Key AgentOps Metrics (Part 2): Quality & Safety

These are the new "**cognitive**" metrics that **measure quality**



The New Dashboard: System + Cognitive Health



Your new **AgentOps** dashboard must show both

AgentOps is the new "**DevOps for Agents**"

It **monitors Cognitive Health**, **not just System Health**

It is **built** on **Traceability** (**logging** the "**thoughts**")

It adds new metrics:
Cost, **Tool Failures**, **User Sentiment** and **Hallucinations**

Next Up: How do we enforce the rules?



Enterprise Considerations: LLM Guardrails for Data Leakage, Prompt Injection, Security & Governance

Learned **how** to **'test'** our **probabilistic agents** with an **Evaluation pipeline** and **how** to **'monitor'** them in **production** with **AgentOps**

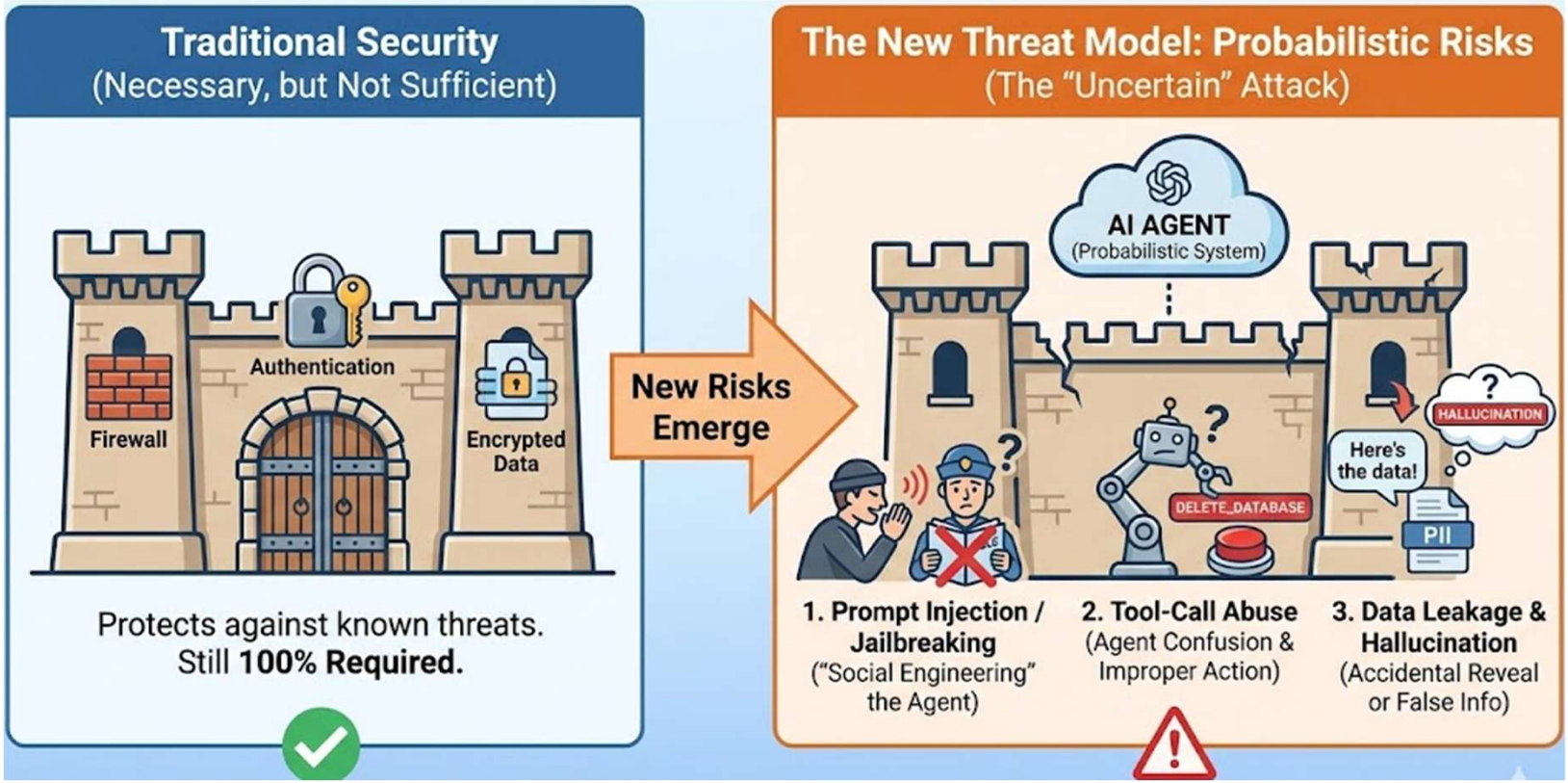
How do we **prevent** things from **going wrong** in the **first place?**

The new **security-first mindset**, the new **threat models**, and the **'LLM Guardrails'** we must build to **protect** our **enterprise**



The New Threat Model: Probabilistic Risks

Our **traditional security** is still 100% necessary, but it is **no longer sufficient**
We now have new **"probabilistic" attack vectors** and **risks**



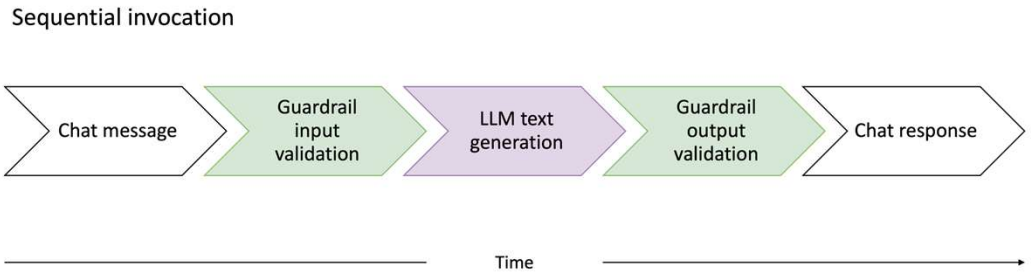
The Solution: Defining LLM Guardrails

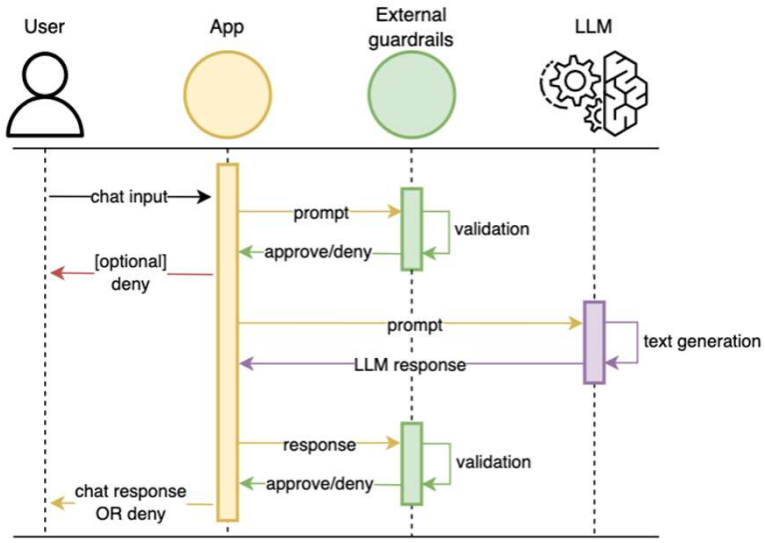
Definition: LLM Guardrails are a set of pre-defined rules, filters and policies that are programmatically enforced around the LLM

The Goal: To constrain the non-deterministic "brain" to be in a specific format or context and to validate its every response

They enforce:
Structure: The output MUST be valid JSON
Type: The output MUST NOT contain toxic language
Quality: The output MUST be fact-checked a source

They are the "**Firewall**" for the **agent's "Mind"**

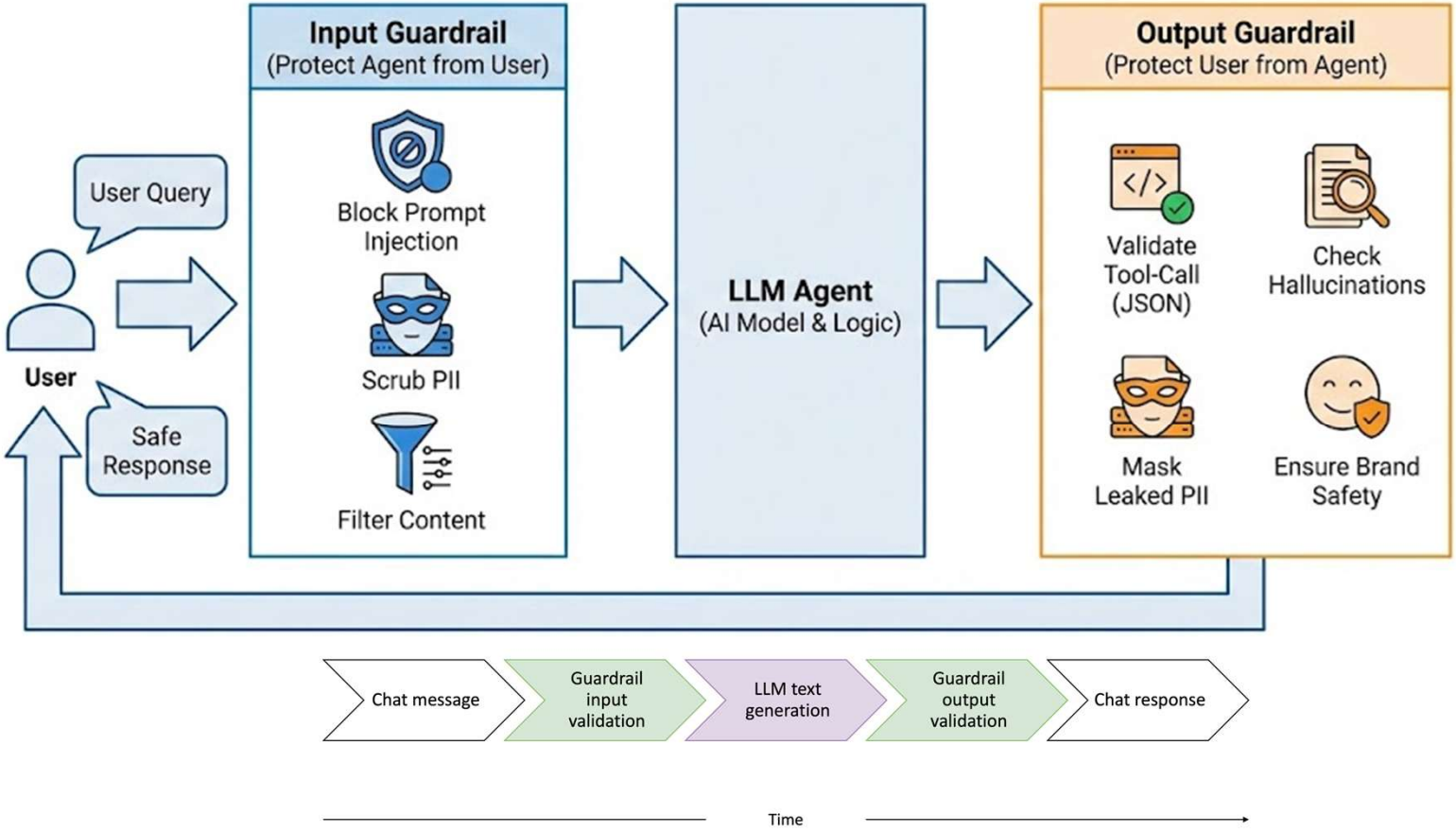




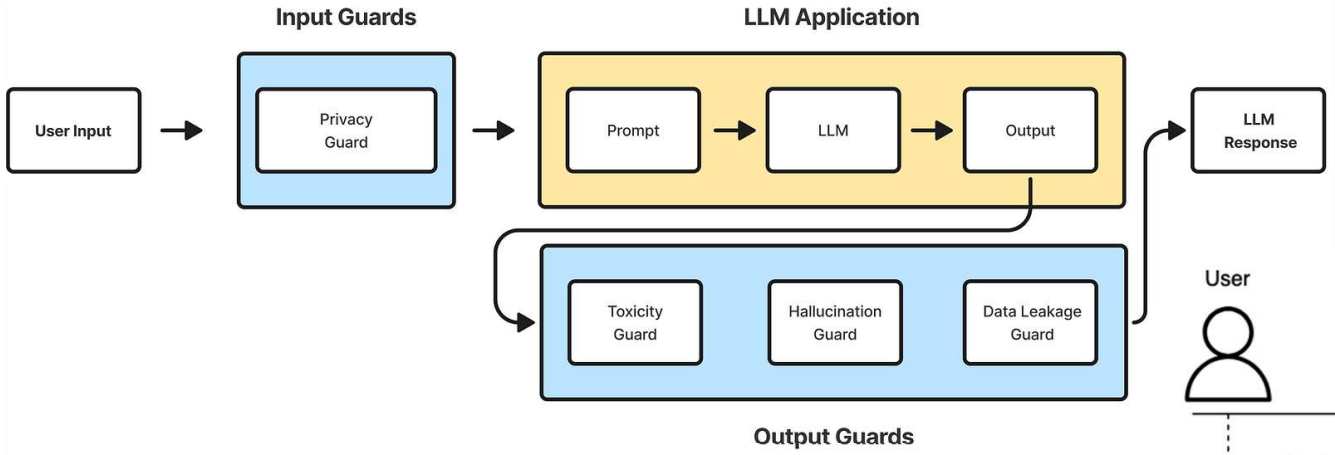
https://aws.amazon.com/blogs/machine-learning/build-safe-and-responsible-generative-ai-applications-with-guardrails/

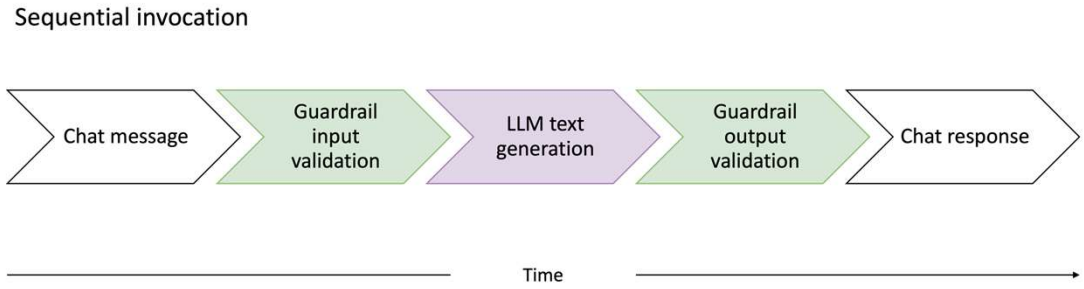
How Guardrails Work: Input vs. Output

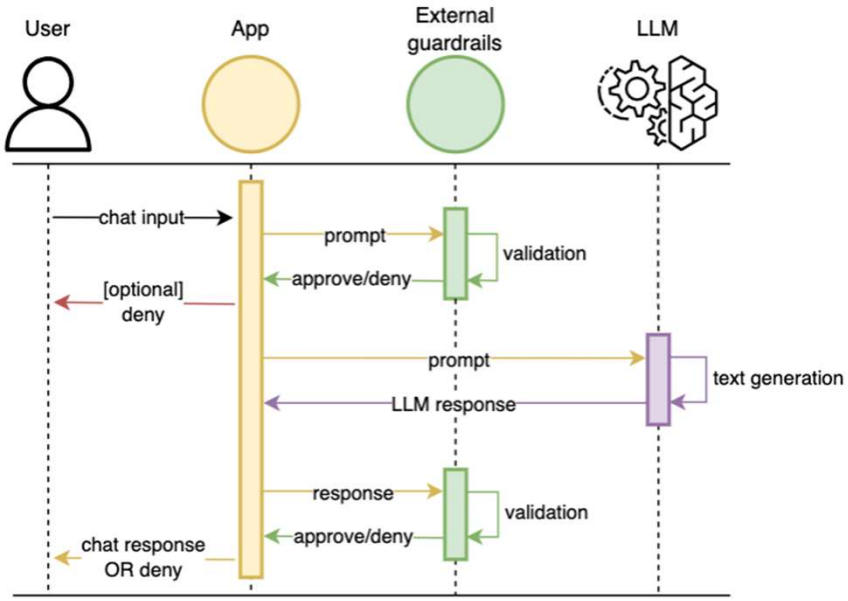
Architecturally, we can **place guardrails** at **2 points in the data flow**



How Guardrails Work: Input vs. Output



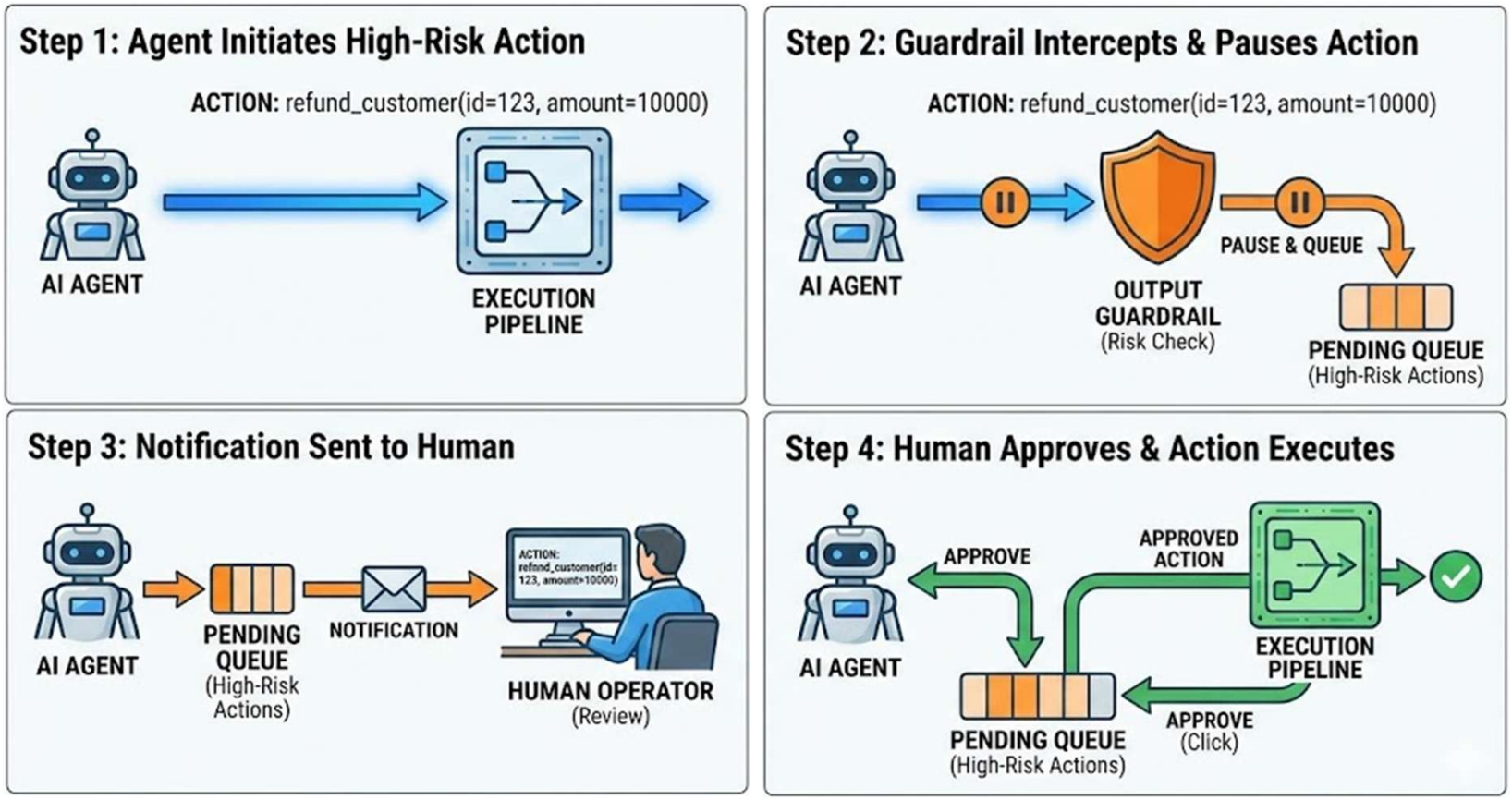




https://aws.amazon.com/blogs/machine-learning/build-safe-and-responsible-generative-ai-applications-with-guardrails/

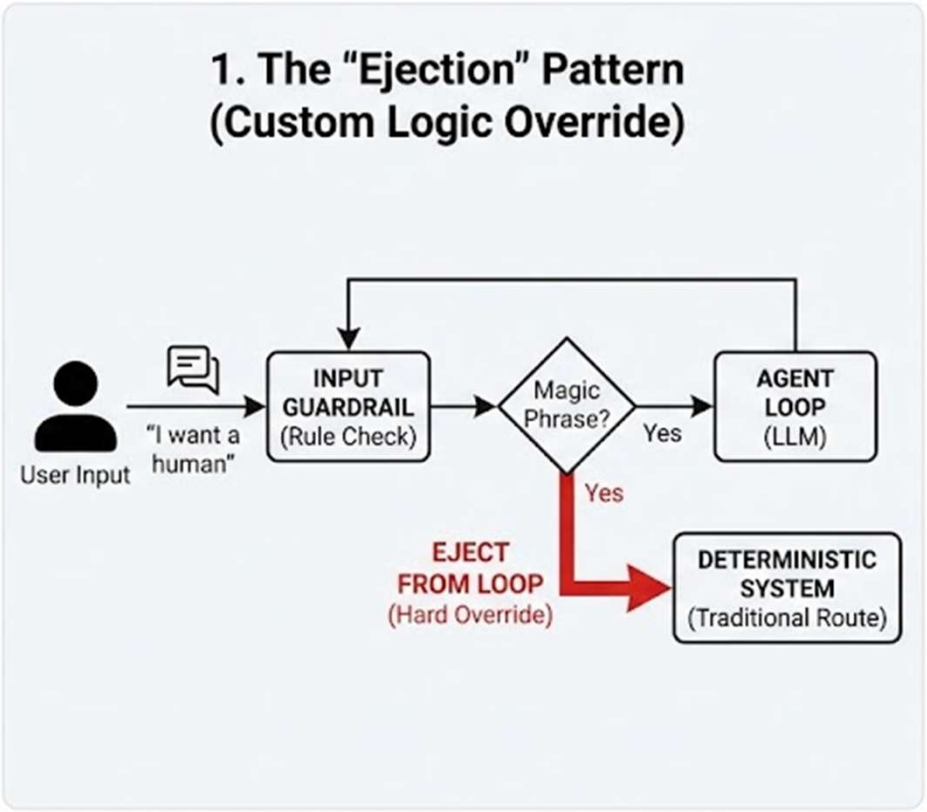
Security Pattern 1: Human-in-the-Loop (HITL)

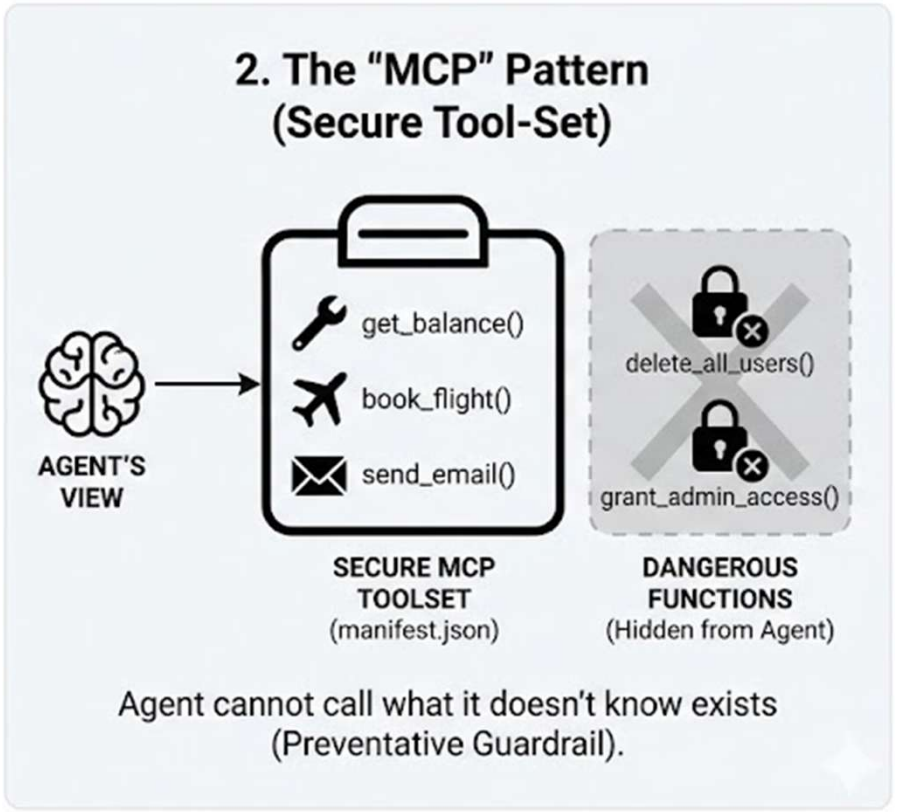
We build an **output guardrail** that **checks** for **"high-risk" actions**



Non-negotiable pattern that can **move money** or **modify critical data**

Security Pattern 2: Ejection & Secure Tooling (MCP)



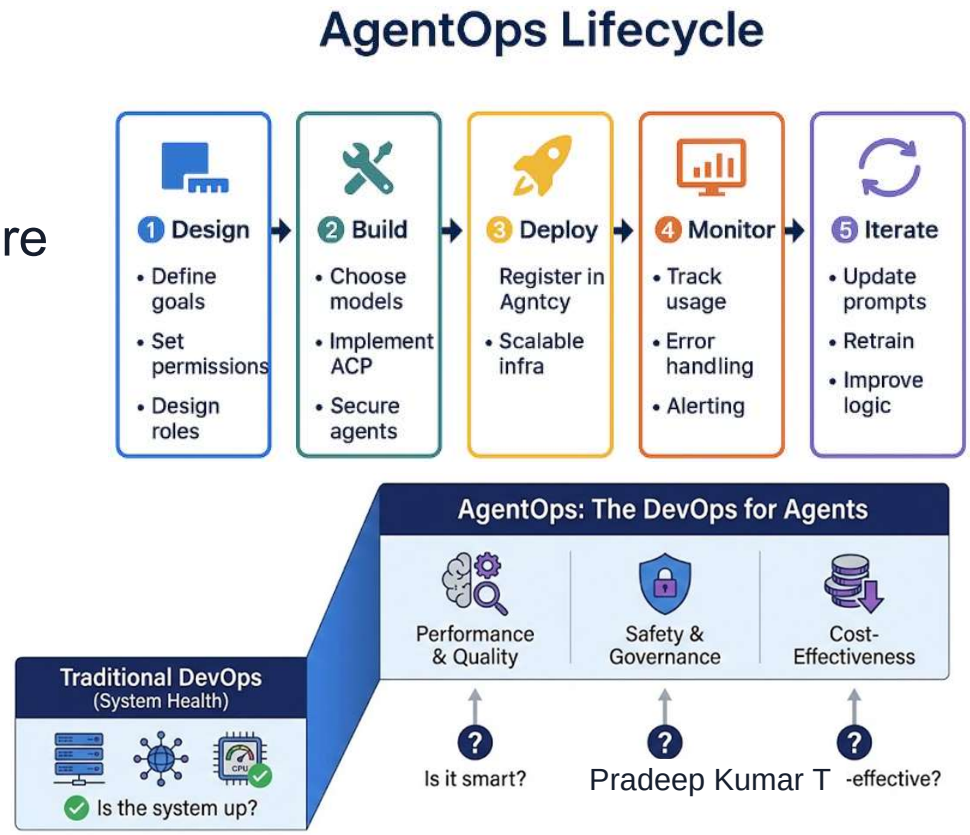


The "Big Question": Governance & Liability

This is a **C-suite**, **legal** and **compliance question**
Who is **liable** when an **agent makes** a **multi-million dollar mistake?**

The Architect's Role (Our Answer):
Traceability is our defense
AgentOps is the key
An immutable, auditable log of the agent's entire reasoning trace
Prove why the agent made the decision it did and what guardrails it passed or failed

Governance is **impossible without Observability**



Summary: Security is a Non-Negotiable Layer

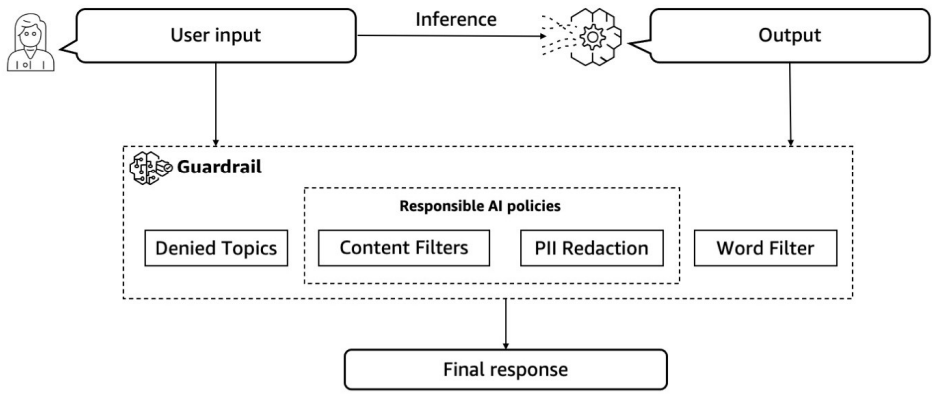
We have a **New Threat Model** (**Injection**, **Abuse**, **Leakage**)

Our **solution** is a **layer** of **LLM Guardrails** (**Input & Output**)

Implement **these guardrails** with **specific Security Patterns** (HITL, Ejection, MCP)

We **manage** our **risk** with **Governance** and **Traceability**

Next Up: We will apply this security mindset to the lifecycle of our tools



The "Tool" Lifecycle: Managing MCP Servers

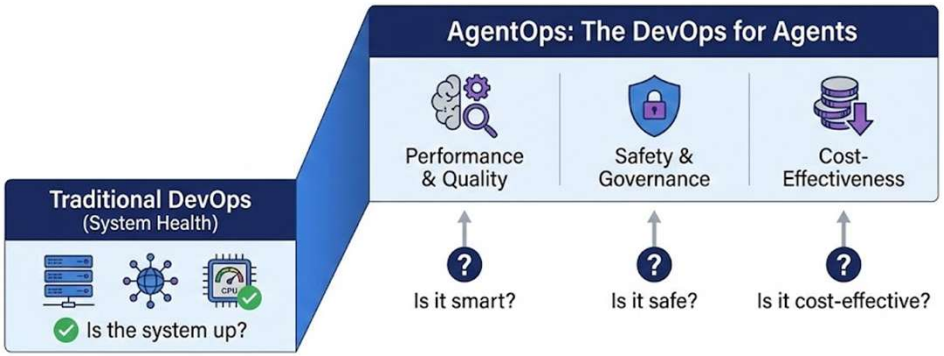
Agent's **development**, its **evaluation**, its **security**, defined the **'ADLC'**

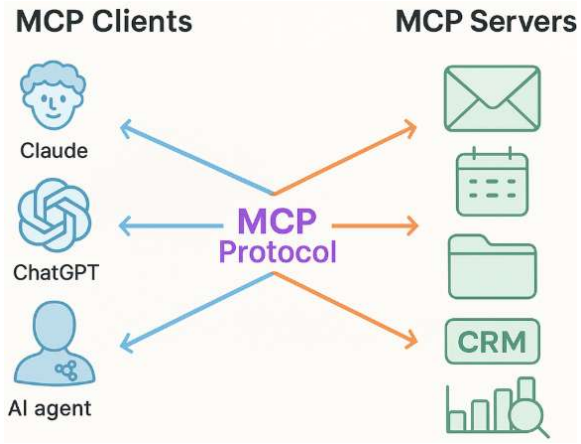
But an **agent** is only **half of** the **equation**. The **other half** is **its tools**

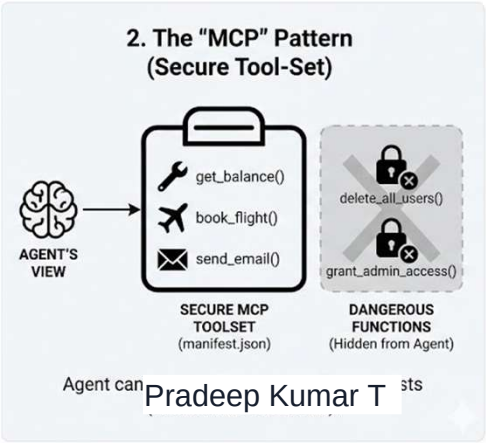
An **agent** is **useless without** the **deterministic**, **reliable tools** it **calls**

These are our **MCP Servers** that are enterprise APIs; they have their own **lifecycle** of **development**, **deployment** and **management**

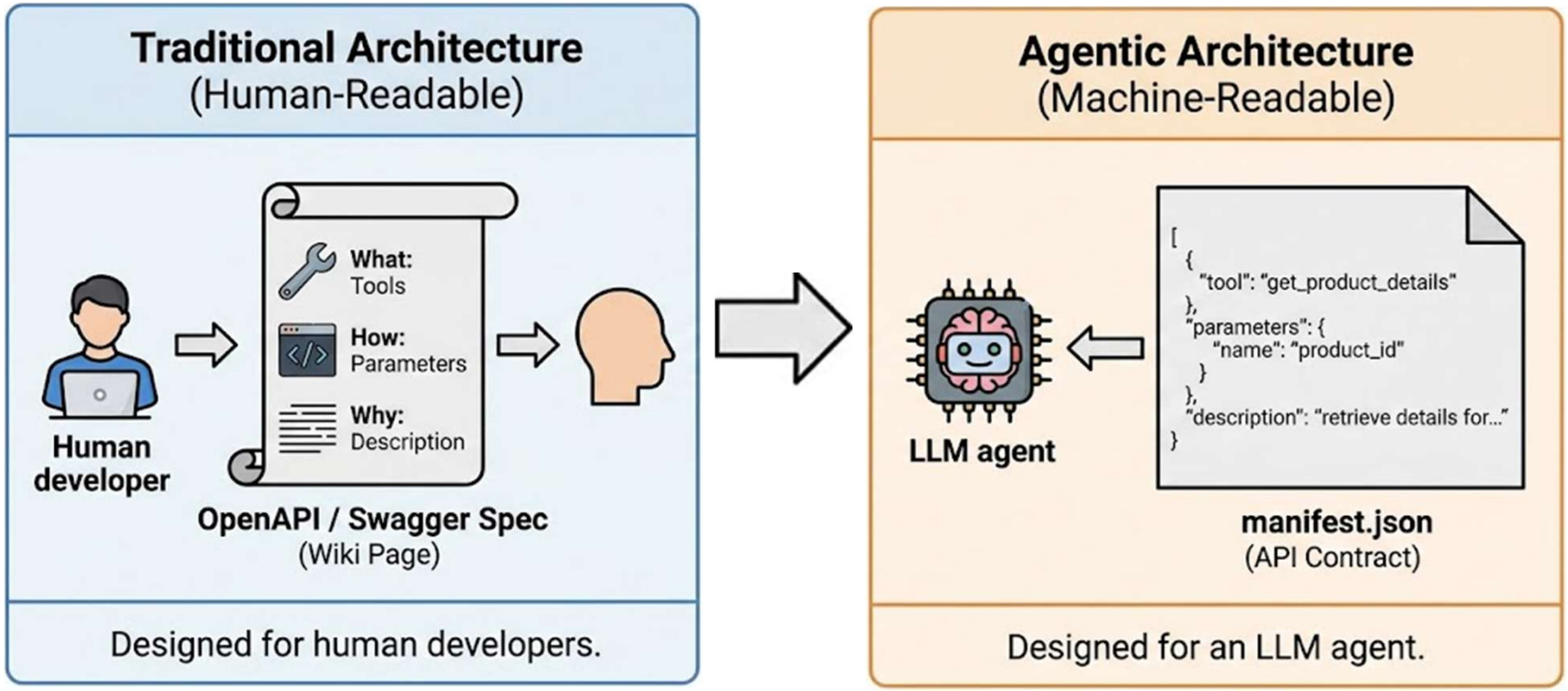
This lecture is about the **'DevOps for Tools'**





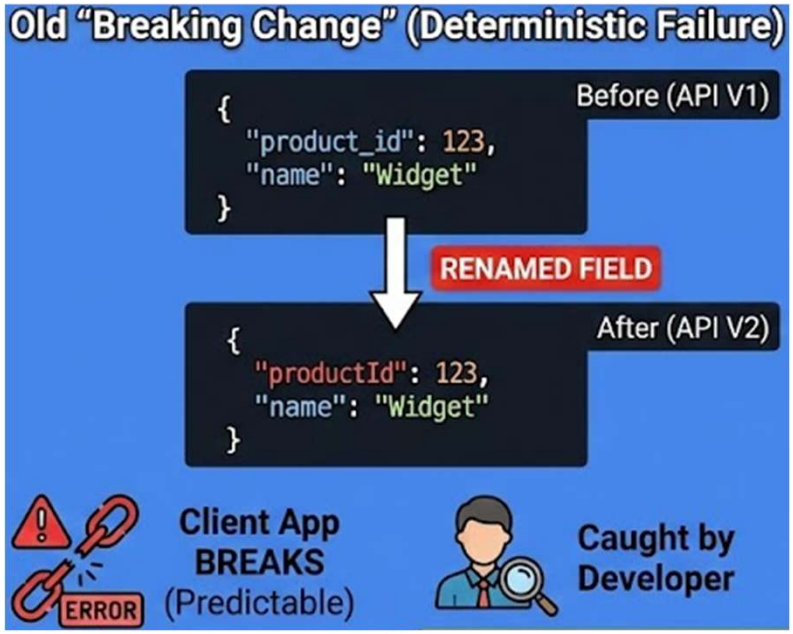


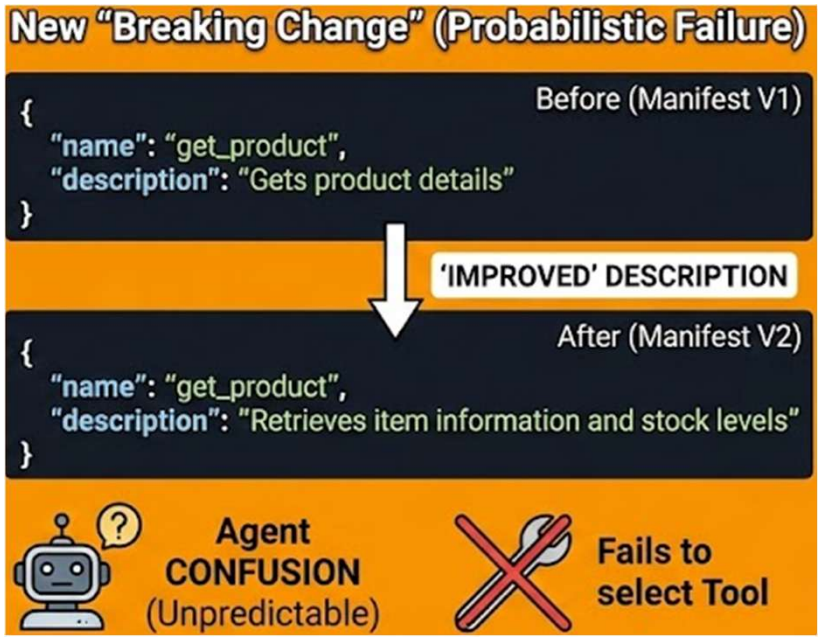
The manifest.json as an "API Contract"



This **shift** from a **human-readable** to a **machine-readable** contract has a **critical**, and **dangerous**, new **implication**

The New "Breaking Change" for Agents

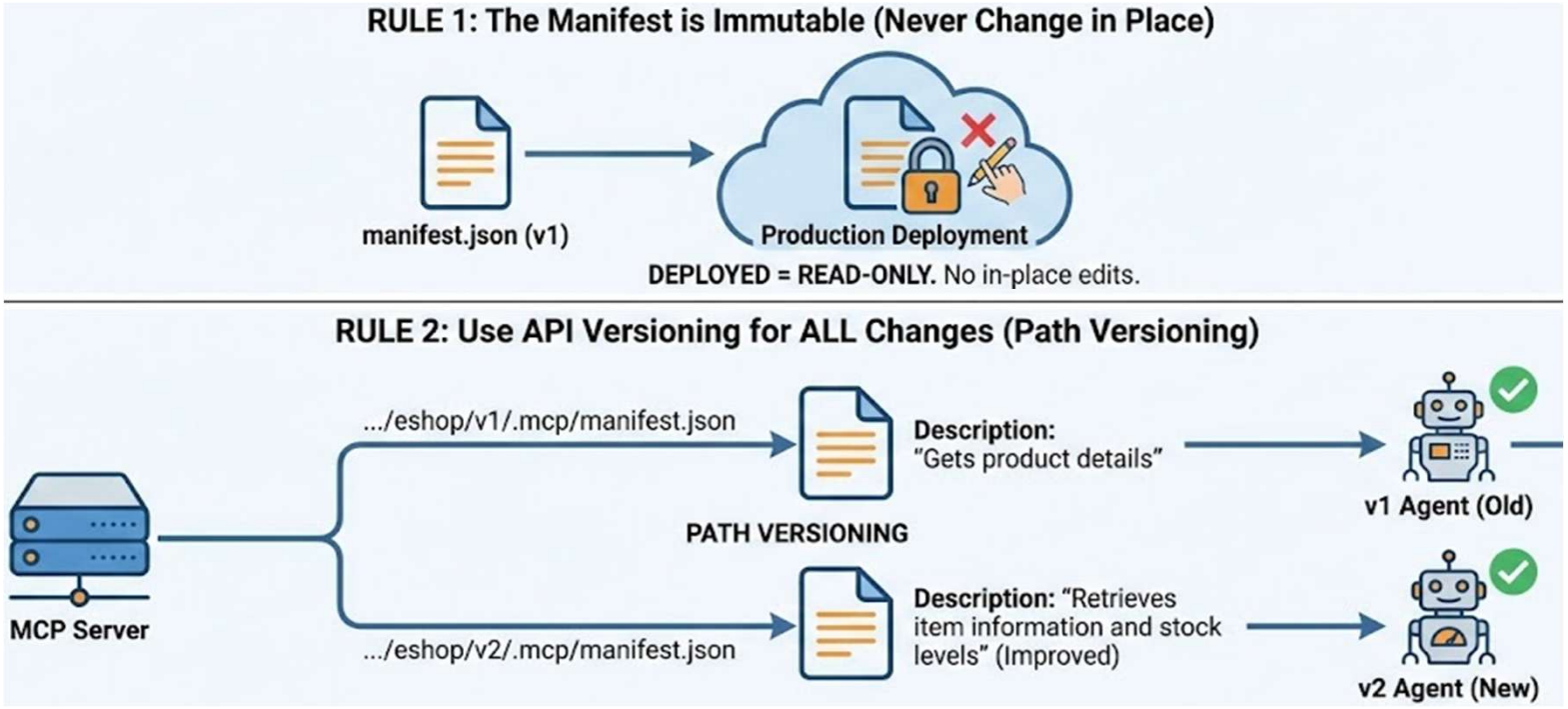




The Problem: The **agent**, which was prompted and trained to understand the old description, may now get **"confused"** It **probabilistically fail** to choose this tool

Key Insight: A simple, well-intentioned text change in your **manifest.json** is now a **P0**, **production-breaking bug**

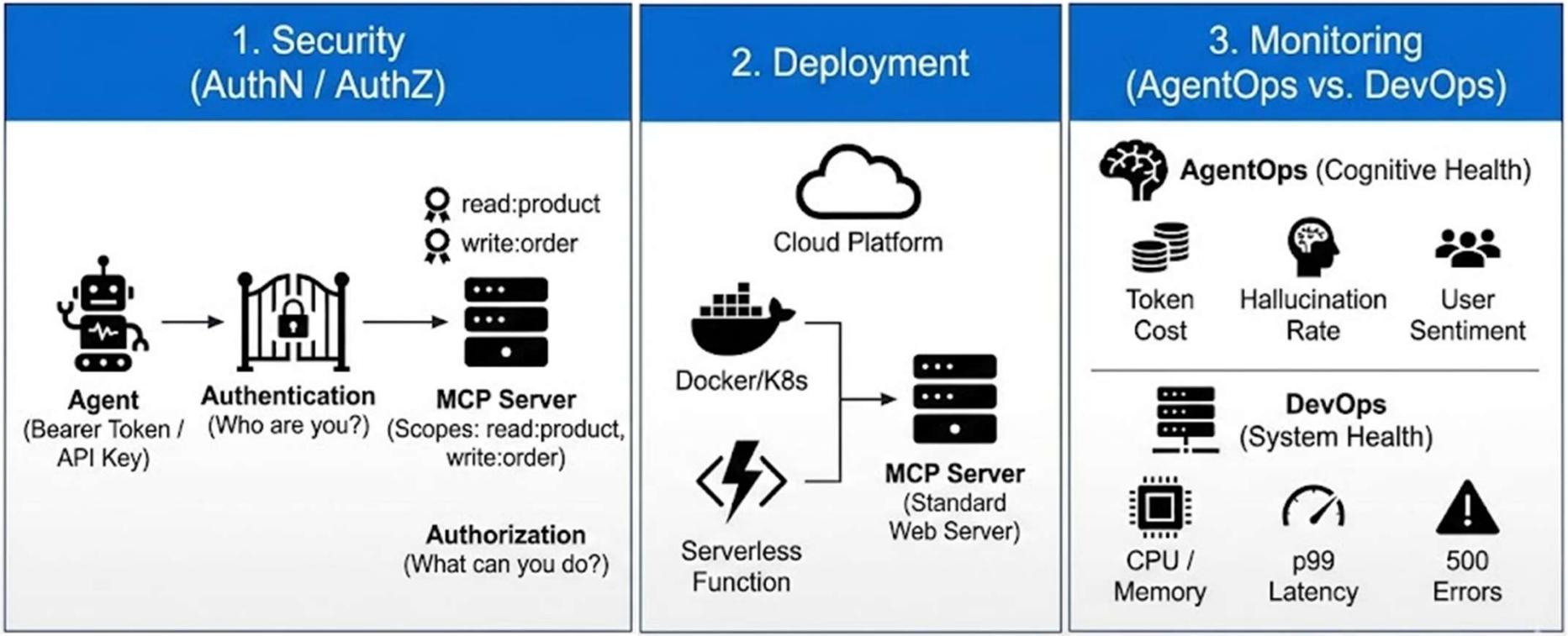
Best Practice 1: Strict Versioning (The Solution)



This allows **v1 agents** to **continue working**, while **new v2 agents** can be **built** to use the **new**, **"improved" tool**

This is the **"DevOps 101" discipline** we must apply

Best Practice 2: Security & Deployment



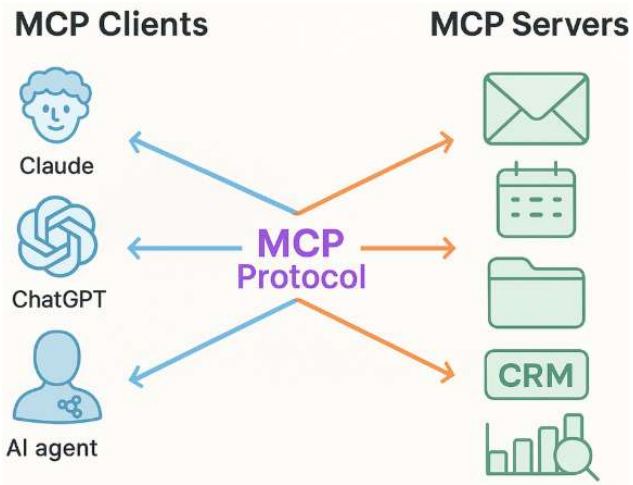
Summary: The Tool Lifecycle

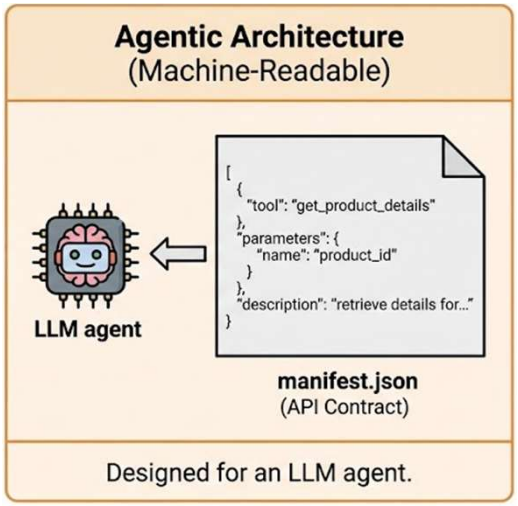
MCP Servers are **enterprise APIs** that serve AI. They are **not AI themselves**

The **manifest.json** is a **fragile**, **breaking contract**

Strict Versioning is the **non-negotiable solution**. Never change a manifest

Secure and **monitor MCP servers** just like any other **mission-critical microservice**







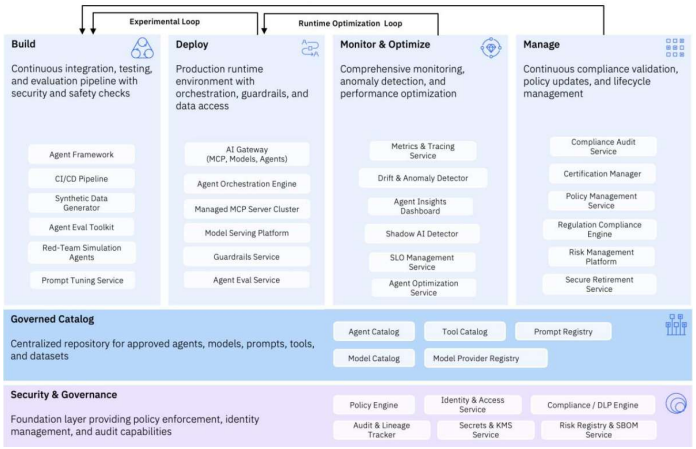
Reference Architecture for an Agentic AI Platform

Assembling the **components** of a **professional**, **enterprise-grade agentic** system

Defined the **7-step ADLC**
Done deep dives into the hardest parts: **Evaluation** and **AgentOps**
Defined our **Security Guardrails** and the **Lifecycle** of our **MCP Tool** Servers

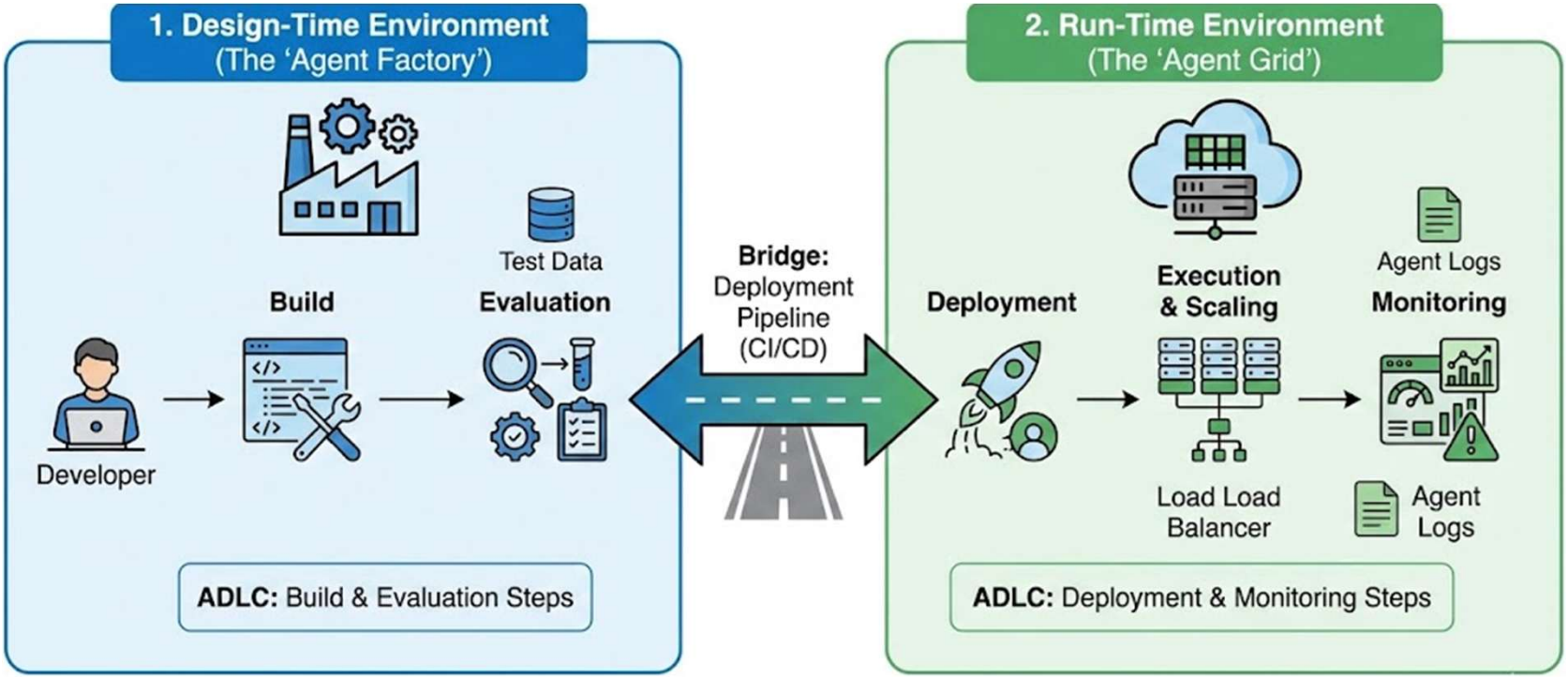
Combine all of these pieces into a **single**, **cohesive**, **vendor-agnostic Reference Architecture**

This is the **blueprint** for a **true**, **enterprise-wide Agentic AI Platform**



The 30,000-Foot View: Design-Time vs. Run-Time

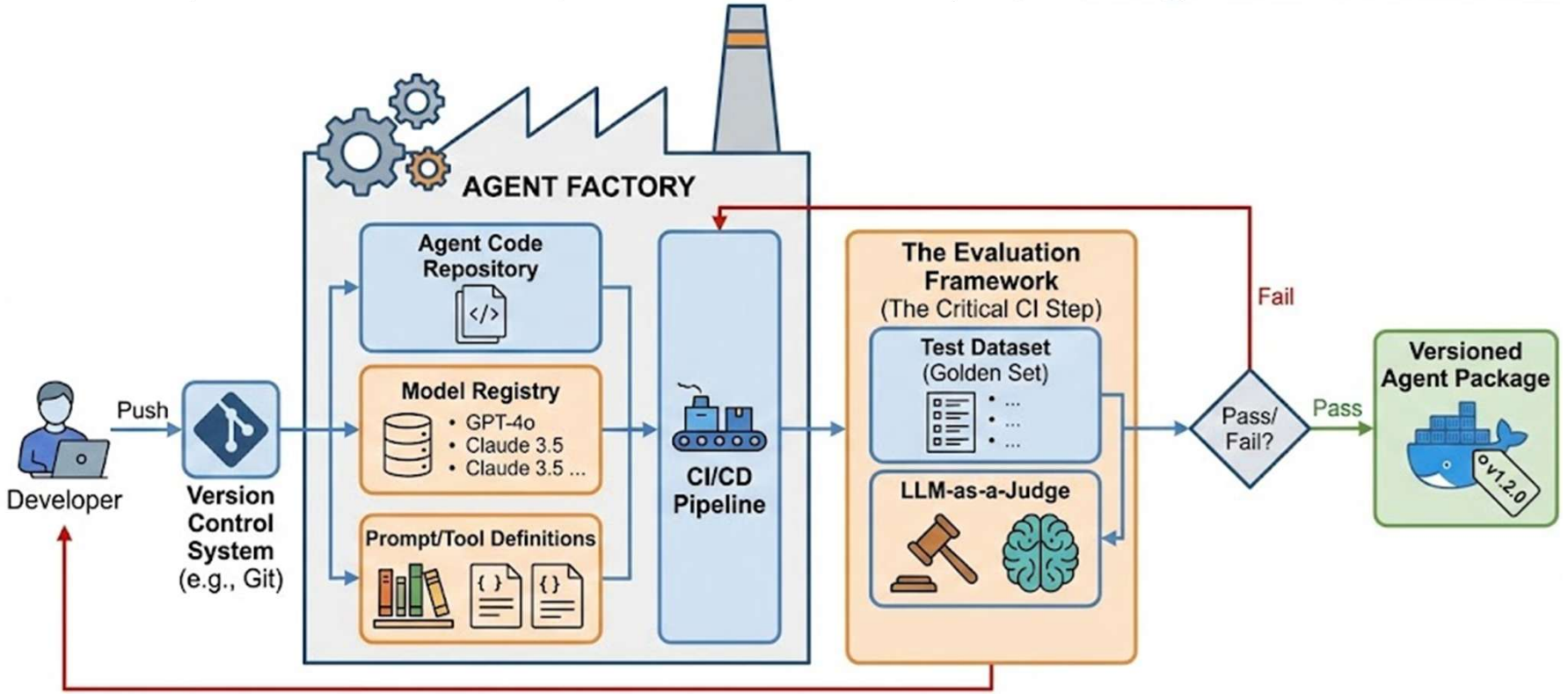
Let's establish the **2 "zones"** of our platform



Our **blueprint** must **clearly define both** and the **bridge between them**

Component 1: The "Agent Factory" (Design-Time)

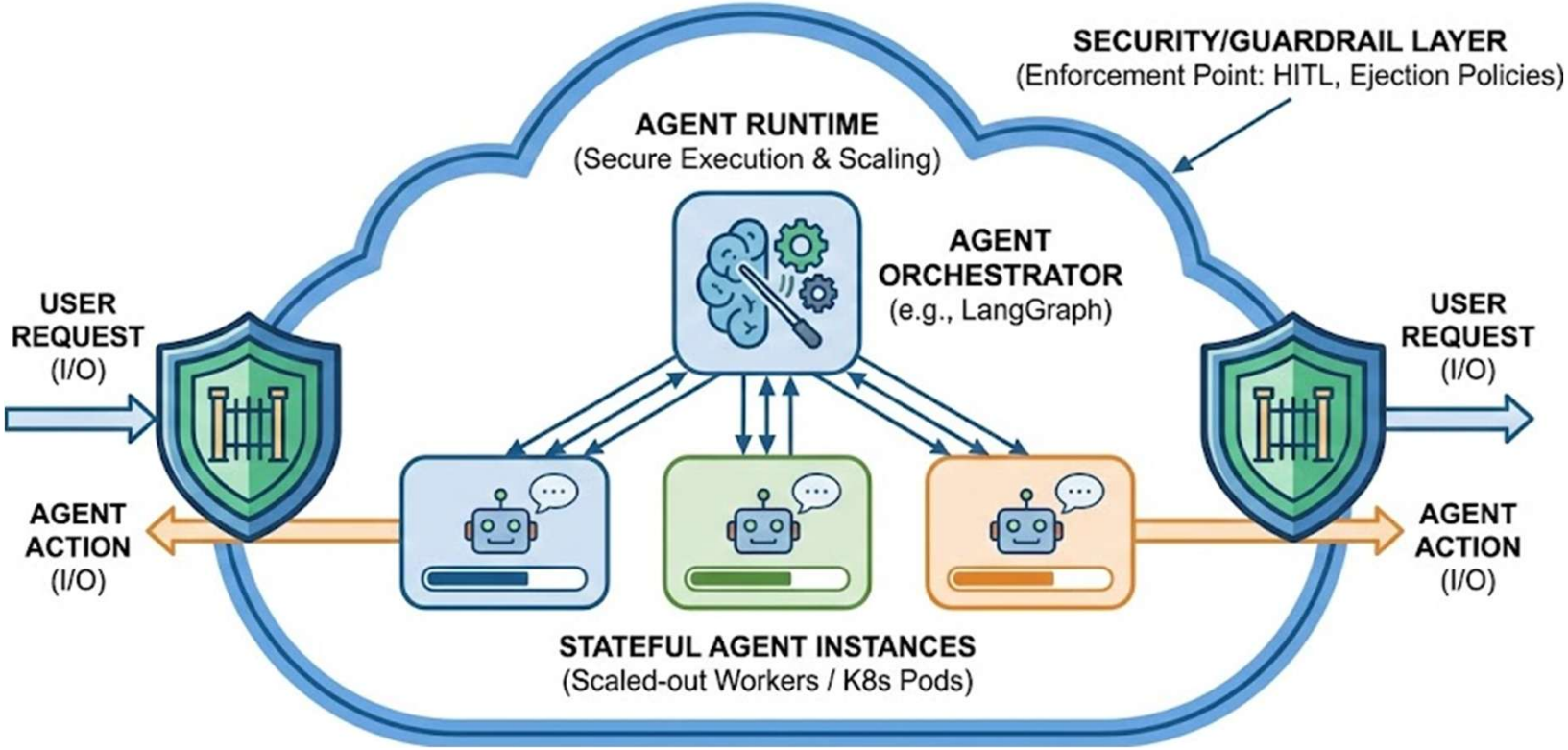
Goal: To produce validated, versioned, and deployable agents



Output: A "**Versioned Agent Package**" (a Docker container)

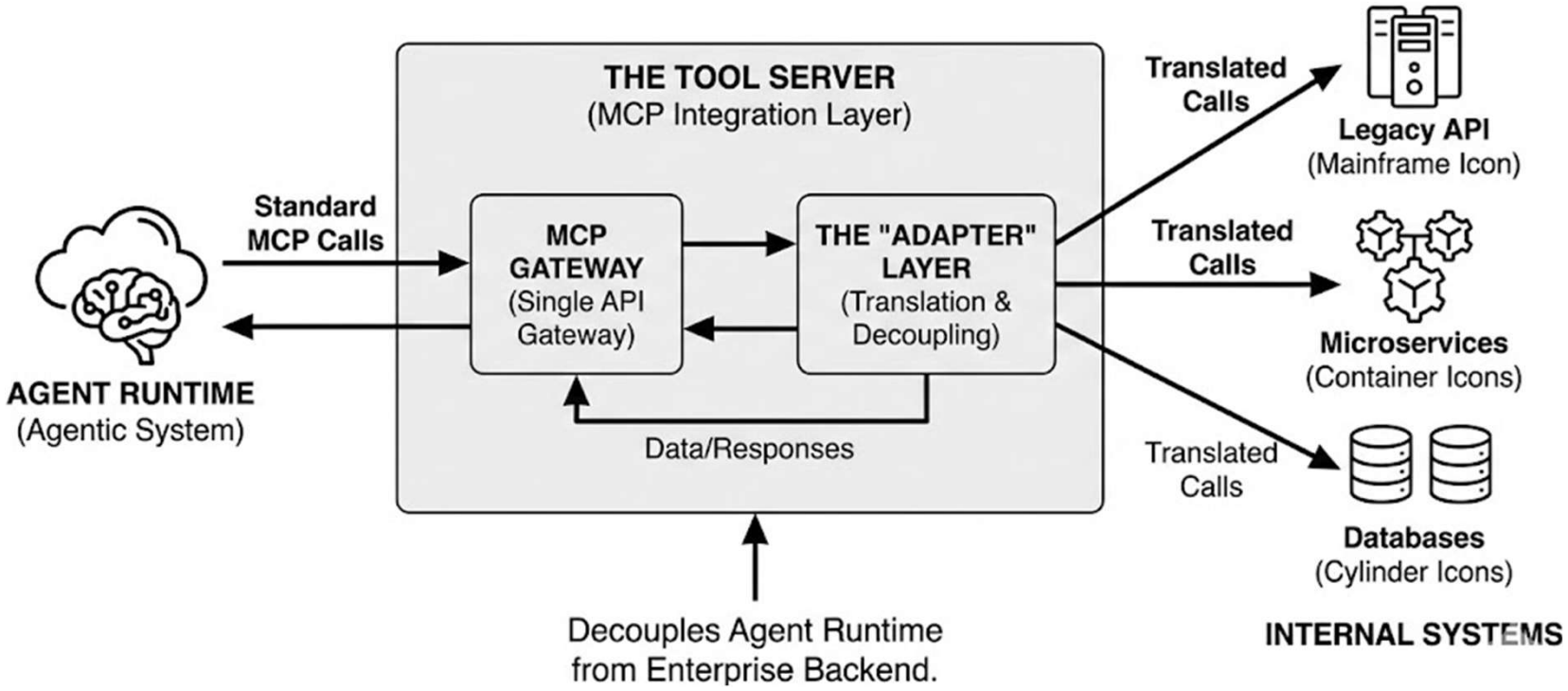
Component 2: The "Agent Runtime" (Run-Time)

Goal: To produce validated, versioned and deployable agents (K8s for Agents)



Component 3: The "Tool Server" (Integration Layer)

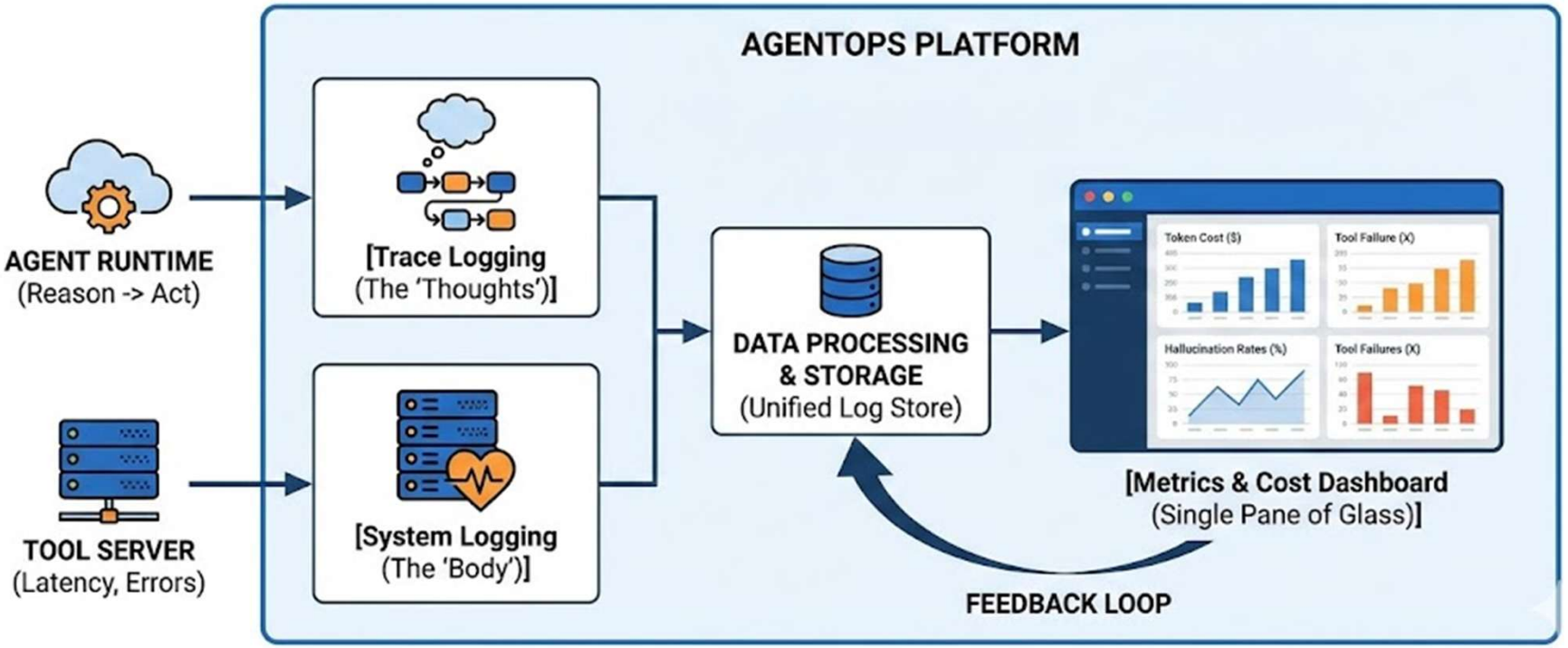
Goal: To provide a single, secure and stable bridge to enterprise systems



This **component decouples** the **"Agent Runtime"** from the **"Enterprise Backend"**

Component 4: The "AgentOps Platform" (Monitoring)

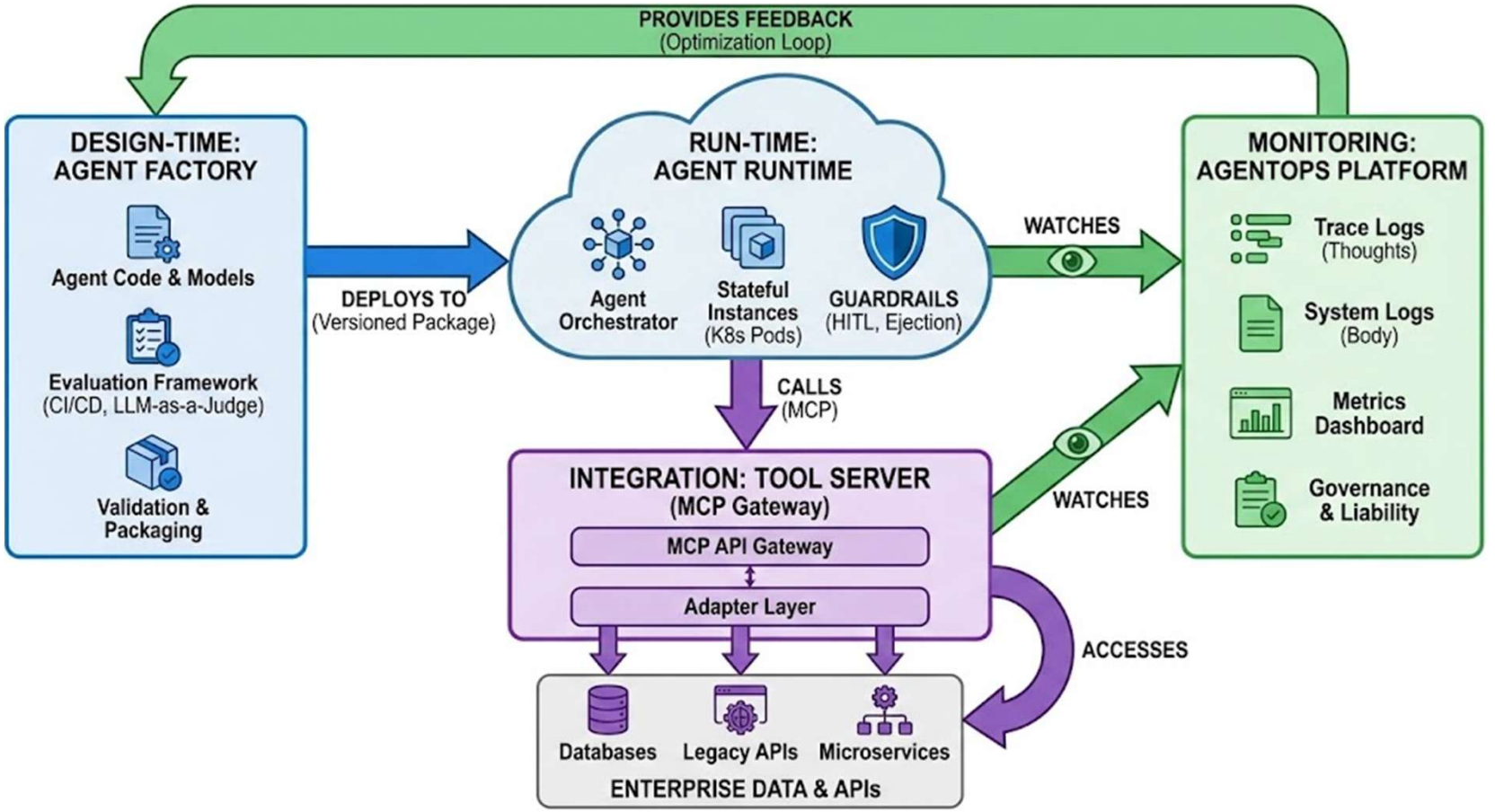
Goal: To provide holistic observability across the entire platform



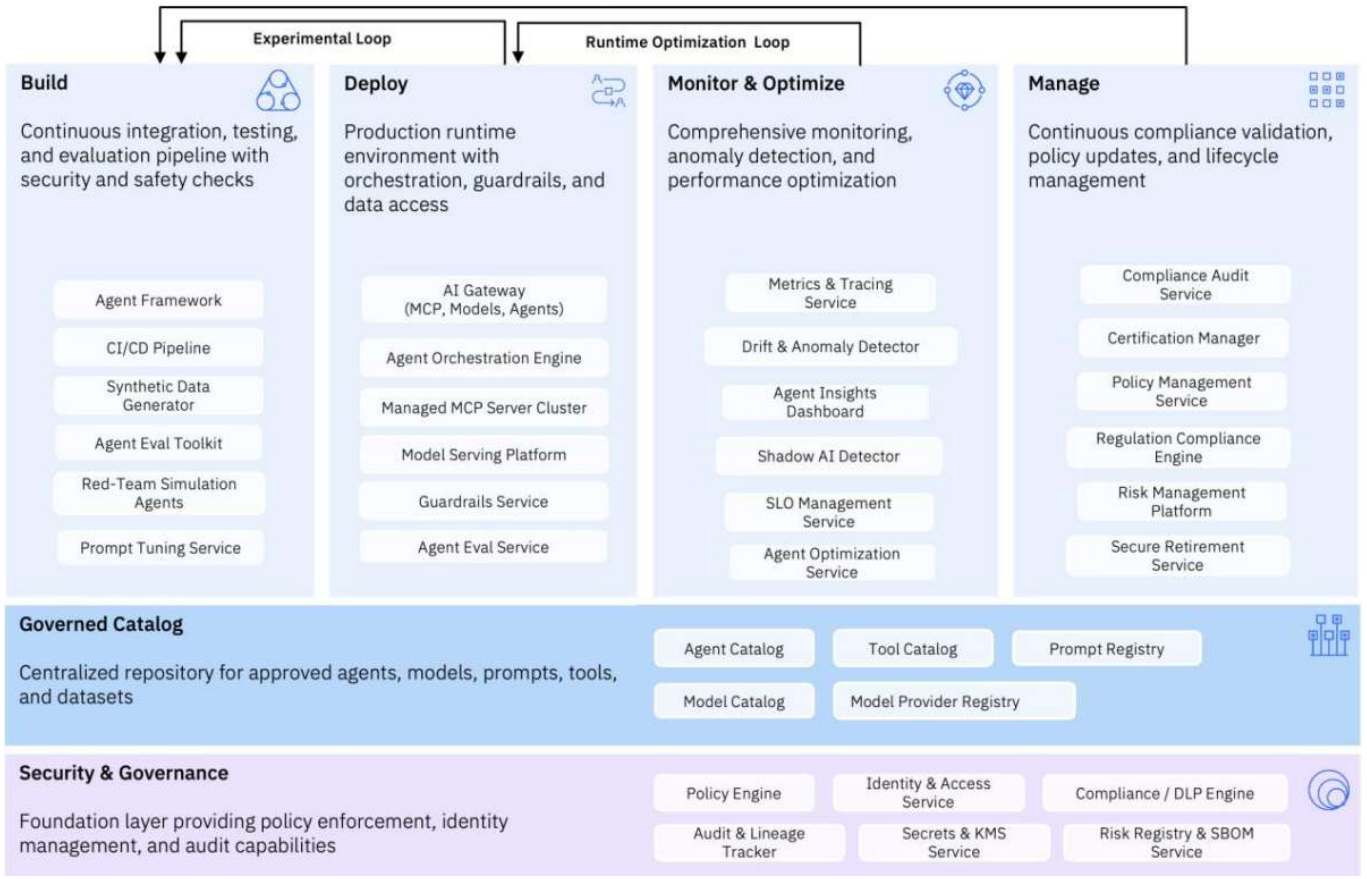
This **platform** is the "**feedback loop**" for the **entire system**

The Complete Reference Architecture (The Blueprint)

A "whiteboard" diagram showing all 4 components connected



The Complete Reference Architecture (The Blueprint)

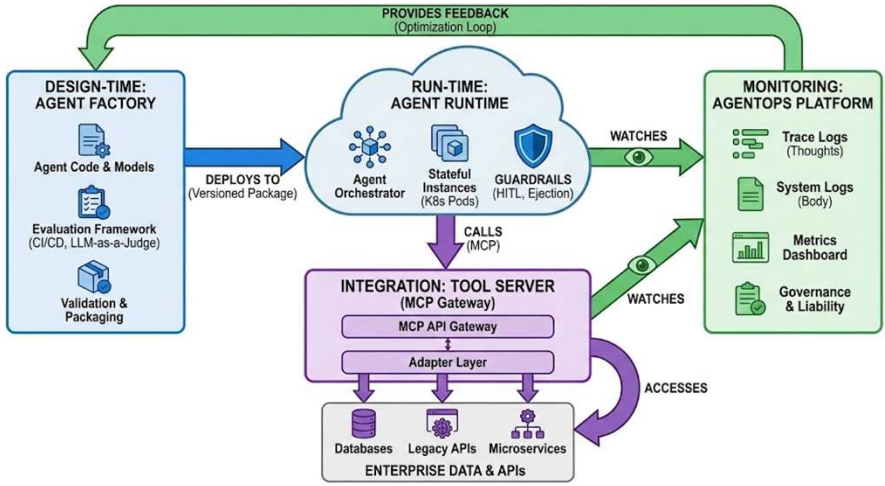


https://www.ibm.com/downloads/documents/us-en/1443d5dd174f42e6

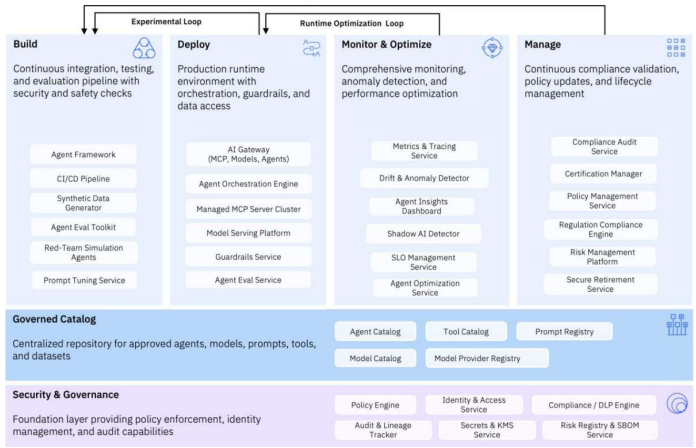
Summary: The Four Pillars of the Platform

Our enterprise blueprint is built on **4 pillars:**

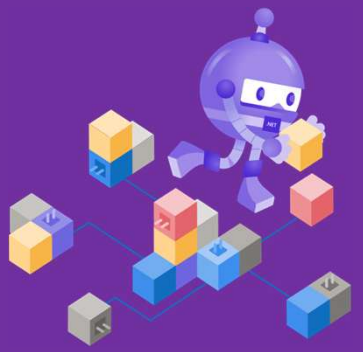
1. **The "Factory":** Design, Build, Evaluate
2. **The "Runtime":** Execute, Scale, Secure
3. **The "Gateway":** Integrate, Decouple, Manage
4. **The "Monitor":** Observe, Trace, Govern



With this **blueprint**, we can **analyze** any **vendor's product** or **build our own**



Thanks



Thank you so much for being with me on this journey.

Reviews and feedback is really encourage to me for pushing forw
create new courses like this.



Pradeep Kumar T